



**UNIVERSIDAD
DE GRANADA**

**TRABAJO FIN DE GRADO
INGENIERÍA INFORMÁTICA**

Análisis Comparativo de Rendimiento entre ECC y RSA en Entornos Deterministas

**Un estudio sobre la eficiencia algorítmica en Criptografía de
Curvas Elípticas frente a esquemas de factorización entera
con una implementación de alta precisión con NTL**

Autor

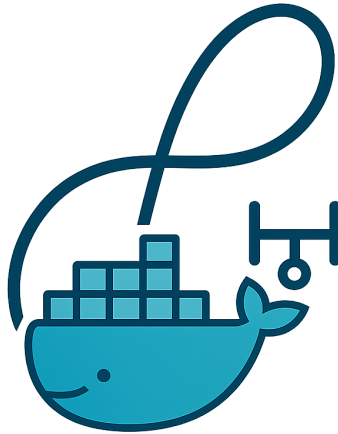
Leon Elliott Fuller

Directores

Jesús García Miranda



**ESCUELA TÉCNICA SUPERIOR DE INGENIERÍAS INFORMÁTICA Y DE
TELECOMUNICACIÓN
Granada, Abril de 2025**



Análisis Comparativo de Rendimiento entre ECC y RSA en Entornos Deterministas

Un estudio en detalle sobre la eficiencia algorítmica en
Criptografía de Curvas Elípticas frente a esquemas de
factorización entera con una implementación de alta
precisión con NTL.

Autor

Leon Elliott Fuller

Directores

Jesús García Miranda

Análisis Comparativo de Rendimiento entre ECC y RSA en Entornos Deterministas

Leon Elliott Fuller

Palabras clave: curvas elípticas, criptografía, aritmética de campo finito, NTL, C++, benchmarking, Docker, ECDH, ECDSA, ECC, RSA

Resumen

En 1985 Neal Koblitz y Victor Miller propusieron de forma independiente el uso de curvas elípticas en criptografía. Hoy en día, los esquemas basados en curvas elípticas (ECC) se han consolidado como una alternativa eficiente a RSA, pues ofrecen niveles de seguridad comparables con tamaños de clave sensiblemente menores. Esto los hace ideales para dispositivos con recursos limitados y para aplicaciones que requieren alto rendimiento.

En este trabajo profundizaremos en los fundamentos de ECC y describiremos los principales protocolos Elliptic Curve Diffie–Hellman (ECDH) y Elliptic Curve Digital Signature Algorithm (ECDSA). A continuación analizaremos tanto los ataques clásicos al problema del logaritmo discreto en curvas elípticas como las amenazas cuánticas, en particular el algoritmo de Shor, que podría comprometer ECC en la era de los ordenadores cuánticos.

Por último, se realizarán comparaciones de tiempo con respecto a la generación de claves entre RSA y ECC.

Comparative Performance Analysis of ECC and RSA Primitives in Deterministic Environments

Leon Elliott Fuller

Keywords: elliptic curves, cryptography, quantum computing, ECDH, ECDSA, ECC, RSA

Abstract

In 1985 Neal Koblitz and Victor Miller independently proposed the use of elliptic curves in cryptography. Today, elliptic curve schemes (ECC) have become a highly efficient alternative to RSA, offering comparable security with significantly smaller key sizes. This makes them ideal for resource-constrained devices and high-performance applications.

In this work we dive into the fundamentals of ECC and describe the main protocols used, such as Elliptic Curve Diffie–Hellman (ECDH) and Elliptic Curve Digital Signature Algorithm (ECDSA). We then analyze both classical attacks on the elliptic curve discrete logarithm problem and quantum threats, in particular Shor’s algorithm, which could compromise ECC in the era of quantum computers.

Finally, we present timing comparisons for key generation between RSA and ECC.

Yo, **Leon Elliott Fuller**, alumno de la titulación Ingeniería Informática de la **Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación de la Universidad de Granada**, con NIE ..., autorizo la ubicación de la siguiente copia de mi Trabajo Fin de Grado en la biblioteca del centro para que pueda ser consultada por las personas que lo deseen. Todo el código fuente así como este documento en formato LaTeX se puede encontrar en el siguiente repositorio de GitHub: <https://github.com/leonfullxr/ECC-Thesis>

Fdo: Leon Elliott Fuller

Granada a X de mayo de 2025.

D. **Jesús García Miranda**, Profesor del Área del Grado en Ingeniería Informática del Departamento de Álgebra de la Universidad de Granada.

Informa:

Que el presente trabajo, titulado *Análisis Comparativo de Rendimiento entre ECC y RSA en Entornos Deterministas*, ha sido realizado bajo su supervisión por **Leon Elliott Fuller**, y autorizamos la defensa de dicho trabajo ante el tribunal que corresponda.

Y para que conste, expiden y firman el presente informe en Granada a X de mayo de 2025.

El tutor:

Jesús García Miranda

Agradecimientos

Antes que nada, quiero expresar mi más profundo agradecimiento a mis queridos padres, Allan y Milena, quienes han sido los pilares de apoyo en todo este recorrido. Sin su amor incondicional, su confianza y su esfuerzo diario, jamás habría llegado hasta aquí. Les estaré eternamente agradecido por regalarme la oportunidad de soñar en grande y de perseguir mis metas.

A continuación, quiero agradecer a mis otros dos pilares, Pablo Balbuena Fernández e Iñigo Zapirain Laboa, dos personas que me habéis apoyado y empujado siempre a por más, a por lo imposible. Me habéis enseñado e inculcado valores y disciplinas que nunca hubiera descubierto por mi cuenta. Gracias por haberme llevado al límite y por mantenerme con los pies en la tierra cuando el camino se ponía cuesta arriba (y menudas cuestas hemos tenido que subir).

Quiero dar también, un fiel agradecimiento a mis compañeros de clase, concretamente a Toni de Potocolom, quien ha sido el fruto de mi camino espiritual e intelectual. Gracias a ti he podido abarcar mucho conocimiento y me has enseñado a mirar el mundo con nuevos ojos. Gracias también a Karim por haberme aguantado todos estos años de carrera, la primera persona con la que tuve contacto nada más pisar la ETSIIT.

Por último, me gustaría agradecer a mi tutor Jesús García Miranda, una persona más que un simple tutor o profesor. Eres una persona con un gran corazón y todos los alumnos que han podido tener el placer de haberte tenido de profesor o haber coincidido contigo, estoy seguro de que le has dejado un recuerdo bonito. Gracias por haberme apoyado y por haberme ayudado en la elaboración de este gran proyecto final que depara mi etapa universitaria.

A todos vosotros: gracias de verdad, de corazón, por formar parte de esta historia.

Índice general

Índice de figuras	v
Índice de cuadros	vii
I Introducción	1
1. Introducción	3
1.1. Contexto histórico y motivación	3
1.2. Objetivos	4
1.3. Planificación	4
1.4. Estructura	4
II Conocimientos Básicos	7
2. Introducción a la criptografía	9
2.1. Estructura de un criptosistema	10
2.2. Criptografía sin clave o Funciones hash	11
2.2.1. MD5	12
2.2.2. SHA-1	12
2.3. Criptografía de clave privada o simétrica	13
2.3.1. Cifrado de bloque	14
2.3.2. Cifrado de flujo	18
2.4. Criptografía de clave pública o asimétrica	19
2.4.1. Intercambio de claves Diffie–Hellman	20
2.4.2. Firma digital	22
2.5. Resumen comparativo	22
2.6. Principios de diseño criptográfico	22
2.7. Seguridad y modelos de ataque	23
2.7.1. Ataques pasivos y activos	23
2.7.2. Modelos clásicos de criptoanálisis	23
2.7.3. Ataques avanzados	24

3. Cuerpos finitos	25
3.1. Introducción a cuerpos finitos	26
3.1.1. Representación binaria y suma de enteros	27
3.2. Aritmética en $\mathbb{F}_p$	28
3.2.1. Suma en $\mathbb{F}_p$	29
3.2.2. Multiplicación de enteros	29
3.3. Aritmética en $\mathbb{F}_{2^m}$	30
3.3.1. Representación polinomial	30
3.3.2. Suma en $\mathbb{F}_{2^m}$	31
3.3.3. Multiplicación en $\mathbb{F}_{2^m}$	32
3.4. Aritmética en cuerpos de extensión óptima (OEF)	33
3.5. Selección de cuerpos para ECC: comparativa y recomendaciones	35
 III Marco Teórico	 37
4. Teoría de las curvas elípticas	39
4.1. Introducción y motivación	39
4.1.1. Historia	40
4.2. Definición de una curva elíptica	41
4.2.1. Comentarios sobre la definición 4.1	42
4.2.2. Ecuaciones simplificadas de Weierstrass	44
4.2.3. Ley de Grupo	48
4.3. Curvas sobre cuerpos finitos	51
4.4. Representación de puntos y la ley de grupo	54
4.4.1. Coordenadas proyectivas: definición general	54
4.4.2. Coordenadas en $\mathbb{F}_p$	55
4.4.3. Coordenadas en $\mathbb{F}_{2^m}$	55
4.4.4. Ventajas y desventajas	57
4.5. Selección de parámetros	57
 5. Criptografía de curvas elípticas (ECC)	 59
5.1. Introducción y motivación	59
5.1.1. Historia	59
5.1.2. Aplicaciones	61
5.2. Fundamentos de ECC	62
5.2.1. El problema del logaritmo discreto en curvas elípticas (ECDLP)	62
5.2.2. Comparativa de tamaños de clave	71
5.2.3. Parámetros del dominio de una curva elíptica	72
5.2.4. Generación y verificación de parámetros de dominio .	73
5.3. Generación de claves	75
5.4. Intercambio de claves: ECDH	76
5.4.1. Protocolo básico	76

5.4.2.	Derivación de claves	77
5.4.3.	Variantes y extensiones	78
5.5.	Firmas digitales: ECDSA	78
5.5.1.	Función hash: SHA-256	78
5.5.2.	Generación de firma	79
5.5.3.	Verificación de firma	79
5.5.4.	Problemas prácticos y vulnerabilidades	80
5.6.	Aspectos de implementación	81
5.6.1.	Coordenadas afines frente a Jacobianas	81
5.6.2.	Curvas sobre cuerpos binarios $\mathbb{F}_{2^m}$	82
5.6.3.	Multiplicación escalar	83
5.7.	Selección de parámetros y curvas estandarizadas	84
5.7.1.	Criterios de selección	84
5.7.2.	Curvas estandarizadas	85
5.7.3.	Controversias y confianza	85
IV	Implementación y Experimentación	87
6.	Implementación y entorno experimental	89
6.1.	Visión general	89
6.2.	Arquitectura del software	89
6.2.1.	Estructura modular	89
6.2.2.	Interfaz de línea de comandos	91
6.3.	Decisiones de diseño	92
6.3.1.	Progresión pedagógica: afines, después Jacobianas	92
6.3.2.	SHA-256 desde cero	92
6.3.3.	Uso de curvas estandarizadas	93
6.3.4.	Validación de parámetros del dominio	93
6.4.	Herramientas y entorno de ejecución	93
6.4.1.	Lenguaje y bibliotecas	93
6.4.2.	Contenedorización con Docker	94
6.5.	Generador de números aleatorios	95
6.5.1.	Diseño del RNG	95
6.5.2.	Validación estadística	95
6.6.	Motor de benchmarking	96
6.6.1.	Metodología de medición	96
6.6.2.	Operaciones medidas	96
6.6.3.	Pipeline de visualización	98
6.7.	Curvas implementadas	98
6.7.1.	Curvas sobre cuerpos primos $\mathbb{F}_p$	98
6.7.2.	Curvas sobre cuerpos binarios $\mathbb{F}_{2^m}$	99
6.8.	Limitaciones del prototipo	99

V	Conclusión	101
	Bibliografía	103

Índice de figuras

2.1.	Esquema de clasificación dentro de la Teoría de la Información	9
2.2.	Clasificación de la criptografía en función del método de cifrado	10
2.3.	Esquema de un criptosistema formal	11
2.4.	Ejemplo de cifrado y descifrado de bloque ECB	16
2.5.	Ejemplo de cifrado y descifrado de bloque CBC	18
2.6.	Ejemplo de cifrado asimétrico	20
3.1.	Representación de $a \in \mathbb{F}_{2^m}$ como un arreglo A de palabras de W bits. Los $s = Wt - m$ bits de orden más alto de $A[t - 1]$ permanecen sin usar.	30
4.1.	Esquema del Algoritmo de Firma Digital sobre Curvas Elípticas (ECDSA).	40
4.2.	Ejemplo de una curva con un punto singular (cúspide en el origen), correspondiente a $\Delta = 0$	43
4.3.	Curvas E_1 y E_2 sobre $\mathbb{R}$, con sus respectivos puntos (excepto $\mathcal{O}$).	44
4.4.	Geometría de la ley de grupo en curvas elípticas.	49
4.5.	Ejemplo de los puntos de una curva elíptica sobre un cuerpo finito	52
5.1.	Jerarquía de la criptografía de Curvas Elípticas	60
5.2.	Estructura ρ de la secuencia. Los primeros λ puntos (azul) forman la cola. A partir de x_λ la secuencia entra en un ciclo de longitud μ (rojo), cumpliéndose $x_{\lambda+\mu} = x_\lambda$	67
5.3.	Visualización del rho de Pollard sobre la curva $y^2 = x^3 + 2x + 1 \pmod{89}$. Panel izquierdo: trayectoria pseudoaleatoria de la tortuga sobre la curva, coloreada por orden de aparición; la estrella marca la colisión en el paso 13. Panel derecho: estructura abstracta de ρ mostrando la cola ($\lambda = 2$, azul) y el ciclo ($\mu = 13$, rojo).	71

- 6.1. Diagrama de dependencias entre los módulos del proyecto. `main.cpp` actúa como punto de entrada y orquesta los módulos criptográficos (RSA, ECC sobre $\mathbb{F}_p$ y $\mathbb{F}_{2^m}$, SHA-256), todos los cuales se apoyan en el generador de números aleatorios `rng` y los tipos comunes definidos en `common.hpp`. 90

Índice de cuadros

2.1. Resumen del intercambio de claves Diffie–Hellman	21
3.1. Parámetros de ejemplo para OEF. Aquí $p = 2^n - c$ es primo y $f(z) = z^m - \omega$ es irreducible en $\mathbb{F}_p[z]$	34
3.2. Órdenes de magnitud orientativos de los costes en una CPU de 3 GHz para cuerpos de 256 bits, según la literatura. Los valores experimentales medidos sobre la implementación de este trabajo se presentan en el capítulo 6.	36
5.1. Probabilidad de que al menos dos personas compartan cum- pleaños en un grupo de k personas ($N = 365$ días).	65
5.2. Número esperado de pasos del rho de Pollard según el tamaño de la curva. Los bits de seguridad son exactamente la mitad de los bits de la curva.	66
5.3. Equivalencia de niveles de seguridad entre RSA y ECC según NIST SP 800-57.	71
5.4. Resumen del intercambio de claves ECDH.	77
5.5. Curvas sobre cuerpos binarios implementadas (SEC 2).	83
6.1. Modos de ejecución del motor de benchmarking.	91
6.2. Herramientas y bibliotecas del proyecto.	94
6.3. Operaciones criptográficas medidas en el benchmark.	97
6.4. Curvas sobre cuerpos primos implementadas.	98
6.5. Curvas sobre cuerpos binarios implementadas (SEC 2).	99

Parte I

Introducción

Capítulo 1

Introducción

1.1. Contexto histórico y motivación

La comunicación confidencial entre sujetos ha sido una necesidad constante a lo largo de la historia. Cuando los mensajes debían transmitirse de forma no presencial o transportarse físicamente, surgía el reto de asegurar que ninguna entidad externa pudiera leerlos o alterarlos sin ser detectada.

Ya en la Antigüedad, civilizaciones como la espartana emplearon la “Escítala” en el siglo V a. C., un dispositivo de transposición que cifraba mensajes al enrollarlos en un cilindro. Más tarde, los griegos documentaron con Polibio un sistema de sustitución basado en el orden de las letras, y Roma adoptó el famoso Cifrado César, que desplazaba el alfabeto para ocultar información militar y diplomática.

Durante el Renacimiento, el cifrado avanzó hacia métodos más sofisticados, como el de Leon Battista Alberti que introdujo en 1465 la idea del cifrado polialfabético mediante discos concéntricos, lo que supuso un gran avance en esa época. Blaise de Vigenère perfeccionó esta técnica en el siglo XVI con una tabla que combinaba alfabetos de forma periódica, y en Europa se publicaron tratados (como el *Cryptomenytices et Cryptographiae* de Selenus) que difundieron estos métodos entre cortes y ejércitos.

El siglo XX presenció el mayor salto tecnológico, donde se partió de los cifrados manuales de la Primera Guerra Mundial hasta la aparición de máquinas electromecánicas como la Enigma alemana en la Segunda Guerra Mundial, una máquina de rotores que automatizaba los cálculos que era necesario realizar para las operaciones de cifrado y descifrado de mensajes. El descifrado de Enigma por los criptógrafos aliados no solo cambió el curso del conflicto, sino que marcó el inicio de la era del cálculo automatizado al servicio de la criptografía. Tras la guerra, Claude Shannon sentó las bases teóricas de la información y la seguridad, y en los años 70 el NIST publicó el Estándar de Cifrado de Datos (DES), consolidando el cifrado simétrico en la informática moderna.

Sin embargo, todos los criptosistemas anteriores tenían un inconveniente, y es que todos se basaban en el hecho de que tanto la parte que descifraba como la que codificaba debían conocer el método de cifrado y la clave para descifrarlo. ¿Pero cómo se transmite la clave de forma segura? Por supuesto, con cifrado. Pero entonces, ¿cómo se puede enviar el cifrado, para que la entidad que descifra el mensaje pueda conocer dicho cifrado? Como se puede apreciar, dicho problema de la criptografía es un bucle infinito entre el cifrado y la distribución de claves que no se puede resolver con seguridad garantizada para todos los lectores deseados.

Hoy día, RSA y sistemas análogos sustentan protocolos críticos como HTTPS, correo firmado, servicios bancarios, etc. Pero presentan dos limitaciones crecientes:

- **Tamaño de clave versus seguridad.** Para alcanzar un nivel de protección comparable al de cifrados simétricos de 128 bits, RSA requiere claves de varios miles de bits, lo que penaliza el rendimiento en cómputo y ancho de banda.
- **Vulnerabilidad cuántica.** Los algoritmos cuánticos plantean un desafío fundamental para la criptografía moderna. En particular, Shor demostró en 1994 que un ordenador cuántico suficientemente grande podría factorizar enteros y resolver problemas de logaritmo discreto en tiempo polinómico, lo que rompería sistemas como RSA, Diffie-Hellman e incluso ECC [9][8]. Esta perspectiva ha impulsado la búsqueda de algoritmos resistentes a estos ataques cuánticos, así como la adopción de curvas elípticas de mayor seguridad y longitudes de clave incrementadas.

Estas limitaciones han impulsado la adopción de la *Criptografía de Curvas Elípticas (ECC)*. Gracias a la dificultad del problema del logaritmo discreto en curvas elípticas, ECC ofrece un nivel de seguridad equivalente con claves drásticamente más pequeñas (por ejemplo, 256 bits en ECC frente a 3072 bits en RSA), mejorando así el rendimiento y la eficiencia. Además, la resistencia actual de ECC frente a los ataques cuánticos es mayor que la de RSA bajo el mismo modelo cuántico, lo que la posiciona como la opción de próxima generación para sistemas criptográficos modernos.

1.2. Objetivos

1.3. Planificación

1.4. Estructura

Antes de pasar a detalles más técnicos, me gustaría detallar el contenido de este proyecto:

1. **Capítulo 1: Introducción** Se encuentra una breve introducción a nuestro proyecto, así como un contexto histórico y las motivaciones que nos han llevado a realizarla.
2. **Capítulo 2: Introducción a la criptografía** Se presentan los conceptos esenciales, donde se definen las bases necesarias para comprender la seguridad de los sistemas actuales.
3. **Capítulo 3: Introducción a la computación cuántica** Define una base sólida sobre la computación cuántica, para próximamente, detallar los posibles ataques que se podrían realizar sobre las curvas elípticas.
4. **Capítulo 4: Teoría de las curvas elípticas** Se expone la teoría algebraica de las curvas elípticas sobre cuerpos finitos, la ley de grupo y las propiedades matemáticas clave. Incluye ejemplos de parametrización e implementaciones sencillas en código.
5. **Capítulo 5: Criptografía con curvas elípticas (ECC)** Describe los esquemas de clave pública basados en ECC: generación de claves, Elliptic Curve Diffie–Hellman (ECDH) y Elliptic Curve Digital Signature Algorithm (ECDSA). Se comparan su rendimiento y tamaño de clave frente a RSA.
6. **Capítulo 6: Ataques clásicos a ECC** Analiza el problema del logaritmo discreto en curvas elípticas, técnicas de criptoanálisis clásicas (baby-step/giant-step, Pollard’s rho) y su complejidad computacional en la práctica.
7. **Capítulo 7: Ataques cuánticos a ECC** Aquí se muestra el impacto del algoritmo de Shor y otros métodos cuánticos sobre ECC, estimando los recursos necesarios y las implicaciones para la seguridad futura. Se introduce la criptografía post-cuántica como posible resistencia.
8. **Capítulo 8: Implementación y experimentos** Detalla las herramientas y librerías empleadas, presenta fragmentos de código para ECC y simulaciones cuánticas, y muestra visualizaciones de curvas y resultados bajo diferentes parámetros.
9. **Capítulo 9: Resultados y pruebas** Recoge métricas de rendimiento, comparaciones entre RSA y ECC, junto con análisis de tiempos de cómputo.
10. **Capítulo 9: Conclusiones** Se pueden encontrar las conclusiones finales así como las recomendaciones para futuros trabajos.

Parte II

Conocimientos Básicos

Capítulo 2

Introducción a la criptografía

La criptografía es el arte de encubrir los mensajes con signos convencionales, que sólo pueden cobrar algún sentido a través de una clave secreta que nace en conjunto con la escritura.

En su clasificación dentro de las ciencias, la criptografía proviene de una rama de las matemáticas, que fue iniciada por el matemático Claude Elwood Shannon en 1948, denominada: "Teoría de la Información". Esta rama de las ciencias se divide en: "Teoría de Códigos" y "Criptología". Y a su vez la Criptología se divide en Criptografía y Criptoanálisis, como se muestra en la siguiente figura 2.1:

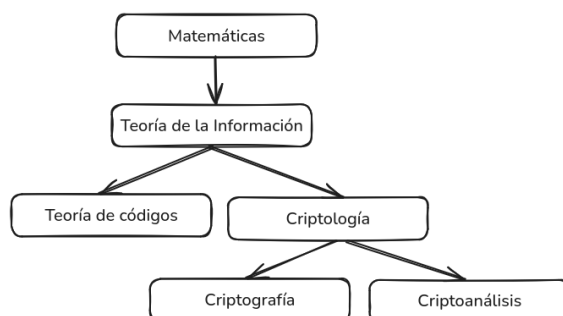


Figura 2.1: Esquema de clasificación dentro de la Teoría de la Información

En un sentido más amplio, la Criptografía es la ciencia encargada de diseñar funciones o dispositivos, capaces de transformar mensajes legibles o en claro a mensajes cifrados de tal manera que esta transformación (cifrar) y su transformación inversa (descifrar) sólo pueden ser factibles con el conocimiento de una o más llaves. En contraparte, el criptoanálisis es la ciencia que estudia los métodos que se utilizan para, a partir de uno o varios mensajes cifrados, recuperar los mensajes en claro en ausencia de la(s) llave(s) y/o encontrar la llave o llaves con las que fueron cifrados dichos mensajes.

La criptografía se puede clasificar históricamente en dos:

- **Criptografía Clásica:** es aquella que se utilizó desde antes de la época actual hasta la mitad del siglo XX. También puede entenderse como la criptografía no computarizada o mejor dicho no digitalizada. Los métodos utilizados eran variados, algunos muy simples y otros muy complicados de criptoanalizar para su época.
 - **Sustitución:** Se reemplazan elementos del mensaje original por otros.
 - **Transposición:** Se reordenan los caracteres del mensaje siguiendo un patrón definido.
- **Criptografía Moderna:** se divide actualmente en cifrado sin clave (funciones hash), cifrado de clave privada (cifrado simétrico) y cifrado de clave pública (cifrado asimétrico). En el cifrado de clave privada las claves de cifrado y descifrado son la misma (o bien se deriva de forma directa una de la otra), debiendo mantenerse en secreto dicha clave para evitar el descifrado de un mensaje. Por el contrario, en el cifrado de clave pública las claves de cifrado y descifrado son independientes, no derivándose una de la otra, por lo cual puede hacerse pública la clave de cifrado siempre que se mantenga en secreto la clave de descifrado. Veremos algunos algoritmos de cifrado modernos

Tanto la criptografía clásica como la moderna se clasifican de acuerdo a las técnicas o métodos que se utilizan para cifrar los mensajes. Esta clasificación la podemos ver en la siguiente figura 2.2:

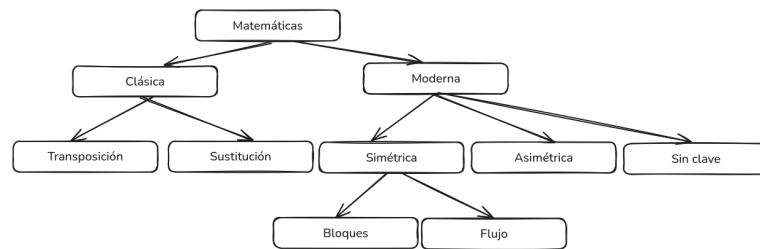


Figura 2.2: Clasificación de la criptografía en función del método de cifrado

2.1. Estructura de un criptosistema

Un criptosistema se define formalmente como la tupla

$$(\mathcal{M}, \mathcal{C}, \mathcal{K}, \mathcal{E}, \mathcal{D}),$$

donde

- $\mathcal{M}$ es el espacio de texto plano.

- $\mathcal{C}$ es el espacio de texto cifrado.
- $\mathcal{K}$ es el espacio de claves.
- $\mathcal{E} = \{E_k : k \in \mathcal{K}\}$ es el conjunto de funciones de cifrado $E_k : \mathcal{M} \rightarrow \mathcal{C}$.
- $\mathcal{D} = \{D_k : k \in \mathcal{K}\}$ es el conjunto de funciones de descifrado $D_k : \mathcal{C} \rightarrow \mathcal{M}$.

Deben cumplirse las condiciones de corrección:

$$D_{k'}(E_k(M)) = M \quad \forall M \in \mathcal{M},$$

siendo k' la clave de descifrado asociada a la clave de cifrado k .

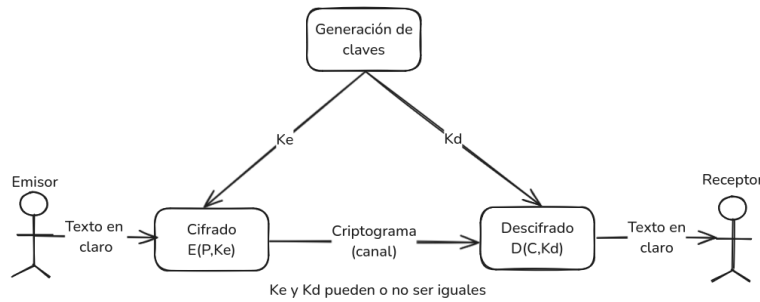


Figura 2.3: Esquema de un criptosistema formal

2.2. Criptografía sin clave o Funciones hash

Las funciones hash son funciones unidireccionales de cálculo sencillo y eficiente, utilizadas para garantizar la integridad de los datos. Se aplican a un mensaje M produciendo un valor de longitud fija:

$$M \mapsto H(M),$$

donde $H(M)$ suele tener 128, 160, 256 o 512 bits. Es computacionalmente inviable recuperar M a partir de $H(M)$.

Una función hash criptográfica es una función eficiente que mapea cadenas binarias de longitud arbitraria a cadenas binarias de longitud fija n . Para una función que produce valores de n bits, la probabilidad de que una cadena aleatoria termine en un valor dado es 2^{-n} . El valor de hash actúa como representante compacto de la cadena de entrada.

Para su uso en criptografía, H debe satisfacer al menos:

- **Resistencia a preimagen:** Dado un valor y de n bits, resulta inviable encontrar x tal que $H(x) = y$.

- **Resistencia a segunda preimagen:** Dado un mensaje x , resulta inviable hallar otro $x' \neq x$ con $H(x') = H(x)$.
- **Resistencia a colisiones:** Resulta inviable encontrar dos entradas distintas x, y tales que $H(x) = H(y)$.

Cuando H es público y sin clave, se le llama *código de detección de modificaciones* (MDC). Si se añade una clave secreta al cómputo de hash, obtenemos un *código de autenticación de mensaje* (MAC), que ofrece además autenticación del origen junto con integridad.

2.2.1. MD5

El MD5 (Message-Digest Algorithm 5) procesa el mensaje de entrada en bloques sucesivos de 512 bits y produce un resumen (digest) de 128 bits [56]. Su diseño sigue la construcción de Merkle–Damgard, inicializando cuatro registros de 32 bits y aplicando cuatro rondas de 16 operaciones cada una por bloque [69]. En 2004 se demostró que MD5 ya no es resistente a colisiones: dos entradas diferentes pueden colisionar y producir el mismo digest en tiempo práctico [69]. Por ejemplo, en 2009, Marc Stevens, Arjen Lenstra y Benne de Weger, un grupo de investigadores, construyeron colisiones con prefijos elegidos y generaron un certificado CA malicioso [65]. Actualmente se desaconseja MD5 para cualquier uso criptográfico y se recomienda SHA-2 o SHA-3 como alternativas seguras [46].

2.2.2. SHA-1

El SHA (Secure Hash Algorithm 1), fue publicado en 1995 como FIPS 180-1 por el NIST [40]. Procesa los datos de entrada en bloques de 512 bits al igual que MD5, pero produce un digest de 160 bits con cinco registros de 32 bits y 80 rondas de mezcla [40]. Su algoritmo es el siguiente:

1. Se toma el mensaje original y se rellena hasta alcanzar una longitud de $448 \bmod 512$ bits. Es decir, al dividir la longitud total entre 512, el resto debe ser 448.
2. Se añade un campo de 64 bits que indica la longitud del mensaje original (incluyendo el propio relleno). Así, la longitud total del mensaje resultante, denotado P' , será un múltiplo de 512 bits.
3. P' se divide en bloques de longitud fija de 512 bits, denotados como $Y_1, Y_2, \dots, Y_L$.
4. Se inicia el cálculo del resumen. Cada bloque de 512 bits se mezcla con un búfer de 160 bits mediante 80 rondas. Cada 20 rondas se modifican las funciones de mezcla utilizadas. Este proceso continúa hasta procesar todos los bloques de entrada.

Una vez terminado el cálculo, el buffer de 160 bits contiene el valor del compendio de mensaje. El SHA es 2^{32} veces más seguro que el MD5, pero su cálculo es más lento.

En 2005 Xiaoyun Wang y colaboradores redujeron la complejidad para hallar colisiones de 2^{80} a 2^{63} operaciones [70], y en 2017 Google y CWI demostraron la primera colisión real de SHA-1 en el experimento *SHAttered* [21]. El NIST anunció en 2011 la deprecación de SHA-1 y ha planificado retirarla completamente para fines de 2030 [46]. Chrome dejó de aceptar certificados TLS firmados con SHA-1 en enero de 2017 [68], Mozilla Firefox siguió en febrero de 2017 [39] y Microsoft finalizó el soporte para contenidos firmados con SHA-1 en 2021 [36].

2.3. Criptografía de clave privada o simétrica

Un cifrado simétrico utiliza una clave k elegida de un espacio (conjunto) de claves posibles K para cifrar un mensaje m perteneciente al conjunto de mensajes posibles M , generando un texto cifrado $c \in C$.

El cifrado puede verse como una función:

$$e : K \times m \rightarrow C$$

cuyo dominio es el conjunto de pares (k, m) y cuyo codominio es el espacio de textos cifrados C .

El descifrado se representa como una función:

$$d : K \times C \rightarrow m$$

Queremos que el descifrado revierta el cifrado:

$$d(k, e(k, m)) = m \quad \forall k \in K, \forall m \in M$$

Usando notación de subíndice para la clave, podemos escribir:

$$e_k : m \rightarrow C \quad \text{y} \quad d_k : C \rightarrow m$$

cumpliendo:

$$d_k(e_k(m)) = m \quad \forall m \in M$$

Esto implica que e_k es una función inyectiva. Es decir, si $e_k(m) = e_k(m')$, entonces:

$$m = d_k(e_k(m)) = d_k(e_k(m')) = m' \quad \forall k \in K, \forall m \in M$$

Propiedades deseables de un cifrado simétrico

Sea (K, m, C, e, d) un sistema de cifrado. Este debe cumplir las siguientes propiedades:

1. Para todo $k \in K$ y $m \in M$, debe ser fácil computar $e_k(m)$.
2. Para todo $k \in K$ y $c \in C$, debe ser fácil computar $d_k(c)$.
3. Dados uno o más textos cifrados $c_1, \dots, c_n$ producidos con una misma clave k , debe ser computacionalmente difícil recuperar $d_k(c_1), \dots, d_k(c_n)$ sin conocer k .

Con este tipo de criptografía podemos garantizar la confidencialidad porque únicamente quien posea la llave secreta será capaz de ver el mensaje. El problema con la criptografía simétrica es que si yo quisiera compartir secretos con n personas, para cada persona tendría que generar una nueva llave secreta. Otro problema asociado con este tipo de criptografía es cómo comparto con otra persona de una forma confidencial e integra la llave secreta. Estos problemas se resuelven de cierta manera con criptografía asimétrica.

2.3.1. Cifrado de bloque

Los cifrados de bloque cifran mensajes dividiéndolos en grupos de símbolos del mismo tamaño (bloques) y aplicando sobre cada uno de ellos un mismo algoritmo de cifra.

Llamamos B el tamaño de bloque del cifrado. Un mensaje de texto claro general consiste entonces en una lista de bloques de mensaje escogidos de M , y la función de cifrado transforma estos bloques en una lista de bloques de texto cifrado en C , donde cada bloque es una secuencia de B bits. Si el texto claro termina con un bloque de menos de B bits, rellenamos el final del bloque con ceros.

El cifrado y el descifrado se realizan bloque a bloque, por lo que basta estudiar el proceso para un único bloque de texto claro, es decir, para un solo $m \in M$. Esto, por supuesto, explica la conveniencia de dividir un mensaje en bloques: un mensaje puede ser de longitud arbitraria, y resulta útil centrar el proceso criptográfico en una pieza de longitud fija. El bloque de texto claro m es una cadena de B bits, que para mayor concreción identificamos con el número correspondiente en forma binaria. En otras palabras, identificamos M con el conjunto de enteros m que satisfacen $0 \leq m < 2^B$ mediante

$$\underbrace{m_{B-1} m_{B-2} \cdots m_2 m_1 m_0}_{\text{lista de } B \text{ bits de } m}$$

$\longleftrightarrow$

$$\underbrace{m_{B-1} \cdot 2^{B-1} + m_{B-2} \cdot 2^{B-2} + \cdots + m_1 \cdot 2 + m_0}_{\text{entero entre } 0 \text{ y } 2^B - 1}$$

Aquí $m_0, m_1, \dots, m_{B-1}$ son cada uno 0 o 1.

De manera similar, identificamos el espacio de claves K y el espacio de textos cifrados C con conjuntos de enteros correspondientes a cadenas de bits de un determinado tamaño de bloque. Para mayor simplicidad notacional, denotamos los tamaños de bloque para claves, textos claros y textos cifrados por B_k , B_m y B_c , respectivamente. No tienen por qué coincidir. Así, hemos identificado

$$K = \{k \in \mathbb{Z} : 0 \leq k < 2^{B_k}\}$$

$$M = \{m \in \mathbb{Z} : 0 \leq m < 2^{B_m}\}$$

$$C = \{c \in \mathbb{Z} : 0 \leq c < 2^{B_c}\}$$

Surge de inmediato una pregunta importante: ¿qué tan grande debe ser el conjunto K que se debe de escoger, equivalentemente, qué tamaño de bloque de clave B_k se debería de elegir? Si B_k es demasiado pequeño, se puede probar cada número entre 0 y $2^{B_k} - 1$ hasta dar con la clave generada. Más precisamente, dado que se asume que se conoce el algoritmo de descifrado d (principio de Kerckhoffs), toma cada $k \in K$ y calcula $d_k(c)$. Suponiendo que se es capaz de distinguir entre textos claros válidos e inválidos, se acabará recuperando el mensaje.

Este ataque se conoce como ataque de búsqueda exhaustiva (o ataque de fuerza bruta), puesto que se explora exhaustivamente el espacio de claves. Con la tecnología actual, una búsqueda exhaustiva se considera inviable si el espacio tiene al menos 2^{80} elementos. Por tanto, se debería de escoger $B_k \geq 80$.

Para muchos criptosistemas, existen refinamientos al ataque de búsqueda exhaustiva que sustituyen efectivamente el tamaño del espacio por su raíz cuadrada. Estos métodos se basan en el principio de que es más fácil encontrar objetos coincidentes (colisiones) en un conjunto que localizar un objeto en particular. Describimos algunos de estos ataques de "man-in-the-middle" de colisiones en la sección 2.6. Si están disponibles ataques de este tipo, se deberá de escoger $B_k \geq 160$.

Existen distintos modos de funcionamiento para los cifrados de bloques. El *Electronic CodeBook* (ECB) y el *Cipher-Block Chaining* (CBC) son dos de los modos más antiguos. Entre los sistemas de cifrado por bloques más conocidos están el Data Encryption Standard, Triple DES (3DES)[10], Advanced Encryption Standard (AES)[42] y Twofish[14][57].

ECB

El *Electronic CodeBook* (ECB) es el modo de funcionamiento más sencillo del cifrado por bloques. Es más fácil porque se cifra directamente cada bloque de texto plano de entrada y la salida es en forma de bloques de texto cifrado. Generalmente, si un mensaje tiene un tamaño superior a b bits, se puede dividir en un montón de bloques y repetir el procedimiento. En la figura se muestra un ejemplo de funcionamiento del mismo 2.4

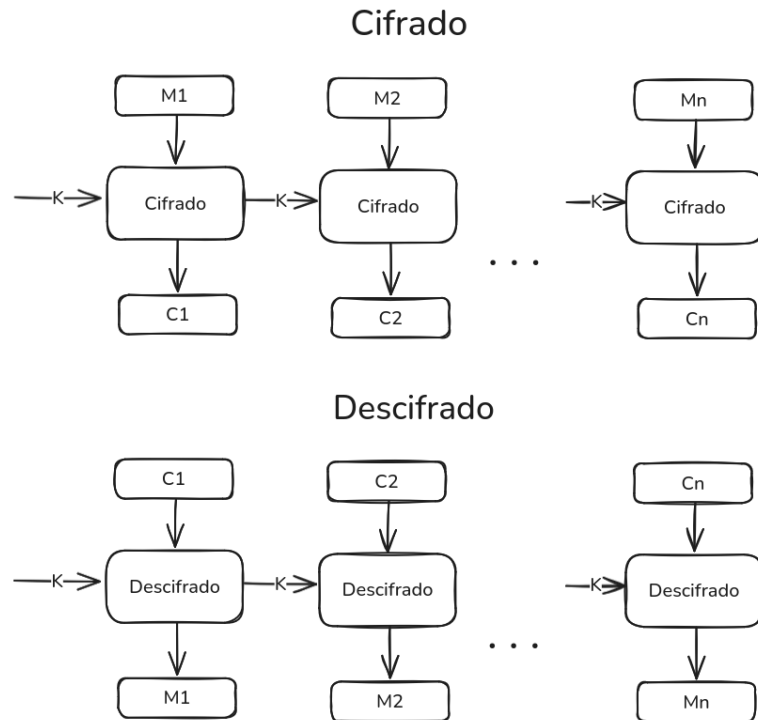


Figura 2.4: Ejemplo de cifrado y descifrado de bloque ECB

CBC

El modo *Cipher Block Chaining* (CBC) mejora la seguridad del modo ECB al encadenar cada bloque cifrado con el siguiente. Antes de entrar en su explicación, en este modo se introduce un nuevo concepto: Un IV es esencialmente otro input (además del texto plano M y la llave K) usado para cifrar el texto. Es un bloque de datos, usado por varios modos de cifrado de bloques para introducir aleatoriedad en el proceso de cifrado para poder generar como resultado distintos textos cifrados incluso con el mismo texto plano cifrado varias veces.

No es necesario que sea secreto, pero si es necesario que no se vuelva a usar. Es recomendable que sea aleatorio, impredecible y de un único uso. De

forma simplificada:

- Para cada bloque M_i de texto claro:

$$C_i = E(M_i \oplus C_{i-1}, K)$$

donde C_{i-1} es el bloque cifrado anterior.

- Para el primer bloque, no existe C_0 , por lo que se usa un vector de inicialización (IV) aleatorio:

$$C_1 = E(M_1 \oplus \text{IV}, K).$$

- De este modo, incluso si $M_i = M_j$, normalmente $C_i \neq C_j$ porque $C_{i-1} \neq C_{j-1}$.

Pasos de cifrado

1. Calcular $X_1 = M_1 \oplus \text{IV}$ y luego $C_1 = E(X_1, K)$.
2. Para $i > 1$, calcular $X_i = M_i \oplus C_{i-1}$ y luego $C_i = E(X_i, K)$.
3. Repetir hasta procesar todos los bloques de texto claro.

Pasos de descifrado

1. Descifrar $D_1 = D(C_1, K)$ y obtener $M_1 = D_1 \oplus \text{IV}$.
2. Para $i > 1$, descifrar $D_i = D(C_i, K)$ y obtener $M_i = D_i \oplus C_{i-1}$.
3. Repetir hasta recuperar todos los bloques de texto original.

En la figura se muestra un ejemplo de funcionamiento del mismo 2.5

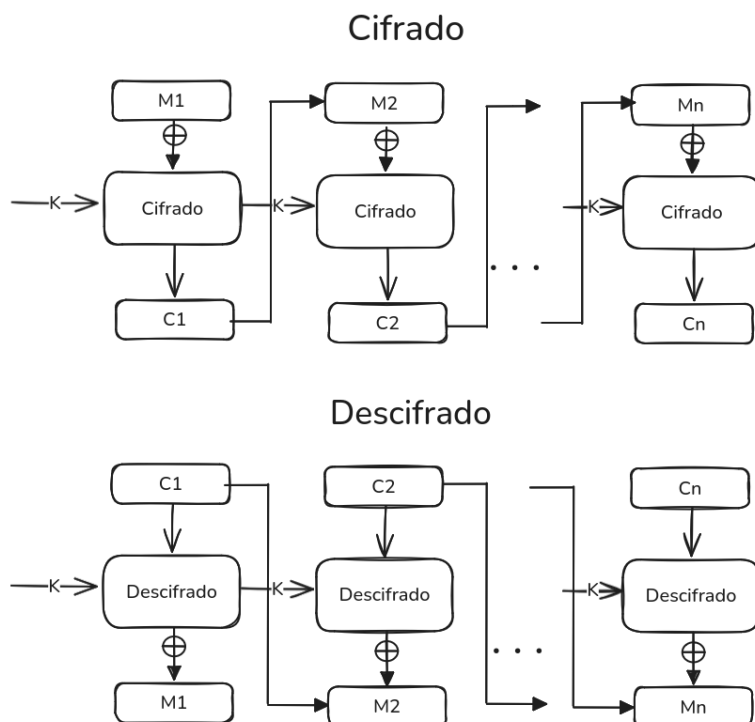


Figura 2.5: Ejemplo de cifrado y descifrado de bloque CBC

Una desventaja del mismo es que si se reutiliza el mismo IV para dos mensajes idénticos, se obtendrán los mismos bloques C_1 , lo que compromete seguridad[14].

2.3.2. Cifrado de flujo

Los cifrados de flujo son ciertamente cifrados de bloque con una longitud de bloque igual a uno. Lo que los hace útiles es que la transformación de cifrado puede cambiar para cada símbolo del texto plano que se cifra. También pueden usarse cuando los datos deben procesarse símbolo por símbolo (por ejemplo, si el equipo no dispone de memoria o el almacenamiento intermedio es limitado).

Sea K el espacio de claves. Una secuencia de símbolos $e_1e_2e_3 \dots \in K$ se denomina **flujo de claves** o *keystream*.

Sea A un alfabeto de q símbolos, y sea E_e un cifrado por sustitución simple con longitud de bloque 1, donde $e \in K$. Dado un texto claro $m_1m_2m_3 \dots$ y un flujo de claves $e_1e_2e_3 \dots$, un cifrado de flujo produce un texto cifrado $c_1c_2c_3 \dots$ tal que:

$$c_i = E_{e_i}(m_i)$$

Si d_i denota la función inversa de e_i , entonces:

$$D_{d_i}(c_i) = m_i$$

Este tipo de criptografía se basa en hacer un cifrado bit a bit, esto se logra usando la operación XOR. Se utiliza un algoritmo determinístico que genera una secuencia pseudoaleatoria de bits que junto con los bits del mensaje se van cifrando utilizando la operación XOR.

Un ejemplo de ello es el cifrado de vernam [16], definido sobre $A = \{0, 1\}$. Dado un mensaje binario $m_1 m_2 \dots m_t$ y una clave binaria $k_1 k_2 \dots k_t$ de la misma longitud, el texto cifrado es:

$$c_i = m_i \oplus k_i, \quad 1 \leq i \leq t$$

Si la clave es aleatoria y no se reutiliza, se denomina **one-time pad**.

Observación: Hay dos cifrados por sustitución posibles sobre A :

- $E_0: 0 \mapsto 0, 1 \mapsto 1$
- $E_1: 0 \mapsto 1, 1 \mapsto 0$

El flujo de claves determina cuál se aplica a cada símbolo.

Algunos ejemplos de cifrado de flujo son RC4 (usado en redes inalámbricas)[67, 20] y A5 (usado en telefonía celular)[66, 31].

2.4. Criptografía de clave pública o asimétrica

Si Alice y Bob quieren intercambiar mensajes usando un cifrado simétrico, deben primero acordar mutuamente una clave secreta k . Esto es factible si tienen la oportunidad de reunirse en secreto o si pueden comunicarse una vez por un canal seguro. Pero ¿qué ocurre si no disponen de esa oportunidad y cada comunicación entre ellos es vigilada por su adversaria Eva? ¿Es posible que Alice y Bob intercambien una clave secreta en estas condiciones?

La primera reacción de la mayoría es que no es posible, ya que el intruso ve cada fragmento de información que intercambian. Fue la brillante intuición de Diffie y Hellman que, bajo ciertas hipótesis, sí lo es. La búsqueda de soluciones eficientes (y demostrables) a este problema, denominado criptografía de clave pública (o asimétrica), constituye una de las partes más interesantes de la criptografía matemática.

Definimos espacios de claves K , de textos claros M y de textos cifrados C . Sin embargo, un elemento k del espacio de claves es en realidad un par de claves,

$$k = (k_{\text{priv}}, k_{\text{pub}}),$$

denominadas clave privada y clave pública, respectivamente. Para cada clave pública k_{pub} existe una función de cifrado correspondiente

$$e_{k_{\text{pub}}} : M \longrightarrow C,$$

y para cada clave privada k_{priv} existe una función de descifrado correspondiente

$$d_{k_{\text{priv}}} : C \longrightarrow M.$$

Estas funciones cumplen que si el par $(k_{\text{priv}}, k_{\text{pub}})$ pertenece al espacio de claves K , entonces

$$d_{k_{\text{priv}}}(e_{k_{\text{pub}}}(m)) = m \quad \forall m \in M.$$

Para que un cifrado asimétrico sea seguro, debe resultar difícil para el intruso calcular la función de descifrado $d_{k_{\text{priv}}}(c)$, incluso si conoce la clave pública k_{pub} . Bajo esta suposición, Alice puede enviar k_{pub} a Bob mediante un canal inseguro, y Bob puede devolver el texto cifrado $e_{k_{\text{pub}}}(m)$ sin temor a que el intruso lo descifre. Para poder descifrar, es necesario conocer la clave privada k_{priv} , y, presumiblemente, solo Alice dispone de ella. A esta clave privada a veces se le llama información de *trapdoor* de Alice, porque proporciona una vía directa para calcular la función inversa de $e_{k_{\text{pub}}}$. El hecho de que las claves de cifrado y descifrado (k_{pub} y k_{priv}) sean diferentes confiere al cifrado su carácter asimétrico, de ahí su nombre.

Resulta fascinante que Diffie y Hellman concibieran este concepto sin contar con un par concreto de funciones; no obstante, sí propusieron un método análogo por el cual Alice y Bob pueden intercambiar de forma segura un dato aleatorio cuyo valor no conocen inicialmente. Describimos el método de intercambio de claves de Diffie y Hellman en la Sección 2.4.1 y, a continuación, analizaremos más adelante varios cifrados asimétricos como RSA (Sección) y ECC (Sección), cuya seguridad se fundamenta en la dificultad presunta de distintos problemas matemáticos.

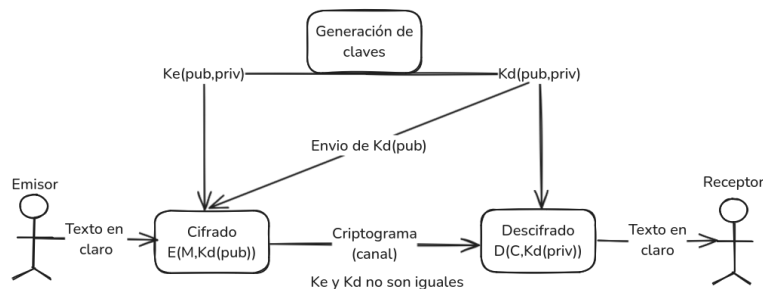


Figura 2.6: Ejemplo de cifrado asimétrico

2.4.1. Intercambio de claves Diffie–Hellman

Como hemos mencionado anteriormente, el protocolo de Diffie-Hellman, publicado en 1976 [17], resuelve el problema de dos entidades que desean compartir una clave secreta para usar en un cifrado simétrico, pero su único canal de comunicación es inseguro y todo lo que intercambian lo observa

el intruso. ¿Cómo pueden establecer una clave que el intruso no pueda deducir? La brillante idea de Diffie y Hellman fue aprovechar la dificultad del problema del logaritmo discreto en $\mathbb{F}_p^*$. A continuación, se muestra el procedimiento del mismo:

1. Alice y Bob acuerdan públicamente un primo grande p y un generador g de $\mathbb{F}_p^*$.

2. Alice elige en secreto un entero a y calcula

$$A \equiv g^a \pmod{p}.$$

3. Bob elige en secreto un entero b y calcula

$$B \equiv g^b \pmod{p}.$$

4. Intercambian A y B por el canal público. Eve conoce p, g, A, B .

5. Alice calcula la clave compartida

$$K = B^a \equiv g^{ba} \pmod{p},$$

y Bob calcula

$$K = A^b \equiv g^{ab} \pmod{p}.$$

Fase	Operación
Intercambio: elegir p y g	Un tercero de confianza elige y publica un primo p grande y un generador g de orden primo en $\mathbb{F}_p^*$.
Generación de claves secretas	Alice elige a , calcula $A \equiv g^a \pmod{p}$; Bob elige b , calcula $B \equiv g^b \pmod{p}$.
Intercambio público	Alice envía A a Bob, Bob envía B a Alice.
Cálculos finales	Alice calcula $K \equiv B^a \pmod{p}$; Bob calcula $K \equiv A^b \pmod{p}$.

Cuadro 2.1: Resumen del intercambio de claves Diffie–Hellman

Ejemplo 2.1. Supongamos que Alice y Bob acuerdan $p = 7$ y $g = 5$. Alice elige $a = 3$ y calcula

$$A = 5^3 \equiv 125 \equiv 6 \pmod{7},$$

mientras que Bob elige $b = 4$ y obtiene

$$B = 5^4 \equiv 625 \equiv 2 \pmod{7}.$$

Tras intercambiar $A = 6$ y $B = 2$, ambos calculan

$$K = 2^3 \equiv 8 \equiv 1 \pmod{7}, \quad K = 6^4 \equiv 1296 \equiv 1 \pmod{7},$$

de modo que la clave compartida es $K = 1$. El intruso ve p, g, A, B pero, para calcular $K = g^{ab}$, tendría que resolver el problema del logaritmo discreto en $\mathbb{F}_7^*$, lo cual es prácticamente imposible cuando p tiene cientos de dígitos.

2.4.2. Firma digital

TODO: preguntar si es necesario esta seccion

2.5. Resumen comparativo

Los sistemas criptográficos actuales suelen combinar ambas: por ejemplo, se utiliza un esquema de clave pública para establecer una clave de sesión y luego un cifrado simétrico para procesar los datos.

Hasta la fecha, la encriptación de clave pública es sustancialmente más lenta que la simétrica; sin embargo, no existe prueba de que deba serlo necesariamente. En la práctica, los puntos clave son:

1. La criptografía de clave pública facilita la firma eficiente, el no repudio) y la gestión de claves.
2. La criptografía de clave simétrica es muy eficiente para cifrar datos y para aplicaciones de integridad.

Observación (tamaños de clave) Para lograr un nivel de seguridad equivalente, las claves privadas en sistemas de clave pública (por ejemplo, 2048 o 3072 bits en RSA) deben ser mucho más largas que las secretas en sistemas simétricos (por ejemplo, 128 bits en AES). Esto se debe a que, mientras que el ataque más eficiente contra la simétrica es la búsqueda exhaustiva, los sistemas de clave pública conocidos admiten algoritmos *atajo* (por ejemplo, factorización) más eficientes que el barrido completo del espacio de claves. Por tanto, para igualar los 128 bits simétricos, se requieren claves RSA de unos 3072 bits, es decir, más de 24 veces más largas, aunque valores del orden de 10 veces mayores se citan frecuentemente en la literatura [49].

2.6. Principios de diseño criptográfico

En 1883, Auguste Kerckhoffs [51] formuló seis principios esenciales para el diseño de sistemas criptográficos:

1. **Indescifrabilidad práctica:** El sistema debe ser, cuanto menos, imposible de romper en la práctica.

2. **Seguridad sólo de la clave:** La seguridad no debe depender del secreto del algoritmo, sino únicamente de la clave.
3. **Clave manejable:** La clave debe poder ser almacenada, pudiendo ser reemplazable.
4. **Transmisión telegráfica:** El texto cifrado debe poder transmitirse sin errores por medios estándar.
5. **Portabilidad:** Los documentos que implementan el sistema deben ser portátiles y operables por una sola persona.
6. **Simplicidad de uso:** El sistema debe ser fácil de usar, sin requerir un esfuerzo mental excesivo.

2.7. Seguridad y modelos de ataque

La seguridad de un sistema criptográfico se evalúa frente a distintos modelos de ataque, que describen el acceso y las capacidades del adversario. A grandes rasgos, estos modelos se agrupan en:

2.7.1. Ataques pasivos y activos

Ataques pasivos: El atacante únicamente observa el tráfico cifrado (o el sistema) sin modificarlo. Incluyen la interceptación de mensajes y el análisis de tráfico, en los que se extrae información de patrones de comunicación [6][3].

Ataques activos: El atacante altera, inyecta o interfiere el flujo de datos. Ejemplos típicos son el man-in-the-middle (interposición entre emisor y receptor) y la modificación de mensajes en tránsito [5].

2.7.2. Modelos clásicos de criptoanálisis

- *Ataque por texto cifrado únicamente (COA):* El atacante dispone solo del texto cifrado. Es el caso más habitual porque en la práctica los atacantes a menudo solo logran capturar mensajes cifrados sin más información.
- *Fuerza bruta (exhaustiva):* Se calculan y prueban todas las claves posibles hasta dar con la correcta. Su dificultad depende únicamente del tamaño de la clave [7][2].
- *Ataque con texto plano conocido (KPA):* El atacante dispone de algún texto original junto con su versión cifrada. Por ejemplo, sabe que un correo empieza con "Estimado cliente:" ve esa misma parte cifrada. Con

esas parejas (texto claro, texto cifrado) puede acortar la búsqueda de la clave.

- *Ataque por texto plano elegido (CPA)*: El atacante elige ciertos textos para cifrar y obtiene sus correspondientes cifrados. Es como pedirle al dueño de la caja fuerte que te abra cosas que tú le entregas. Con los pares que él devuelve, puedes analizar el comportamiento del sistema y encontrar debilidades.
- *Ataque adaptativo por texto plano elegido (CPA2)*: Variante de CPA donde el atacante en tiempo real elige un mensaje, ve su cifrado, y luego, basándose en lo que aprendió, decide el siguiente mensaje a cifrar. Y así sucesivamente, mejorando su estrategia con cada consulta.
- *Ataque por texto cifrado elegido (CCA)*: El atacante no solo puede cifrar, sino que también puede solicitar descifrar ciertos textos cifrados de su elección (salvo el que realmente le interesa). Con eso obtiene planos válidos que le ayudan a deducir información y, finalmente, descifrar el mensaje objetivo.
- *Ataque adaptativo por texto cifrado elegido (CCA2)*: Versión más poderosa de CCA, donde tras cada petición de descifrado, el atacante ajusta sus futuras preguntas para exprimir al máximo el sistema y extraer todos los datos posibles.

2.7.3. Ataques avanzados

- *Ataques de canal lateral (side-channel)*: No atacan el cifrado matemático, sino que, se explota información derivada de la implementación (tiempos, consumo eléctrico, emisiones electromagnéticas) para reconstruir las claves sin romper directamente el algoritmo [4].
- *Ataque de clave relacionada (related-key)*: El atacante conoce cifrados de textos planos bajo claves que guardan alguna relación matemática con la clave objetivo (por ejemplo, variantes que difieren solo en unos bits). Si conoce cómo varía el cifrado al pasar de una clave a otra, puede deducir la clave original más rápido que con fuerza bruta.
- *Ataque por sesgo estadístico (bias attacks)*: Se buscan patrones no aleatorios en la generación de claves o estados intermedios para acotar posibles valores [1]. Si el generador de claves o el proceso intermedio tiende a preferir ciertos valores, se reduce drásticamente el número de opciones a probar.
- *Ataque Evil Maid*: Consiste en acceso físico al dispositivo para instalar malware o keyloggers y robar claves cuando el usuario no está presente.

Capítulo 3

Cuerpos finitos

La aritmética en cuerpos finitos es fundamental para numerosos algoritmos de clave pública, incluidos los basados en el problema del logaritmo discreto en cuerpos finitos, los esquemas de curvas elípticas y las aplicaciones emergentes de curvas hiperelípticas. El rendimiento de estos protocolos depende en gran medida de nuestra capacidad para realizar rápidamente operaciones básicas en el cuerpo subyacente.

Los cuerpos finitos (también llamados *cuerpos de Galois*) se denotan como $\text{GF}(p^m)$, donde p es un número primo y m un entero positivo. Para que los sistemas de curvas elípticas sean eficientes, es imprescindible implementar de manera óptima la suma, la resta, la multiplicación y la inversión en dicho cuerpo. Existen tres familias de cuerpos que resultan especialmente adecuadas para ECC (Criptografía de Curvas Elípticas):

- **Cuerpos primos** ($\mathbb{F}_p$),
- **Cuerpos binarios** ($\mathbb{F}_{2^m}$),
- **Cuerpos de extensión óptima** (OEF).

En las secciones siguientes se presenta una introducción informal a la teoría de cuerpos finitos y se describen en detalle los algoritmos eficientes para cada tipo de cuerpo.

Dado que en muchas aplicaciones los tamaños de p^m son muy grandes, entender la complejidad computacional de estas operaciones es clave para evaluar el rendimiento global de los esquemas criptográficos basados en curvas elípticas.

A lo largo de este trabajo emplearemos indistintamente las notaciones $\text{GF}(p^m)$ y $\mathbb{F}_{p^m}$ para referirnos al cuerpo finito de orden p^m ; ambas son habituales en la literatura.

3.1. Introducción a cuerpos finitos

Los cuerpos son abstracciones de sistemas numéricos familiares (como los racionales $\mathbb{Q}$, los reales $\mathbb{R}$ o los complejos $\mathbb{C}$) que reúnen ciertas propiedades esenciales. Un cuerpo $\mathbb{F}$ consta de un conjunto $\mathbb{F}$ junto con dos operaciones, suma $(+)$ y producto $(\cdot)$, que satisfacen:

1. $(\mathbb{F}, +)$ es un grupo abeliano con identidad suma denotada por 0.
2. $(\mathbb{F} \setminus \{0\}, \cdot)$ es un grupo abeliano con identidad multiplicativa denotada por 1.
3. La multiplicación distribuye sobre la suma:

$$(a + b) \cdot c = a \cdot c + b \cdot c \quad \text{para todo } a, b, c \in \mathbb{F}.$$

Si el conjunto $\mathbb{F}$ es finito, se dice que el cuerpo es finito.

En un cuerpo, la resta se define mediante la suma de inversos aditivos: para $a, b \in \mathbb{F}$,

$$a - b = a + (-b),$$

donde $-b$ es el elemento único que satisface $b + (-b) = 0$.

La división se define usando el inverso multiplicativo: para $a, b \in \mathbb{F}$ con $b \neq 0$,

$$\frac{a}{b} = a \cdot b^{-1},$$

siendo b^{-1} el único elemento que cumple $b \cdot b^{-1} = 1$.

El *orden* de un cuerpo finito es el número de sus elementos. Existe un cuerpo finito de orden q si y solo si q es una potencia prima, es decir, $q = p^m$ con p primo y m entero positivo.

- Si $m = 1$, el cuerpo se llama *cuerpo primo* y se denota $\mathbb{F}_p$.
- Si $m \geq 2$, se llama *cuerpo de extensión* y se denota $\mathbb{F}_{p^m}$.

Para cada potencia prima q , existe esencialmente un único cuerpo finito de orden q : todos ellos son isomorfos (idénticos en estructura, aunque difiera la representación de sus elementos). Por ello se usa la notación general $\mathbb{F}_q$.

Definición 3.1 (Característica de un cuerpo). *La característica de un cuerpo K , denotada $\text{char}(K)$, es el menor entero positivo n tal que*

$$\underbrace{1 + 1 + \cdots + 1}_{n \text{ sumandos}} = 0 \quad \text{en } K.$$

Si no existe tal n , se dice que K tiene característica 0. Para cualquier cuerpo, la característica es 0 o un número primo.

Ejemplo 3.1. Los cuerpos $\mathbb{Q}$, $\mathbb{R}$ y $\mathbb{C}$ tienen característica 0. El cuerpo finito $\mathbb{F}_q$ con $q = p^m$ tiene característica p ; en particular, en $\mathbb{F}_p$ se cumple que $p \cdot a = 0$ para todo $a \in \mathbb{F}_p$.

Observación 3.1. La característica del cuerpo base es decisiva en la teoría de curvas elípticas: las fórmulas para suma y duplicación de puntos, y la propia ecuación simplificada de Weierstrass, dependen del valor de $\text{char}(K)$ (véanse las secciones 4.2.2 y 4.2.3). En particular, los casos $\text{char}(K) \in \{2, 3\}$ requieren un tratamiento separado, porque no se puede dividir entre 2 o entre 3 en estos cuerpos.

3.1.1. Representación binaria y suma de enteros

Antes de entrar en los cuerpos primos y binarios, conviene fijar la notación de bajo nivel: cómo representamos un entero en binario y cómo se realizan en hardware las operaciones básicas, ya que toda la aritmética de cuerpo se construye sobre estas primitivas.

Todo entero no negativo a tiene una representación binaria única

$$a = \sum_{i=0}^{n-1} a_i 2^i, \quad a_i \in \{0, 1\}, \quad a_{n-1} \neq 0.$$

Los dígitos binarios a_i se llaman *bits*, y decimos que a es un entero de n bits. Para representar enteros negativos se añade un bit de signo.

Para sumar dos enteros en binario aplicamos el método enseñado en primaria, sumando uno a uno (bit a bit) y propagando el acarreo (la *llevada*) hacia la siguiente posición cuando sea necesario. Por ejemplo, para calcular $43 + 37 = 80$ en binario, donde $43 = 101011_2$ y $37 = 100101_2$:

$$\begin{array}{r} 101111 \\ 101011 \\ + 100101 \\ \hline 1010000 \end{array}$$

Los acarreo que entran en cada posición aparecen en rojo en la fila superior. El resultado es $1010000_2 = 80$.

La implementación en hardware suele usar un *sumador de 1 bit* que toma dos bits a_i, b_i y una llevada de entrada c_i , y calcula una suma s_i y una llevada de salida c_{i+1} como:

$$\begin{aligned} c_{i+1} &= (a_i \wedge b_i) \vee (c_i \wedge a_i) \vee (c_i \wedge b_i), \\ s_i &= a_i \oplus b_i \oplus c_i, \end{aligned}$$

donde $\wedge$, $\vee$ y $\oplus$ son las funciones booleanas AND, OR y XOR, respectivamente.

Encadenando $n + 1$ de estos sumadores de 1 bit podemos sumar dos números de n bits usando $7n + 7 = O(n)$ operaciones booleanas sobre bits individuales.

Observación 3.2. *El encadenamiento de sumadores, o ripple addition, impone un cómputo secuencial y ya casi no se utiliza en hardware real. En su lugar se emplean esquemas de carry-lookahead que permiten paralelizar la generación de las llevadas. Gracias a esto, los procesadores modernos pueden sumar en un solo ciclo de reloj enteros de 64 o incluso 128 bits, y con instrucciones SIMD (Single Instruction Multiple Data) realizar varias sumas de 64 bits en paralelo por ciclo.*

Otra opción es agrupar los bits en palabras de 64 bits:

$$a = \sum_{i=0}^{k-1} a_i 2^{64i}, \quad k = \lceil \frac{n}{64} \rceil.$$

Con ello, sumamos dos números de n bits en $O(k) = O(n)$ operaciones de palabra de 64 bits. Cada operación de palabra se ejecuta internamente con circuitos de bits de coste constante, de modo que el coste total sigue siendo $O(n)$ en términos de operaciones a nivel de bit.

3.2. Aritmética en $\mathbb{F}_p$

Sea p un número primo. El conjunto

$$\mathbb{F}_p = \{0, 1, 2, \dots, p-1\},$$

con las operaciones de suma y multiplicación módulo p , $\mathbb{Z}/p\mathbb{Z}$, constituye un cuerpo finito de orden p . Denotaremos este cuerpo por $\mathbb{F}_p$ y llamaremos a p el *módulo* de $\mathbb{F}_p$. Para cualquier entero a , $a \bmod p$ denota el único residuo r , $0 \leq r \leq p-1$, obtenido al dividir a por p ; esta operación se llama *reducción módulo p* .

Todos los elementos no nulos de $\mathbb{F}_p$ tienen inverso multiplicativo, y forman un grupo cíclico de orden $p-1$ (consecuencia de que el grupo multiplicativo de cualquier cuerpo finito es siempre cíclico).

Ejemplo 3.2 (Cuerpo primo $\mathbb{F}_{31}$). *Los elementos de $\mathbb{F}_{31}$ son $\{0, 1, 2, \dots, 30\}$. A continuación se muestran algunas operaciones en $\mathbb{F}_{31}$:*

1. **Suma:** $9 + 25 = 34 \equiv 3 \pmod{31}$.
2. **Resta:** $9 - 25 = -16 \equiv 15 \pmod{31}$.
3. **Multiplicación:** $9 \cdot 25 = 225 \equiv 8 \pmod{31}$.
4. **Inverso multiplicativo:** $9^{-1} = 7$ ya que $9 \cdot 7 = 63 \equiv 1 \pmod{31}$.

Para sumar dos elementos de $\mathbb{F}_p \cong \mathbb{Z}/p\mathbb{Z}$, representados como enteros únicos en $[0, p - 1]$, basta sumar los enteros y comprobar si el resultado es $\geq p$; en tal caso restamos p para obtener un valor en $[0, p - 1]$. De forma análoga, tras restar dos enteros, sumamos p si el resultado es negativo. El coste total sigue siendo $O(n)$ operaciones a nivel de bit, donde $n = \lg p$ es el número de bits necesarios para representar un elemento del cuerpo.

3.2.2. Multiplicación de enteros

Calculemos $43 \times 37 = 1591$ con el método de la escuela en binario. Como $37 = 100101_2$ tiene tres bits activos (en las posiciones 0, 2 y 5), el producto se obtiene como suma de tres copias desplazadas de $43 = 101011_2$:

$$\begin{array}{rcccccccc} & & & & & 1 & 0 & 1 & 0 & 1 & 1 \\ \times & & & & & 1 & 0 & 0 & 1 & 0 & 1 \\ \hline & & & & & 1 & 0 & 1 & 0 & 1 & 1 \\ & & & 1 & 0 & 1 & 0 & 1 & 1 & & \\ + & 1 & 0 & 1 & 0 & 1 & 1 & & & & \\ \hline & 1 & 1 & 0 & 0 & 0 & 1 & 1 & 0 & 1 & 1 & 1 \end{array}$$

La suma final $11000110111_2 = 1591$ confirma el cálculo. Multiplicar bits individuales es sencillo (una compuerta AND), pero requiere n^2 multiplicaciones de bit, seguidas de n sumas de números de n bits (desplazados adecuadamente). La complejidad de este algoritmo es $\Theta(n^2)$, lo que da la cota superior $M(n) = O(n^2)$. La única cota inferior conocida es la trivial $M(n) = \Omega(n)$, por lo que podríamos esperar mejorar $O(n^2)$, y de hecho se han desarrollado algoritmos asintóticamente más rápidos (Karatsuba, Toom–Cook, Schönhage–Strassen, Harvey–van der Hoeven). En la práctica, las bibliotecas GMP y NTL utilizadas en este trabajo seleccionan automáticamente el algoritmo de multiplicación más adecuado según el tamaño de los operandos, por lo que no abordamos su descripción detallada aquí.

La eficiencia de la multiplicación en cuerpos finitos es crucial en esquemas de curvas elípticas, ya que las limitaciones de los multiplicadores enteros y el coste del acarreo pueden convertirse en cuellos de botella en implementaciones directas.

3.3. Aritmética en $\mathbb{F}_{2^m}$

Los cuerpos finitos de orden 2^m , también llamados *cuerpos binarios*, se construyen como extensiones de polinomios:

$$\mathbb{F}_{2^m} \cong \mathbb{F}_2[x]/(f(x)),$$

donde $f(x)$ es un polinomio irreducible de grado m sobre $\mathbb{F}_2$. En tal cuerpo, cada elemento puede representarse como un polinomio de grado $\leq m$ con coeficientes en $0, 1$ (equivalentemente, como un vector binario de m bits).

3.3.1. Representación polinomial

Sea $f(z)$ un polinomio binario irreducible de grado m , y escríbalo como

$$f(z) = z^m + r(z).$$

Los elementos de $\mathbb{F}_{2^m}$ son los polinomios binarios de grado a lo sumo $m-1$. La suma de elementos es la suma habitual de polinomios sobre $\mathbb{F}_2$, y la multiplicación se realiza mód $f(z)$.

A un elemento

$$a(z) = a_{m-1}z^{m-1} + \cdots + a_1z + a_0$$

se le asocia el vector binario $a = (a_{m-1}, \dots, a_1, a_0)$ de longitud m . Definamos

$$t = \left\lceil \frac{m}{W} \right\rceil, \quad s = Wt - m.$$

En software, a puede almacenarse en un arreglo de t palabras de W bits:

$$A = (A[t-1], \dots, A[1], A[0]),$$

donde el bit menos significativo de $A[0]$ es a_0 , y los s bits más significativos de $A[t-1]$ quedan sin usar (siempre a cero).

$$\begin{array}{c} \underbrace{A[t-1] \quad A[t-2] \quad \cdots \quad A[1] \quad A[0]}_{t \text{ palabras de } W \text{ bits}} \\ \underbrace{a_{m-1} \quad \cdots \quad a_{tW}}_{s \text{ bits sin usar}} \quad a_{tW-1} \quad \cdots \quad a_{2W} \quad a_{2W-1} \quad \cdots \quad a_W \quad a_{W-1} \quad \cdots \quad a_0 \end{array}$$

Figura 3.1: Representación de $a \in \mathbb{F}_{2^m}$ como un arreglo A de palabras de W bits. Los $s = Wt - m$ bits de orden más alto de $A[t-1]$ permanecen sin usar.

La suma de elementos se realiza como la suma de polinomios coeficiente a coeficiente (operación equivalente a XOR bit a bit, sin acarreo), y la multiplicación se reduce módulo $f(z)$ tomando el residuo de grado $< m$ tras la división larga:

$$a(z) \cdot b(z) \text{ mód } f(z).$$

Ejemplo 3.3 (Cuerpo $\mathbb{F}_{2^3}$). Sea $f(x) = x^3 + x + 1$. Entonces

$$\mathbb{F}_{2^3} = \{0, 1, x, x+1, x^2, x^2+1, x^2+x, x^2+x+1\}.$$

1. Suma: $(x^2 + x + 1) + (x + 1) = x^2$.
2. Multiplicación: $(x^2 + x + 1)(x + 1) = x^3 + x + 1 \equiv x + 1 \text{ (mód } f(x))$.
3. Inverso: $(x^2 + x + 1)^{-1} = x^2$, pues $(x^2 + x + 1)x^2 = x^4 + x^3 + x^2 \equiv 1 \text{ (mód } f(x))$.

Ejemplo 3.4 (Isomorfismo de cuerpos). Para $m = 3$ hay dos polinomios irreducibles sobre $\mathbb{F}_2$: $f_1(x) = x^3 + x + 1$ y $f_2(x) = x^3 + x^2 + 1$. Los cuerpos $\mathbb{F}_2[x]/(f_1)$ y $\mathbb{F}_2[x]/(f_2)$ tienen los mismos elementos como conjunto y son isomorfos: existe un mapeo que envía la clase de x en uno a una raíz de f_1 en el otro.

3.3.2. Suma en $\mathbb{F}_{2^m}$

La suma de elementos de $\mathbb{F}_{2^m}$ se realiza bit a bit (XOR) en cada palabra, por lo que únicamente requiere t operaciones de palabra.

Algorithm 1 Suma en $\mathbb{F}_{2^m}$

Require: Arreglos $A[0..t-1]$ y $B[0..t-1]$ que representan los coeficientes de $a(z)$ y $b(z)$.

Ensure: Arreglo $C[0..t-1]$ que representa $c(z) = a(z) + b(z)$.

```

1: for  $i = 0$  to  $t - 1$  do
2:    $C[i] \leftarrow A[i] \oplus B[i]$ 
3: end for
4: return  $C$ 

```

Ejemplo de suma en $\mathbb{F}_{2^m}$ (bit a bit)

Consideremos $m = 4$, de modo que cada elemento se representa con 4 bits. Sean los polinomios

$$a(z) = z^3 + z + 1 \quad \longleftrightarrow \quad A = [1, 0, 1, 1],$$

$$b(z) = z^2 + z \quad \longleftrightarrow \quad B = [0, 1, 1, 0].$$

La suma $c(z) = a(z) + b(z)$ se obtiene bit a bit por XOR:

$$\begin{array}{r} 1\ 0\ 1\ 1\ (A) \\ \oplus\ 0\ 1\ 1\ 0\ (B) \\ \hline 1\ 1\ 0\ 1\ (C) \end{array}$$

Por tanto,

$$c(z) \longleftrightarrow C = [1, 1, 0, 1],$$

que corresponde al polinomio $z^3 + z^2 + 1$.

3.3.3. Multiplicación en $\mathbb{F}_{2^m}$

La técnica “shift-and-add” para la multiplicación en cuerpos finitos se basa en la observación:

$$a(z) \cdot b(z) = \sum_{i=0}^{m-1} a_i z^i b(z).$$

En la iteración i , se calcula

$$b \leftarrow b \cdot z \text{ mód } f(z)$$

(mediante un desplazamiento y, si el bit de más alto orden de b era 1, una reducción por $r(z)$) y, si $a_i = 1$, se acumula

$$c \leftarrow c + b.$$

Algorithm 2 Multiplicación “shift-and-add” en $\mathbb{F}_{2^m}$

Require: Polinomios binarios $A[0..t-1]$ y $B[0..t-1]$ que representan $a(z)$ y $b(z)$.

Ensure: Polinomio $C[0..t-1]$ que representa $c(z) = a(z) \cdot b(z) \text{ mód } f(z)$.

```

1:  $C \leftarrow 0$ 
2: if  $a_0 = 1$  then
3:    $C \leftarrow B$ 
4: end if
5: for  $i = 1$  to  $m - 1$  do
6:   Desplazar  $B$  un bit a la izquierda
7:   if el bit desplazado era 1 then
8:      $B \leftarrow B \oplus R$ 
9:   end if
10:  if  $a_i = 1$  then
11:     $C \leftarrow C \oplus B$ 
12:  end if
13: end for
14: return  $C$ 
```

Ejemplo de “shift-and-add”

Trabajemos en $\mathbb{F}_{2^4}$ con polinomio de reducción $f(x) = x^4 + x + 1$. Sean

$$a(x) = x^3 + x, \quad b(x) = x^2 + 1.$$

Los representamos como vectores de 4 bits:

$$a = [1, 0, 1, 0], \quad b = [0, 1, 0, 1].$$

1. Inicializar $C \leftarrow 0$. Como $a_0 = 0$, C sigue siendo 0.
2. $i = 1$: Desplazamos $b \rightarrow xb = x^3 + x$. Como $a_1 = 1$,

$$C \leftarrow C \oplus b = x^3 + x.$$

3. $i = 2$: Desplazamos $b \rightarrow x^4 + x^2 \equiv (x + 1) + x^2$ (reducción de x^4).
Ahora $b = x^2 + x + 1$. Como $a_2 = 0$, C no cambia.

4. $i = 3$: Desplazamos $b \rightarrow x^3 + x^2 + x$. Como $a_3 = 1$,

$$C \leftarrow (x^3 + x) \oplus (x^3 + x^2 + x) = x^2.$$

El resultado final es

$$c(x) = x^2.$$

3.4. Aritmética en cuerpos de extensión óptima (OEF)

En las secciones anteriores hemos detallado la aritmética en $\mathbb{F}_{p^m}$ para los dos casos relevantes en este trabajo: $p = 2$ (cuerpos binarios) y $m = 1$ (cuerpos primos). Las técnicas de la representación polinomial empleadas en el caso binario admiten una generalización natural a cualquier cuerpo de extensión $\mathbb{F}_{p^m}$, llevando la aritmética sobre los coeficientes a $\mathbb{F}_p$. Sea p un primo y $m \geq 2$. Denotamos por $\mathbb{F}_p[z]$ el anillo de polinomios en la variable z con coeficientes en $\mathbb{F}_p$. Sea $f(z) \in \mathbb{F}_p[z]$ un polinomio irreducible de grado m —existen para cualquier par (p, m) y pueden hallarse eficientemente. La irreducibilidad significa que $f(z)$ no se factoriza como producto de polinomios en $\mathbb{F}_p[z]$ de grado menor que m . Entonces:

$$\mathbb{F}_{p^m} = \{a_{m-1}z^{m-1} + \cdots + a_1z + a_0 : a_i \in \mathbb{F}_p\}$$

con suma habitual de polinomios y producto reducido módulo $f(z)$.

Observación 3.3. *Los OEF se incluyen aquí por completitud teórica y para enmarcar la elección de cuerpos en ECC. La implementación desarrollada en el capítulo 6 no hace uso de OEF: se limita a $\mathbb{F}_p$ y $\mathbb{F}_{2^m}$, que son las dos familias estandarizadas para criptografía de curvas elípticas (NIST, SECG). El material restante de esta sección se ofrece como referencia.*

Para implementaciones en hardware, los cuerpos binarios son atractivos porque las operaciones solamente requieren desplazamientos y sumas bit a bit (módulo 2). En software, esto puede resultar lento si existe un multiplicador de enteros en hardware. Por otro lado, la aritmética en cuerpos primos exige gestionar la propagación de acarreo, lo que puede ser costoso.

La idea de los *cuerpos de extensión óptima* (OEF) es elegir p , m y el polinomio de reducción de forma que se ajusten mejor al hardware; en concreto, escoger p de modo que quepa en una sola palabra simplifica el manejo del acarreo.

Definición 3.2. *Un cuerpo de extensión óptima (OEF) es un cuerpo finito $\mathbb{F}_{p^m}$ tal que:*

1. $p = 2^n - c$ para enteros n y c con $\log_2 |c| \leq n/2$.
2. Existe un polinomio irreducible $f(z) = z^m - \omega$ en $\mathbb{F}_p[z]$.

Si $c \in \{\pm 1\}$ se dice que el OEF es de Tipo I (y p es primo de Mersenne si $c = 1$); si $\omega = 2$, es de Tipo II.

p	$f(z)$	Parámetros	Tipo
$2^7 + 3$	$z^{13} - 5$	$n = 7, c = -3, m = 13, \omega = 5$	–
$2^{13} - 1$	$z^{13} - 2$	$n = 13, c = 1, m = 13, \omega = 2$	I, II
$2^{31} - 19$	$z^6 - 2$	$n = 31, c = 19, m = 6, \omega = 2$	II
$2^{31} - 1$	$z^6 - 7$	$n = 31, c = 1, m = 6, \omega = 7$	I
$2^{32} - 5$	$z^5 - 2$	$n = 32, c = 5, m = 5, \omega = 2$	II
$2^{57} - 13$	$z^3 - 2$	$n = 57, c = 13, m = 3, \omega = 2$	II
$2^{61} - 1$	$z^3 - 37$	$n = 61, c = 1, m = 3, \omega = 37$	I
$2^{89} - 1$	$z^2 - 3$	$n = 89, c = 1, m = 2, \omega = 3$	I

Cuadro 3.1: Parámetros de ejemplo para OEF. Aquí $p = 2^n - c$ es primo y $f(z) = z^m - \omega$ es irreducible en $\mathbb{F}_p[z]$.

Los siguientes resultados permiten verificar la irreducibilidad de un binomio $z^m - \omega$ en $\mathbb{F}_p[z]$.

Teorema 3.1. *Sea $m \geq 2$ un entero y $\omega \in \mathbb{F}_p^*$. El binomio $f(z) = z^m - \omega$ es irreducible en $\mathbb{F}_p[z]$ si y solo si:*

1. Cada factor primo de m divide al orden e de ω en $\mathbb{F}_p^*$, pero no divide a $(p-1)/e$.
2. Si $m \equiv 0 \pmod{4}$, entonces $p \equiv 1 \pmod{4}$.

Corolario 3.1. *Si ω es un elemento primitivo de $\mathbb{F}_p^*$ y $m \mid (p-1)$, entonces $z^m - \omega$ es irreducible en $\mathbb{F}_p[z]$.*

Ejemplo 3.5 (Cuerpo $\mathbb{F}_{251^5}$). Sea $p = 251$, $m = 5$ y $f(z) = z^5 + z^4 + 12z^3 + 9z^2 + 7$, irreducible en $\mathbb{F}_{251}[z]$. Entonces $\mathbb{F}_{251^5}$ consta de polinomios de grado < 5 con coeficientes en $\{0, 1, \dots, 250\}$. Por ejemplo, sean:

$$a = 123z^4 + 76z^2 + 7z + 4, \quad b = 196z^4 + 12z^3 + 225z^2 + 76.$$

1. Suma: $a + b = 68z^4 + 12z^3 + 50z^2 + 7z + 80$.
2. Resta: $a - b = 178z^4 + 239z^3 + 102z^2 + 7z + 179$.
3. Multiplicación: $a \cdot b = 117z^4 + 151z^3 + 117z^2 + 182z + 217$.
4. Inverso: $a^{-1} = 109z^4 + 111z^3 + 250z^2 + 98z + 85$.

3.5. Selección de cuerpos para ECC: comparativa y recomendaciones

Como se ha mencionado anteriormente, la elección del cuerpo finito sobre el que definimos una curva elíptica tiene un impacto decisivo en el rendimiento, la seguridad y la facilidad de implementación del sistema criptográfico. A continuación, resumimos los principales criterios y trade-offs:

■ **Cuerpos primos $\mathbb{F}_p$:**

- Muy usados en estándares (p.ej. NIST P-256, SECP256k1) [43].
- Operaciones de multiplicación y reducción pueden aprovechar multiplicadores de enteros en hardware [23].
- Más sencillos de proteger frente a canales laterales (menos ramificaciones en tiempo de reducción) [33].

■ **Cuerpos binarios $\mathbb{F}_{2^m}$:**

- Ventajosos en hardware ligero (solo desplazamientos y XOR) [64].
- Menos populares en software general-propósito porque requieren multiplicación bit-a-bit [23].
- Históricamente usados en tarjetas inteligentes y estándares ISO/IEC [64].

■ **Cuerpos de extensión óptima (OEF):**

- Diseñados para casar la longitud de palabra del procesador con la aritmética de $\mathbb{F}_p$.
- Permiten reducir costes de acarreo y de reducción polinómica.
- Para aplicaciones de alto rendimiento, se emplean de Tipo I o II cuyas operaciones en $\mathbb{F}_p$ caben en una palabra de máquina y el polinomio de reducción es un binomio simple ($z^m - \omega$) [37, 12]. Ejemplos concretos se recogen en la Tabla 3.1.

Criterios de selección

Al diseñar un sistema ECC, se ha de considerar lo siguiente:

1. *Rendimiento*: mediante tiempos de multiplicación e inversión. En CPU modernas, usar primos especiales (Mersenne, pseudo-Mersenne) acelera la reducción[13].
2. *Resistencia a canales laterales*: operaciones condicionales mínimas, tiempo de ejecución constante y operaciones atómicas [33].
3. *Compatibilidad y estandarización*: preferir curvas en $\mathbb{F}_p$ ampliamente auditadas (p.ej. curvas NIST, Brainpool, Curve25519) [30].
4. *Restricciones de hardware*: en dispositivos con ALU reducida puede convenir $\mathbb{F}_{2^m}$ o OEF para evitar multiplicadores de enteros avanzados [37].

Comparativa orientativa de costes

A modo de referencia, el cuadro 3.2 reproduce órdenes de magnitud típicos extraídos de la literatura [50, 23]; los tiempos exactos dependen del hardware, la versión de la biblioteca y el nivel de optimización. Una comparativa empírica obtenida mediante mediciones propias se presenta en el capítulo 6.

Cuerpo	Multiplicación	Inversión	Uso típico
$\mathbb{F}_p$ (Mersenne)	$\approx 1 \mu s$	$\approx 3 \mu s$	TLS, SSH, Bitcoin
$\mathbb{F}_{2^m}$ (binario)	$\approx 2 \mu s$	$\approx 5 \mu s$	Smart cards, RFID
OEF ($p = 2^{32} - 5$)	$\approx 0.8 \mu s$	$\approx 2.5 \mu s$	Hardware especializado

Cuadro 3.2: Órdenes de magnitud orientativos de los costes en una CPU de 3 GHz para cuerpos de 256 bits, según la literatura. Los valores experimentales medidos sobre la implementación de este trabajo se presentan en el capítulo 6.

Parte III

Marco Teórico

Capítulo 4

Teoría de las curvas elípticas

4.1. Introducción y motivación

Los sistemas criptográficos basados en curvas elípticas dependen, en última instancia, de la aritmética de los puntos de la curva. Como se mostró en el capítulo 3, la eficiencia de estos esquemas se apoya directamente en la velocidad de los algoritmos de aritmética de curva, que a su vez se construyen sobre las operaciones del cuerpo subyacente: suma, multiplicación e inversión.

La figura 4.1 muestra los bloques necesarios para entender e implementar un protocolo como el algoritmo de firma digital sobre curvas elípticas (ECD-SA). La aritmética de curvas no solo descansa en las operaciones del cuerpo, sino que en algunos casos también requiere aritmética de grandes enteros y operaciones modulares. ECDSA emplea, además, una función hash; pero los pasos computacionalmente más costosos son las propias operaciones sobre la curva.

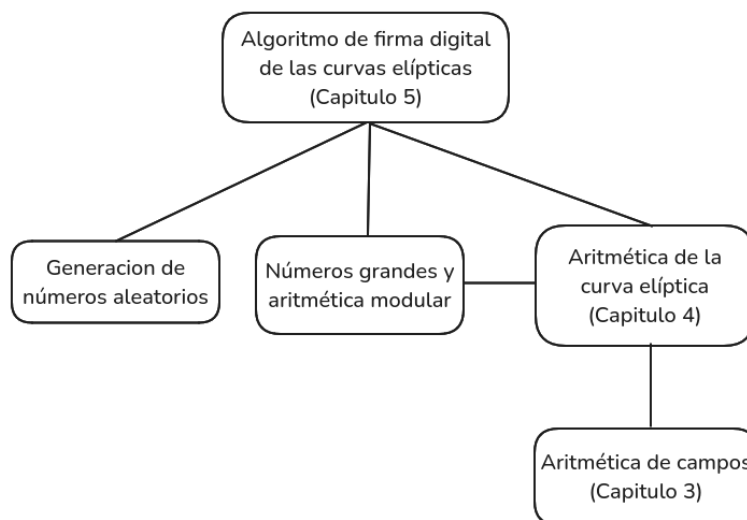


Figura 4.1: Esquema del Algoritmo de Firma Digital sobre Curvas Elípticas (ECDSA).

Este capítulo desarrolla los fundamentos teóricos necesarios para abordar el capítulo de criptografía y la implementación. La sección 4.1.1 ofrece un breve recorrido histórico desde las integrales elípticas hasta su uso criptográfico moderno. La sección 4.2 introduce la definición formal de curva elíptica como modelo de Weierstrass generalizado, junto con sus invariantes asociados (discriminante, condición de no singularidad, punto en el infinito). En la sección 4.2.2 se obtienen las formas simplificadas según la característica del cuerpo base, distinguiendo los casos $\text{char}(K) \neq 2, 3$, $\text{char}(K) = 2$ y $\text{char}(K) = 3$. La sección 4.2.3 construye la *ley de grupo* sobre los puntos de la curva mediante la regla de la cuerda y la tangente, primero de forma geométrica y después en fórmulas algebraicas explícitas para cada caso. La sección 4.3 estudia el comportamiento del grupo de puntos cuando el cuerpo base es finito $\mathbb{F}_q$, incluyendo el teorema de Hasse sobre el orden del grupo. Finalmente, la sección 4.4 introduce las *coordenadas proyectivas* (estándar, Jacobianas, Chudnovsky y López–Dahab), motivadas por la necesidad de evitar la inversión modular en cada operación, lo cual conecta directamente con las decisiones de implementación detalladas en el capítulo 6.

4.1.1. Historia

Las curvas elípticas nacen del problema de calcular la longitud de arco de una elipse, que John Wallis abordó en 1655, y se articulan primero como integrales elípticas gracias a los pioneros Giulio Fagnano y Leonhard Euler en la primera mitad del siglo XVIII. En 1786, Adrien-Marie Legendre sistematiza estas integrales en tres “tipos” y sienta las bases de las funciones

elípticas. A finales de la década de 1820, Niels Abel y Carl Jacobi invierten estas integrales, dando lugar a la teoría clásica de funciones elípticas. A mediados del siglo XIX, Karl Weierstrass y Bernhard Riemann las conectan con la estructura algebraico-geométrica que hoy reconocemos en la forma normal de Weierstrass. Finalmente, en el siglo XX, Louis Mordell y André Weil desarrollan la teoría aritmética de puntos racionales, abriendo paso a la geometría algebraica moderna.

4.2. Definición de una curva elíptica

Definición 4.1 (Curva elíptica sobre un cuerpo). *Sea K un cuerpo. Una curva elíptica E sobre K es un modelo de Weierstrass generalizado de la forma*

$$E : y^2 + a_1 x y + a_3 y = x^3 + a_2 x^2 + a_4 x + a_6, \quad (4.1)$$

donde

$$a_1, a_2, a_3, a_4, a_6 \in K,$$

sujeto a la condición de no singularidad que expresaremos a continuación.

Invariantes auxiliares. Definimos los siguientes invariantes (denotados habitualmente b_2, b_4, b_6, b_8 en la literatura clásica [61, 23]; aquí los llamamos d_i para evitar confusión con el coeficiente b de la ecuación corta):

$$\begin{aligned} d_2 &= a_1^2 + 4a_2, & d_4 &= 2a_4 + a_1a_3, \\ d_6 &= a_3^2 + 4a_6, & d_8 &= a_1^2a_6 + 4a_2a_6 - a_1a_3a_4 + a_2a_3^2 - a_4^2. \end{aligned}$$

Discriminante. El discriminante de la ecuación (4.1) se define por

$$\Delta = -d_2^2 d_8 - 8d_4^3 - 27d_6^2 + 9d_2 d_4 d_6.$$

Para que (4.1) defina una curva elíptica se exige $\Delta \neq 0$ (condición de no singularidad, discutida más adelante).

Punto en el infinito y conjunto de puntos racionales. Denotamos por $\mathcal{O}$ el único punto en el infinito que completa la curva al considerar su cierre proyectivo: corresponde al punto $(0 : 1 : 0)$ en $\mathbb{P}^2(K)$ y desempeña el papel de elemento neutro de la ley de grupo (sección 4.2.3). Para cualquier extensión de cuerpos L/K , el conjunto de puntos de E con coordenadas en L viene dado por

$$E(L) = \{(x, y) \in L^2 : y^2 + a_1 x y + a_3 y - x^3 - a_2 x^2 - a_4 x - a_6 = 0\} \cup \{\mathcal{O}\}.$$

4.2.1. Comentarios sobre la definición 4.1

1. La ecuación (4.1) se conoce como la *ecuación de Weierstrass*.
2. Decimos que E está *definida sobre* K porque los coeficientes a_i pertenecen a K . A menudo escribimos E/K y diremos que K es un *cuerpo base*.
3. La condición $\Delta \neq 0$ garantiza que la curva es *suave*, es decir, no tiene puntos singulares. Recordemos que un punto (x_0, y_0) de la curva se denomina *singular* cuando ambas derivadas parciales de

$$f(x, y) = y^2 + a_1xy + a_3y - x^3 - a_2x^2 - a_4x - a_6$$

se anulan simultáneamente en él, es decir, cuando $\partial f / \partial x(x_0, y_0) = \partial f / \partial y(x_0, y_0) = 0$. Geométricamente, en un punto singular la curva no admite una recta tangente bien definida: puede tener dos tangentes distintas (*nodo*) o una sola tangente con multiplicidad doble (*cúspide*). Un ejemplo clásico de cúspide aparece en la curva $y^2 = x^3$, singular en el origen, donde las dos derivadas parciales se anulan a la vez. La condición algebraica $\Delta \neq 0$ excluye precisamente la presencia de tales puntos. En la figura 4.2 se muestra un ejemplo de curva singular (con $\Delta = 0$).

4. El punto $\mathcal{O}$ es el único punto de la “línea en el infinito” que satisface la forma proyectiva de la ecuación de Weierstrass.
5. Los *puntos L -rationales* de E son aquellos cuyos pares (x, y) pertenecen al cuerpo L . Siempre incluimos $\mathcal{O}$.

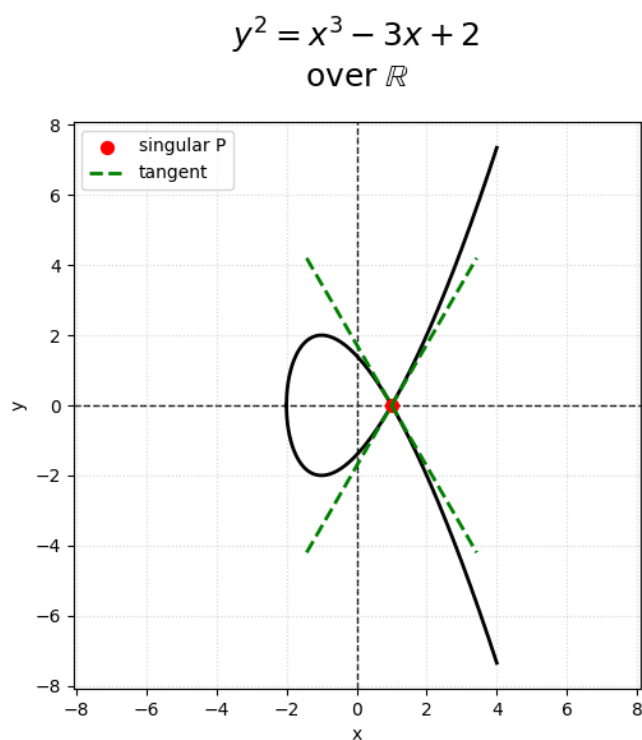


Figura 4.2: Ejemplo de una curva con un punto singular (cúspide en el origen), correspondiente a $\Delta = 0$.

Ejemplo 4.1. *Consideremos dos curvas elípticas sobre $\mathbb{R}$:*

$$E_1 : y^2 = x^3 - 12x - 5, \quad E_2 : y^2 = x^3 + 3x + 15.$$

Las gráficas de los conjuntos $E_1(\mathbb{R}) \setminus \{\mathcal{O}\}$ y $E_2(\mathbb{R}) \setminus \{\mathcal{O}\}$ se muestran en la figura 4.3.

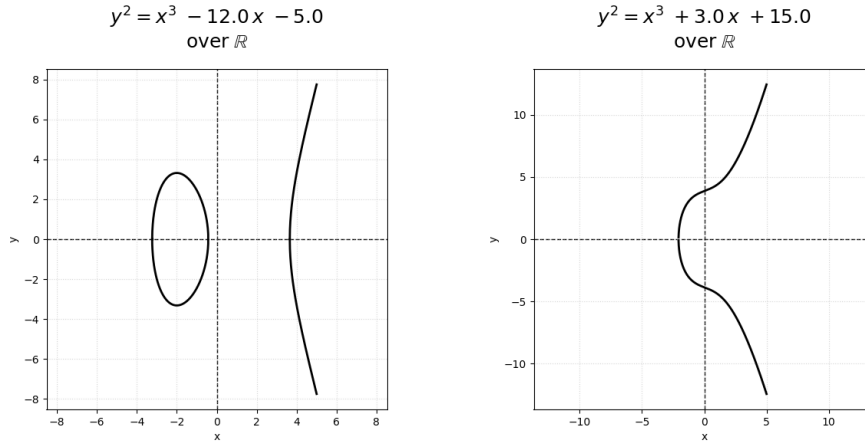


Figura 4.3: Curvas E_1 y E_2 sobre $\mathbb{R}$, con sus respectivos puntos (excepto $\mathcal{O}$).

4.2.2. Ecuaciones simplificadas de Weierstrass

Definición 4.2. Sean dos curvas elípticas E_1 y E_2 definidas sobre un cuerpo K , dadas por las ecuaciones de Weierstrass

$$\begin{aligned} E_1 : y^2 + a_1 xy + a_3 y &= x^3 + a_2 x^2 + a_4 x + a_6, \\ E_2 : y'^2 + a'_1 x' y' + a'_3 y' &= x'^3 + a'_2 x'^2 + a'_4 x' + a'_6, \end{aligned}$$

con coeficientes $a_i, a'_i \in K$. Decimos que E_1 y E_2 son isomorfas sobre K si existen $u, r, s, t \in K$ con $u \neq 0$ tales que el cambio de variables

$$(x, y) \mapsto (u^2 x' + r, u^3 y' + u^2 s x' + t) \quad (4.2)$$

transforma la ecuación de E_1 en la de E_2 . A dicho cambio se le llama admisible, y la relación así definida es una relación de equivalencia sobre el conjunto de curvas elípticas definidas sobre K .

1. Característica $\neq 2, 3$

Si $\text{char}(K) \neq 2, 3$, el cambio de variables admisible

$$(x, y) \mapsto \left(x - \frac{3a_1^2 - 12a_2}{36}, y - \frac{3a_1 x}{216} - \frac{a_1^3 + 4a_1 a_2 - 12a_3}{24} \right)$$

lleva

$$y^2 + a_1 xy + a_3 y = x^3 + a_2 x^2 + a_4 x + a_6$$

a la forma simplificada

$$y^2 = x^3 + ax + b, \quad (4.3)$$

donde $a, b \in K$. El discriminante de esta curva es

$$\Delta = -16(4a^3 + 27b^2) \neq 0.$$

Razón algebraica de la simplificación Los cambios de variable implementan los pasos de “completar cuadrados” y “completar cubos” para eliminar los *términos cruzados* (aquellos que mezclan x e y o que no son puramente y^2 o x^3) en la ecuación general:

- Para suprimir $a_1xy + a_3y$, completamos el cuadrado en $y^2 + a_1xy + a_3y$ mediante

$$y = y' - \frac{a_1}{2}x - \frac{a_3}{2},$$

que requiere $\frac{1}{2} \in K$ (por tanto $2 \neq 0$).

- Para eliminar el término a_2x^2 de la parte derecha, completamos el cubo en

$$x^3 + a_2x^2 + \dots$$

aplicando

$$x = x' + \frac{a_2}{3},$$

que exige $\frac{1}{3} \in K$ (por tanto $3 \neq 0$).

Ejemplo detallado: Sea $K = \mathbb{Q}$ y la curva

$$y^2 + 6xy + 2y = x^3 + 9x^2 + 5x + 1.$$

Paso 1: Eliminación de xy , con

$$y = y' - 3x - 1.$$

Sustituyendo en el lado izquierdo obtenemos:

$$\begin{aligned} \text{LHS} &= (y' - 3x - 1)^2 + 6x(y' - 3x - 1) + 2(y' - 3x - 1) \\ &= y'^2 - 6xy' - 2y' + 9x^2 + 6x + 1 \\ &\quad + 6xy' - 18x^2 - 6x + 2y' - 6x - 2 \\ &= y'^2 - 9x^2 - 6x - 1. \end{aligned}$$

Por tanto, la ecuación se convierte en

$$y'^2 - 9x^2 - 6x - 1 = x^3 + 9x^2 + 5x + 1.$$

Reordenamos:

$$y'^2 = x^3 + (9 + 9)x^2 + (5 + 6)x + (1 + 1) = x^3 + 18x^2 + 11x + 2.$$

Aquí $a'_2 = 18$.

Paso 2: Eliminación de x^2 , con

$$x = x' + \frac{a'_2}{3} = x' + 6.$$

Sustituyendo en el lado derecho y expandiendo:

$$\begin{aligned} x^3 + 18x^2 + 11x + 2 &= (x' + 6)^3 + 18(x' + 6)^2 + 11(x' + 6) + 2 \\ &= x'^3 + 18x'^2 + 108x' + 216 \\ &\quad + 18(x'^2 + 12x' + 36) + 11x' + 66 + 2 \\ &= x'^3 + (18 + 18)x'^2 + (108 + 216 + 11)x' + (216 + 648 + 68) \\ &= x'^3 + 36x'^2 + 335x' + 932. \end{aligned}$$

Finalmente, tras restar el término $36x'^2$, obtenemos

$$y'^2 = x'^3 + 335x' + 932,$$

que ya está libre de x'^2 .

2. Característica 2

En $\text{char}(K) = 2$ hay dos casos:

2.1. Caso $a_1 \neq 0$ (no supersingular) El cambio

$$(x, y) \mapsto \left(\frac{a_1^2 x + a_3}{a_1}, \frac{a_3^2 y + a_1^2 a_4 + a_3^2}{a_1^2} \right)$$

simplifica la ecuación general a

$$y^2 + xy = x^3 + Ax^2 + B, \tag{4.4}$$

con $A, B \in K$. Su discriminante es $\Delta = B \neq 0$.

2.2. Caso $a_1 = 0$ (supersingular) El cambio

$$(x, y) \mapsto (x + a_2, y)$$

transforma la curva a

$$y^2 + Cy = x^3 + Ax + B, \tag{4.5}$$

donde $A, B, C \in K$. Su discriminante es $\Delta = C^4 \neq 0$.

3. Característica 3

En $\text{char}(K) = 3$ también hay dos casos:

3.1. Caso $a_1^2 \neq -a_2$ (no supersingular) Definimos

$$d_2 = a_1^2 + a_2, \quad d_4 = a_4 - a_1 a_3,$$

y aplicamos el cambio

$$(x, y) \mapsto \left(x + \frac{d_4}{d_2}, y + a_1 x + \frac{a_1 d_4}{d_2} + a_3 \right).$$

La ecuación resultante es

$$y^2 = x^3 + Ax^2 + B, \quad (4.6)$$

con $A, B \in K$, y discriminante $\Delta = -A^3 B \neq 0$.

3.2. Caso $a_1^2 = -a_2$ (supersingular) El sencillo cambio

$$(x, y) \mapsto (x, y + a_1 x + a_3)$$

conduce a la forma

$$y^2 = x^3 + Ax + B, \quad (4.7)$$

con $A, B \in K$ y discriminante $\Delta = -A^3 \neq 0$.

Interpretación y uso de las formas simplificadas

La razón de distinguir tres familias de ecuaciones simplificadas responde a la imposibilidad de dividir por 2 o 3 cuando $\text{char}(K)$ es 2 o 3, lo que impide completar cuadrados y cubos para despejar los términos cruzados. Cada forma reducida se adapta a:

- **Característica $\neq 2, 3$ (ecuación corta de Weierstrass)**

$$y^2 = x^3 + ax + b.$$

Es la forma *short* más habitual en criptografía de curva elíptica sobre primos grandes, ya que minimiza el número de parámetros y simplifica las fórmulas de suma y duplicación [61]. Todas las curvas estándar de NIST y SECG usan este modelo [48].

- **Característica 2 (cuerpos binarios)**

$$\begin{aligned} y^2 + xy &= x^3 + Ax^2 + B && \text{(no supersingular),} \\ y^2 + Cy &= x^3 + Ax + B && \text{(supersingular).} \end{aligned}$$

En hardware ligero y tarjetas inteligentes, trabajar en $\mathbb{F}_{2^m}$ sólo requiere desplazamientos y XOR, sin multiplicaciones completas de enteros [32]. Las curvas no supersingulares se eligen para evitar vulnerabilidades a ataques basados en endomorfismos especiales.

■ **Característica 3**

$$y^2 = x^3 + Ax^2 + B \quad \text{o} \quad y^2 = x^3 + Ax + B,$$

con distinción similar entre casos *supersingular* y *ordinarios* [61]. Aunque menos comunes en criptografía práctica, estas curvas sirven en estudios teóricos y en construcciones avanzadas de pares bilineales.

Curvas ordinarias vs. supersingulares La condición de *supersingularidad* (discriminante especial, ver cada caso) refleja que el grupo de puntos de la curva tiene un anillo de endomorfismos más grande. Si bien ciertas construcciones pairing-based usan curvas supersingulares (por sus endomorfismos explícitos), para ECC estándar se prefieren curvas *ordinarias* (no supersingulares) para evitar ataques que explotan ese extra de simetría [71].

4.2.3. Ley de Grupo

Sea E una curva elíptica definida sobre un cuerpo K . El conjunto de puntos $E(K)$, junto con un elemento especial $\mathcal{O}$ (punto en el infinito), adquiere una estructura de grupo abeliano mediante la llamada *regla de la cuerda y la tangente*. A continuación describimos primero la construcción geométrica y, después, distinguimos los casos algebraicos según la posición relativa de los dos operandos.

Descripción geométrica

- **Identidad.** El punto en el infinito $\mathcal{O}$ actúa como elemento neutro:

$$P + \mathcal{O} = \mathcal{O} + P = P, \quad \forall P \in E(K).$$

- **Opuesto.** Dado $P = (x, y) \in E(K)$, su opuesto es $-P = (x, -y)$ (la reflexión sobre el eje x), de modo que $P + (-P) = \mathcal{O}$.

- **Suma de puntos distintos (cuerda).** Para $P \neq Q$ trazamos la recta secante que los une; ésta corta a la curva en un tercer punto R' . Entonces

$$P + Q = -R'$$

es la reflexión de R' respecto al eje x (véase la figura 4.4(a)).

- **Duplicación (tangente).** Para $P = Q$ trazamos la recta tangente a la curva en P ; ésta vuelve a cortar a la curva en un punto R' . De forma análoga,

$$2P = -R'$$

es la reflexión de R' sobre el eje x (véase la figura 4.4(b)).

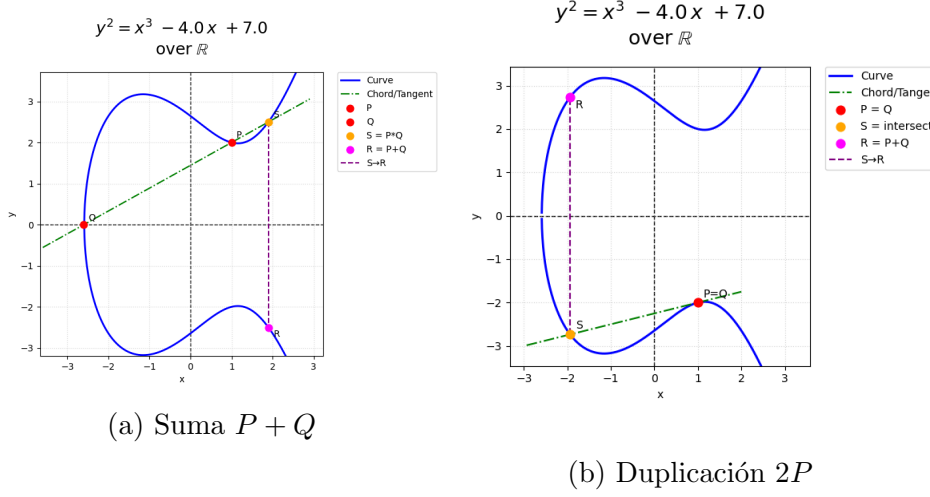


Figura 4.4: Geometría de la ley de grupo en curvas elípticas.

Casos algebraicos para calcular $P + Q$

Sean $P = (x_1, y_1)$ y $Q = (x_2, y_2)$ dos puntos de $E(K)$. El cálculo de $R = P + Q$ se descompone en los cinco casos disjuntos siguientes:

1. **Caso 1: alguno de los operandos es $\mathcal{O}$.** Si $P = \mathcal{O}$, entonces $R = Q$; análogamente si $Q = \mathcal{O}$, entonces $R = P$.
2. **Caso 2: $Q = -P$ (puntos opuestos).** Si $x_1 = x_2$ y $y_2 = -y_1 - a_1x_1 - a_3$ (en la forma corta, simplemente $y_2 = -y_1$), entonces los puntos son inversos y

$$R = P + (-P) = \mathcal{O}.$$

3. **Caso 3: $P \neq Q$ y $x_1 \neq x_2$ (suma genérica con cuerda).** Se calcula la pendiente de la cuerda y se obtiene el tercer punto.
4. **Caso 4: $P = Q$ y $y_1 \neq 0$ (duplicación con tangente).** Se calcula la pendiente de la tangente y se obtiene la duplicación $2P$.
5. **Caso 5: $P = Q$ y $2y_1 + a_1x_1 + a_3 = 0$ (tangente vertical).** En la forma corta esto equivale a $y_1 = 0$. En tal situación la tangente es vertical y

$$R = 2P = \mathcal{O}.$$

Los casos 3 y 4 son los únicos que requieren aritmética no trivial en el cuerpo: las fórmulas concretas se exponen a continuación, primero para característica $\neq 2, 3$ y después para cuerpos binarios.

Propiedades del grupo abeliano

El conjunto de puntos de la curva, junto con la operación $+$ y el elemento neutro $\mathcal{O}$, forma un *grupo abeliano* que denotamos

$$(E(K), +, \mathcal{O}).$$

Este grupo verifica las siguientes propiedades:

1. **Clausura.** Para todo $P, Q \in E(K)$, se tiene

$$P + Q \in E(K).$$

2. **Asociatividad.** Para todo $P, Q, R \in E(K)$,

$$(P + Q) + R = P + (Q + R).$$

3. **Elemento neutro.** Existe $\mathcal{O} \in E(K)$ tal que

$$P + \mathcal{O} = \mathcal{O} + P = P, \quad \forall P \in E(K).$$

4. **Inverso.** Para cada $P \in E(K)$ existe $-P \in E(K)$ (el opuesto de P) tal que

$$P + (-P) = (-P) + P = \mathcal{O}.$$

5. **Conmutatividad.** Para todo $P, Q \in E(K)$,

$$P + Q = Q + P.$$

En conjunto, estas cinco cláusulas definen un grupo abeliano (en particular, las cuatro primeras lo definen como grupo, y la quinta asegura que es abeliano).

Fórmulas en cuerpos de característica $\neq 2, 3$

Para la forma corta de Weierstrass

$$E: y^2 = x^3 + ax + b, \quad a, b \in K, \quad \text{char}(K) \neq 2, 3,$$

las operaciones en coordenadas afines (x, y) sobre K se concretan, según el caso identificado arriba, como sigue:

1. *Negativo:* $-P = (x, -y)$.

2. *Suma con $P \neq \pm Q$ (caso 3, $x_1 \neq x_2$).* Sea

$$\lambda = \frac{y_2 - y_1}{x_2 - x_1}, \quad x_3 = \lambda^2 - x_1 - x_2, \quad y_3 = \lambda(x_1 - x_3) - y_1.$$

Entonces $P + Q = (x_3, y_3)$.

3. *Duplicación con $y_1 \neq 0$ (caso 4).* Sea

$$\lambda = \frac{3x_1^2 + a}{2y_1}, \quad x_3 = \lambda^2 - 2x_1, \quad y_3 = \lambda(x_1 - x_3) - y_1.$$

Entonces $2P = (x_3, y_3)$.

Ley de grupo en cuerpos binarios

Cuando $K = \mathbb{F}_{2^m}$ y la curva es no supersingular,

$$E: y^2 + xy = x^3 + Ax^2 + B,$$

la ley de grupo en $\mathbb{F}_{2^m}$ (coordenadas afines) se implementa así:

1. *Negativo:* $-P = (x, x + y)$.

2. *Suma* $P = (x_1, y_1)$, $Q = (x_2, y_2)$, $x_1 \neq x_2$. Sea

$$\lambda = \frac{y_1 + y_2}{x_1 + x_2}, \quad x_3 = \lambda^2 + \lambda + x_1 + x_2 + A, \quad y_3 = \lambda(x_1 + x_3) + x_3 + y_1.$$

Entonces $P + Q = (x_3, y_3)$.

3. *Duplicación* $P = (x_1, y_1)$, $x_1 \neq 0$. Sea

$$\lambda = x_1 + \frac{y_1}{x_1}, \quad x_3 = \lambda^2 + \lambda + A, \quad y_3 = x_1^2 + \lambda x_3 + x_3.$$

Entonces $2P = (x_3, y_3)$.

Para curvas supersingulares en $\mathbb{F}_{2^m}$ de la forma

$$E: y^2 + Cy = x^3 + Ax + B,$$

las fórmulas cambian levemente (con $-P = (x, y + C)$), pero siguen el mismo patrón de suma y duplicación mediante λ .

4.3. Curvas sobre cuerpos finitos

Cuando trabajamos sobre un cuerpo finito $\mathbb{F}_q$, la gráfica de una curva elíptica deja de ser una curva continua y pasa a ser simplemente un conjunto discreto de puntos como se puede apreciar en la figura 4.5. Sin embargo, el conjunto

$$E(\mathbb{F}_q) = \{(x, y) \in \mathbb{F}_q \times \mathbb{F}_q : y^2 + a_1xy + a_3y = x^3 + a_2x^2 + a_4x + a_6\} \cup \{\mathcal{O}\}$$

sigue formando un *grupo abeliano finito* bajo la ley de la cuerda y la tangente.

Ventajas de trabajar sobre $\mathbb{F}_q$.

- **Aritmética exacta:** Todas las sumas y multiplicaciones de coordenadas se hacen *módulo* q , con lo que no existen errores de redondeo ni acumulación de imprecisión.

- **Cierre y existencia de inversos:** En $\mathbb{F}_q$, todo elemento no nulo tiene inverso multiplicativo, lo que permite definir de forma sencilla la operación de división en las fórmulas de suma y de duplicación de puntos.
- **Rendimiento:** Los procesadores suelen integrar instrucciones nativas para operar *módulo* un primo p (por ejemplo, reducción rápida, multiplicadores de enteros), de modo que las operaciones en $\mathbb{F}_p$ resultan muy eficientes.
- **Seguridad:** El tamaño finito del grupo y su estructura abeliana permiten cuantificar con precisión la resistencia ante ataques de fuerza bruta o algoritmos de logaritmo discreto; elegir q y la curva de modo que $\#E(\mathbb{F}_q)$ (véase la sección 4.3) tenga un gran factor primo es esencial para garantizar la dificultad del problema.

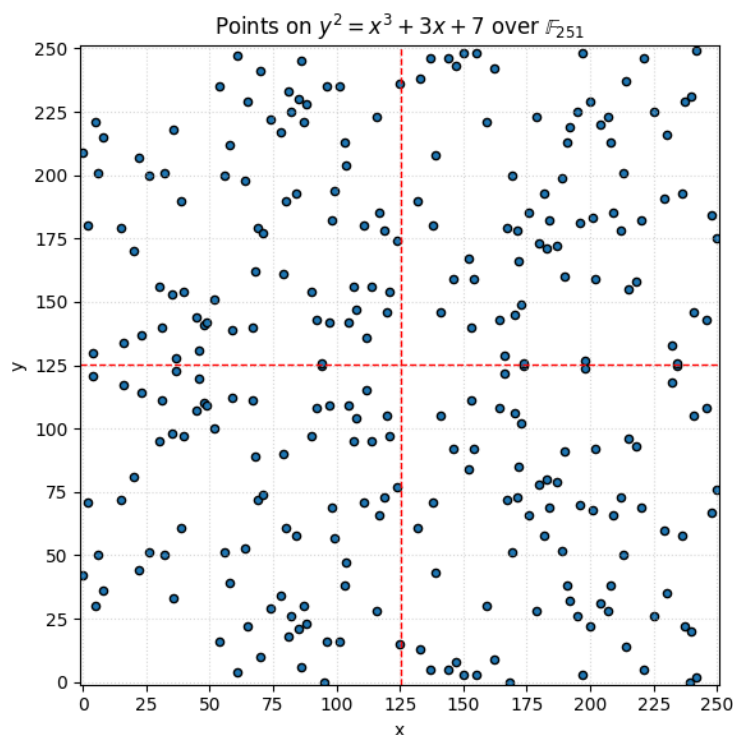


Figura 4.5: Ejemplo de los puntos de una curva elíptica sobre un cuerpo finito

Orden del grupo

Algo a destacar de las curvas elípticas sobre cuerpos finitos es el número de puntos racionales que forman. El valor del número de puntos es esencial

para determinar la dificultad de resolver el problema del logaritmo discreto en $E(\mathbb{F}_q)$ y garantiza la seguridad del sistema que depende de que ese valor tenga un factor primo grande.

Sea E una curva elíptica definida sobre el cuerpo finito $\mathbb{F}_q$. El número de puntos en $E(\mathbb{F}_q)$, denotado $\#E(\mathbb{F}_q)$, se llama *orden* de la curva sobre $\mathbb{F}_q$.

Como la ecuación de Weierstrass

$$y^2 + a_1xy + a_3y = x^3 + a_2x^2 + a_4x + a_6$$

puede tener a lo sumo dos soluciones en y para cada $x \in \mathbb{F}_q$, sabemos trivialmente que

$$1 \leq \#E(\mathbb{F}_q) \leq 2q + 1.$$

El teorema de Hasse ofrece cotas mucho más ajustadas:

Teorema 4.1 (Hasse). *Sea E una curva elíptica definida sobre $\mathbb{F}_q$. Entonces*

$$q + 1 - 2\sqrt{q} \leq \#E(\mathbb{F}_q) \leq q + 1 + 2\sqrt{q}.$$

El intervalo

$$[q + 1 - 2\sqrt{q}, q + 1 + 2\sqrt{q}]$$

se conoce como el *intervalo de Hasse*. Una formulación equivalente dice que

$$\#E(\mathbb{F}_q) = q + 1 - t, \quad |t| \leq 2\sqrt{q},$$

donde t se denomina *traza* de la curva E sobre $\mathbb{F}_q$. Dado que $2\sqrt{q} \ll q$ para cuerpos grandes, se tiene

$$\#E(\mathbb{F}_q) \approx q,$$

lo cual es fundamental para la seguridad criptográfica (pues el tamaño del grupo resulta comparable al del cuerpo base).

Órdenes admisibles

El siguiente resultado caracteriza exactamente qué números pueden ser órdenes de curvas sobre $\mathbb{F}_q$:

Teorema 4.2 (Órdenes admisibles). *Sea $q = p^m$ con p primo. Existe una curva elíptica $E/\mathbb{F}_q$ con $\#E(\mathbb{F}_q) = q + 1 - t$ si y solo si se cumple una de las condiciones:*

1. $t \equiv 0 \pmod{p}$ y $t^2 \leq 4q$.
2. m es impar y, además, $t = 0$; o $t^2 = 2q$ si $p = 2$; o $t^2 = 3q$ si $p = 3$.
3. m es par y, además, $t^2 = 4q$; o $t^2 = q$ con $p \equiv 1 \pmod{3}$; o $t = 0$ con $p \equiv 1 \pmod{4}$.

Una consecuencia inmediata es que, para cada primo p y cualquier t con $|t| \leq 2\sqrt{p}$, existe una curva $E/\mathbb{F}_p$ de orden $\#E(\mathbb{F}_p) = p + 1 - t$.

Ejemplo. Consideremos la curva

$$E: y^2 = x^3 + ax + b \quad \text{sobre } \mathbb{F}_p, \quad p > 3.$$

Cada punto (x, y) se calcula resolviendo la ecuación en $\mathbb{F}_p$. Por ejemplo, para

$$p = 97, \quad a = 2, \quad b = 3,$$

podemos listar todos los $x \in \{0, 1, \dots, 96\}$, calcular $x^3 + 2x + 3$ (mód 97) y buscar las raíces cuadradas en $\mathbb{F}_{97}$. Al final obtendremos $\#E(\mathbb{F}_{97}) \approx 97$ puntos (incluido $\mathcal{O}$), que forman el grupo $(E(\mathbb{F}_{97}), +, \mathcal{O})$.

4.4. Representación de puntos y la ley de grupo

En §4.2.3 presentamos la ley de grupo para puntos en curvas elípticas en coordenadas afines. Sin embargo, las operaciones de suma y duplicación requieren invertir en el cuerpo base, lo cual puede ser costoso si la inversión es mucho más lenta que la multiplicación. Para reducir el número de inversiones es útil emplear *coordenadas proyectivas*, que representan cada punto por un triple $(X : Y : Z)$ en lugar de (x, y) . A continuación describimos esta representación de forma general y sus variantes más utilizadas en $\mathbb{F}_p$ y $\mathbb{F}_{2^m}$.

4.4.1. Coordenadas proyectivas: definición general

Sea K un cuerpo, y fijos $c, d \geq 1$. Definimos una relación de equivalencia en

$$K^3 \setminus \{(0, 0, 0)\},$$

donde

$$(X_1, Y_1, Z_1) \sim (X_2, Y_2, Z_2) \iff \exists \lambda \in K^\times : X_1 = \lambda^c X_2, Y_1 = \lambda^d Y_2, Z_1 = \lambda Z_2.$$

La clase de equivalencia $(X : Y : Z)$ se llama *punto proyectivo*. Si $Z \neq 0$ elegimos el representante único $(X/Z^c, Y/Z^d, 1)$, que corresponde al punto afín $(x, y) = (X/Z^c, Y/Z^d)$. Los puntos con $Z = 0$ forman la *recta en el infinito*, que en las curvas elípticas se identifica con el punto neutro $\mathcal{O}$.

Para pasar de la ecuación afín $y^2 = x^3 + ax + b$ a su forma proyectiva estándar (con $c = d = 1$), reemplazamos

$$x \mapsto \frac{X}{Z}, \quad y \mapsto \frac{Y}{Z},$$

llevando la curva a

$$Y^2 Z = X^3 + a X Z^2 + b Z^3,$$

que es homogénea de grado 3 en (X, Y, Z) .

4.4.2. Coordenadas en $\mathbb{F}_p$

Para curvas en forma corta de Weierstrass

$$E: y^2 = x^3 + ax + b, \quad a, b \in \mathbb{F}_p, \quad p > 3,$$

las coordenadas proyectivas más comunes son:

1. Coordenadas proyectivas estándar ($c = d = 1$)

■ *Representación:* $(X : Y : Z)$ con $x = X/Z$, $y = Y/Z$.

■ *Ecuación:*

$$E: Y^2Z = X^3 + aXZ^2 + bZ^3.$$

■ *Punto en el infinito:* $\mathcal{O} = (0 : 1 : 0)$.

■ *Opuesto:* $-(X : Y : Z) = (X : -Y : Z)$.

2. Coordenadas Jacobianas ($c = 2, d = 3$)

■ *Representación:* $(X : Y : Z)$ con $x = X/Z^2$, $y = Y/Z^3$.

■ *Ecuación:*

$$E: Y^2 = X^3 + aXZ^4 + bZ^6.$$

■ *Punto en el infinito:* $\mathcal{O} = (1 : 1 : 0)$.

■ *Opuesto:* $-(X : Y : Z) = (X : -Y : Z)$.

3. Coordenadas de Chudnovsky

■ *Representación extendida:* $(X : Y : Z : Z^2 : Z^3)$.

■ *Punto en el infinito:* $\mathcal{O} = (1 : 1 : 0 : 0 : 0)$.

■ *Opuesto:* $-(X : Y : Z : Z^2 : Z^3) = (X : -Y : Z : Z^2 : Z^3)$.

4.4.3. Coordenadas en $\mathbb{F}_{2^m}$

Para curvas binarias no supersingulares

$$E: y^2 + xy = x^3 + Ax^2 + B, \quad A, B \in \mathbb{F}_{2^m},$$

la inversión en el cuerpo puede ser muy cara en software, por lo que resulta útil trabajar en coordenadas proyectivas. A continuación se presentan las más comunes:

1. Coordenadas proyectivas estándar ($c = 1, d = 1$)

- *Correspondencia con afines:*

$$(X : Y : Z) \longleftrightarrow (x, y) = (X/Z, Y/Z), \quad Z \neq 0.$$

- *Ecuación homogénea:*

$$Y^2 Z + X Y Z = X^3 + A X^2 Z + B Z^3.$$

- *Punto en el infinito:* $\mathcal{O} = (0 : 1 : 0)$.

- *Negativo:*

$$-(X : Y : Z) = (X : X + Y : Z),$$

que corresponde a reflejar en la “línea” $y = x$.

2. Coordenadas Jacobianas ($c = 2, d = 3$)

- *Correspondencia con afines:*

$$(X : Y : Z) \longleftrightarrow (x, y) = (X/Z^2, Y/Z^3), \quad Z \neq 0.$$

- *Ecuación homogénea:*

$$Y^2 + X Y Z = X^3 + A X^2 Z^2 + B Z^6.$$

- *Punto en el infinito:* $\mathcal{O} = (1 : 1 : 0)$.

- *Negativo:*

$$-(X : Y : Z) = (X : X + Y : Z).$$

- *Ventaja principal:* Todas las fórmulas de suma y duplicación usan únicamente multiplicaciones y sumas en $\mathbb{F}_{2^m}$, sin inversiones explícitas, lo que reduce drásticamente el coste en implementaciones software sobre hardware general.

3. Coordenadas de López–Dahab (LD) ($c = 1, d = 2$)

- *Correspondencia con afines:*

$$(X : Y : Z) \longleftrightarrow (x, y) = (X/Z, Y/Z^2), \quad Z \neq 0.$$

- *Ecuación homogénea:*

$$Y^2 + X Y Z = X^3 Z + A X^2 Z^2 + B Z^4.$$

- *Punto en el infinito:* $\mathcal{O} = (1 : 0 : 0)$.

- *Negativo:*

$$-(X : Y : Z) = (X : X + Y : Z).$$

- *Balance de coste:*

- Las fórmulas de suma y duplicación requieren menos cuadrados que en Jacobianas, a costa de un ligero aumento en multiplicaciones.

4.4.4. Ventajas y desventajas

- **Sin inversiones:** el principal beneficio de las coordenadas proyectivas es evitar el paso costoso de la inversión en K , cambiándolo por un mayor número de multiplicaciones y cuadrados, mucho más rápidos en hardware y software moderno.
- **Redundancia:** el almacenamiento adicional (el parámetro Z y, en Chudnovsky, Z^2, Z^3) aumenta el uso de memoria y complica la normalización final si necesitamos el resultado en afín.
- **Flexibilidad:** diferentes elecciones (c, d) adaptan la curva a remates algorítmicos específicos (e.g. $a = -3$ permite optimizar el término $3X_1^2 + aZ_1^4$).

En la práctica, para curvas de uso criptográfico sobre primos grandes, las coordenadas de Jacobi con $a = -3$ son el estándar de facto, pues combinan rapidez en duplicación y sencillez de implementación.

4.5. Selección de parámetros

La elección de los parámetros del dominio de una curva elíptica —el cuerpo base $\mathbb{F}_q$, los coeficientes (a, b) (o (a, b, c) en característica 2), el punto generador G , su orden n y el cofactor h — determina simultáneamente la seguridad y el rendimiento del criptosistema resultante. Como esta selección está íntimamente ligada al uso criptográfico (tamaño de clave equivalente, resistencia frente al ECDLP, ataques especializados como Pohlig–Hellman, MOV/Frey–Rück o Smart–Semaev–Satoh), su discusión se difiere al capítulo 5: en particular, la sección 5.2.2 compara tamaños de clave entre RSA y ECC para distintos niveles de seguridad, la sección 5.2.3 enumera los requisitos formales que debe satisfacer un dominio seguro, y la sección 5.7.2 describe las curvas estandarizadas usadas en la práctica (NIST P-256, P-384, secp256k1, Curve25519, las cinco curvas binarias SEC 2 implementadas en este trabajo, etc.).

Capítulo 5

Criptografía de curvas elípticas (ECC)

5.1. Introducción y motivación

Los investigadores llevaban años buscando funciones más seguras para criptografía y en 1985 se propuso por primera vez el uso de curvas elípticas como base del problema del logaritmo discreto en un grupo [32, 38]. Se cree que las curvas elípticas ofrecen un nivel de seguridad equivalente al de RSA pero con tamaños de clave mucho menores. Para ilustrar la diferencia en términos intuitivos, comparemos la energía necesaria para romper los diferentes sistemas con la cantidad de agua que esa energía podría hervir [55, 34]:

- Romper una clave RSA de 228 bits requiere menos energía que la necesaria para hervir una cucharadita de agua.
- Romper una clave ECC de 228 bits equivaldría a la energía necesaria para hervir toda el agua de la Tierra.
- Para obtener el mismo nivel de seguridad con RSA haría falta una clave de unos 2380 bits, lo cual es muy ineficiente en entornos con recursos limitados.

En la figura 5.1 se muestra la jerarquía de la criptografía de las curvas elípticas, y cómo es necesario poder tener de base lo que hemos explicado en los capítulos anteriores.

5.1.1. Historia

En 1997 Certicom lanzó el *ECC Challenge* para evaluar la dificultad práctica del problema del logaritmo discreto en curvas elípticas [19]. Pocos años después, en 1998, el comité ANSI X9 publicó la norma X9.62 definiendo



Figura 5.1: Jerarquía de la criptografía de Curvas Elípticas

el algoritmo de firma ECDSA, marcando el primer estándar formal para firmas basadas en curvas elípticas [11]. En el año 2000, el NIST incorporó curvas elípticas en el FIPS 186-2 (Digital Signature Standard), consolidando su uso en aplicaciones gubernamentales y financieras [41].

En 2005 la NSA anunció la Suite B de algoritmos criptográficos, recomendando las curvas P-256, P-384 y P-521 para uso en comunicaciones seguras de nivel gubernamental [27]. Con la adopción de TLS 1.2 y la publicación del RFC 8422 en 2018, las curvas secp256r1 (también conocida como prime256v1) y X25519 se convirtieron en cimientos de la seguridad de Internet, al ofrecer un excelente equilibrio entre rendimiento y seguridad [29]. Más recientemente, TLS 1.3 (RFC 8446) ha consolidado aún más la posición de ECC al eliminar los *cipher suites* basados exclusivamente en RSA para el intercambio de claves, favoreciendo ECDHE como mecanismo predeterminado [54].

En la actualidad, la criptografía de curvas elípticas es omnipresente y continúa evolucionando. Sin embargo, no ha estado exenta de desafíos: se han descubierto ataques teóricos contra curvas mal escogidas, y hubo polémicas como la del generador Dual-EC-DRBG (que involucraba curvas elípticas y una posible puerta trasera). Aun así, las curvas bien elegidas siguen considerándose seguras. Un punto a futuro es la computación cuántica: si llegaran a construirse ordenadores cuánticos a gran escala, los algoritmos de Shor podrían resolver el ECDLP eficientemente, rompiendo ECC al igual que RSA. Por ello, existe una carrera por desarrollar criptografía poscuántica. Pero hasta la fecha, ECC sigue siendo segura frente a ataques clásicos conocidos y representa uno de los pilares de la criptografía de clave pública contemporánea.

5.1.2. Aplicaciones

Gracias a sus ventajas en seguridad y eficiencia, ECC se ha incorporado en numerosas aplicaciones modernas. A continuación destacamos algunas de las más relevantes:

Blockchain y Criptomonedas: Las curvas elípticas son la columna vertebral de la seguridad en blockchain. Por ejemplo, Bitcoin y Ethereum emplean la curva estándar secp256k1 junto con el algoritmo de firma digital ECDSA para gestionar las claves públicas y firmas de transacciones [72]. Esto asegura que solo el poseedor de la clave privada correspondiente pueda autorizar movimientos de fondos.

Certificados Digitales y TLS/SSL: ECC se utiliza en certificados X.509 para establecer identidades seguras en Internet. Muchas Autoridades de Certificación ofrecen certificados TLS basados en curvas elípticas (ECDSA o ECDH) como alternativa a los tradicionales RSA. La principal ventaja es que un certificado ECC puede lograr el mismo nivel de seguridad que RSA con claves de menor tamaño, acelerando el *handshake* SSL/TLS y reduciendo la carga en servidores y clientes [26, 28]. Por ejemplo, un servidor web con un certificado ECDSA de 256 bits ofrece 128 bits de seguridad, equivalente a un RSA de 3072 bits [23].

Sistemas de Mensajería y Cifrado de Extremo a Extremo (E2EE):

Protocolos modernos de mensajería segura, como Signal, WhatsApp o el protocolo de doble-ratchet de Olvid, utilizan intercambios de clave basados en curvas elípticas (por ejemplo, X25519, que es una curva elíptica de Montgomery). Estas aplicaciones aprovechan ECC para generar secretos compartidos de forma rápida y con alta seguridad en cada sesión de chat.

Firmas Digitales y Autenticación: Más allá de los certificados TLS, ECC sustenta esquemas de firma digital ampliamente usados, como ECDSA y variantes más recientes como EdDSA (por ejemplo Ed25519). Estos se emplean en autenticación de documentos, en el DNIE de algunos países, en pasaportes biométricos, en tokens de hardware, etc. Por su eficiencia, ECC permite firmas y verificaciones rápidas incluso en dispositivos de recursos limitados (tarjetas inteligentes, dispositivos embebidos).

IoT y Dispositivos Embebidos: En el Internet de las Cosas, donde muchos dispositivos tienen poca potencia de cómputo y batería limitada, ECC ha ganado protagonismo [24]. La posibilidad de usar claves más cortas y operaciones más eficientes hace viable implementar criptografía robusta en sensores, actuadores y *wearables*.

Además, las propiedades algebraicas de las curvas han sido la base para esquemas avanzados como sistemas de emparejamientos (*pairing-based cryptography*), identidades basadas en curva (IBE) y firmas Schnorr y EdDSA, ampliando enormemente el catálogo de construcciones criptográficas eficientes [15].

5.2. Fundamentos de ECC

5.2.1. El problema del logaritmo discreto en curvas elípticas (ECDLP)

La seguridad de todos los protocolos basados en curvas elípticas (ECDH, ECDSA, etc.) reposa sobre la dificultad computacional de un único problema: el *problema del logaritmo discreto en curvas elípticas*.

Definición 5.1 (ECDLP). Sea E una curva elíptica definida sobre un cuerpo finito $\mathbb{F}_q$ y sea $P \in E(\mathbb{F}_q)$ un punto de orden primo n . Dado un punto $Q \in \langle P \rangle$ (el subgrupo cíclico generado por P), el problema del logaritmo discreto en curvas elípticas (ECDLP) consiste en encontrar el entero d , con $0 \leq d \leq n - 1$, tal que

$$Q = d \cdot P = \underbrace{P + P + \cdots + P}_{d \text{ veces}}.$$

La operación $d \cdot P$ (multiplicación escalar) es eficiente: usando el algoritmo *double-and-add* se calcula en $O(\log d)$ operaciones de grupo, es decir, $O(\log n)$ sumas y duplicaciones de puntos en la curva. Sin embargo, la operación inversa —dado Q y P , encontrar d — no admite ningún algoritmo eficiente conocido en el caso general.

Comparación con el DLP en $\mathbb{F}_p^*$. En el grupo multiplicativo $\mathbb{F}_p^*$, el problema del logaritmo discreto clásico (dado g y $h = g^a$ mód p , encontrar a) admite algoritmos sub-exponenciales como el *index calculus*, cuya complejidad es $L_p\left[\frac{1}{3}, c\right] = e^{c(\ln p)^{1/3}(\ln \ln p)^{2/3}}$. Estos métodos explotan la estructura aritmética de $\mathbb{Z}/p\mathbb{Z}$ para “factorizar” elementos del grupo sobre una base de factores.

En curvas elípticas, no se conoce una versión efectiva del *index calculus* para curvas generales sobre cuerpos primos: los mejores algoritmos conocidos son genéricos (es decir, se aplican a cualquier grupo cíclico) y no explotan ninguna estructura específica de la curva. Los más relevantes son:

- **Baby-step/Giant-step** (Shanks): complejidad $O(\sqrt{n})$ en tiempo y espacio.
- **Rho de Pollard**: complejidad $O(\sqrt{n})$ en tiempo con espacio $O(1)$.

Ambos son de complejidad *exponencial completa* respecto al tamaño de la entrada ($\log n$ bits). Es decir, para una curva de k bits de seguridad, el mejor ataque conocido requiere $O(2^{k/2})$ operaciones. Esto implica que una curva de 256 bits ofrece aproximadamente 128 bits de seguridad, mientras que un RSA de 3072 bits es necesario para alcanzar ese mismo nivel [23, 47].

Baby-step / Giant-step (Shanks)

El algoritmo de Shanks [23] se basa en una idea elegante: descomponer el escalar desconocido d en dos mitades y buscar una colisión entre dos conjuntos precalculados.

Idea. Si escribimos $d = j \cdot m + i$ donde $m = \lceil \sqrt{n} \rceil$, con $0 \leq i < m$ y $0 \leq j < m$, entonces

$$Q = dP = (jm + i)P = jmP + iP \implies Q - j(mP) = iP.$$

El lado izquierdo depende solo de j (“giant steps”) y el derecho solo de i (“baby steps”). Si precalculamos todos los valores iP y los almacenamos en una tabla hash, basta recorrer los giant steps hasta encontrar una coincidencia.

Algorithm 3 Baby-step / Giant-step para el ECDLP

Require: Puntos $P, Q \in E(\mathbb{F}_q)$ con $Q = dP$, orden $n = |P|$

Ensure: Entero d tal que $Q = dP$

```

1:  $m \leftarrow \lceil \sqrt{n} \rceil$ 
2:                                     ▷ Baby steps: precalcular  $iP$  para  $i = 0, 1, \dots, m-1$ 
3: for  $i = 0$  to  $m-1$  do
4:   Almacenar  $(iP, i)$  en tabla hash
5: end for
6:                                     ▷ Giant steps: buscar colisión
7:  $S \leftarrow Q$ 
8: for  $j = 0$  to  $m-1$  do
9:   if  $S$  está en la tabla hash con valor  $i$  then
10:    return  $d = jm + i \bmod n$ 
11:   end if
12:    $S \leftarrow S - mP$                                      ▷ Equivalente a  $Q - (j+1)mP$ 
13: end for

```

Complejidad. La fase de baby steps realiza m multiplicaciones escalares y requiere $O(m)$ espacio para la tabla hash. La fase de giant steps realiza como máximo m sumas de puntos. El coste total es $O(m) = O(\sqrt{n})$ tanto en tiempo como en espacio. Para una curva de 256 bits, esto supone almacenar $\sim 2^{128}$ puntos, lo que es inviable con la tecnología actual.

Ejemplo 5.1. Sea la curva $E : y^2 = x^3 + 2x + 3$ sobre $\mathbb{F}_{97}$, con generador $P = (0, 10)$ de orden $n = 50$ y objetivo $Q = (92, 81) = 16P$.

Calculamos $m = \lceil \sqrt{50} \rceil = 8$. Los baby steps son:

i	iP
0	$\mathcal{O}$
1	(0, 10)
2	(65, 32)
3	(23, 24)
4	(52, 68)
5	(88, 56)
6	(95, 66)
7	(10, 76)

Para los giant steps, calculamos $8P = (84, 37)$ y recorremos $S_j = Q - j \cdot 8P$ para $j = 0, 1, 2, \dots$ hasta encontrar una coincidencia con la tabla de baby steps. Concretamente, $S_0 = (92, 81)$ y $S_1 = (84, 37)$, ninguno de los cuales aparece en la tabla; en cambio, $S_2 = Q - 16P = \mathcal{O}$ coincide con la entrada $i = 0$. La colisión se produce, por tanto, para $j = 2$, $i = 0$, dando $d = 2 \cdot 8 + 0 = 16$, como cabía esperar de la construcción $Q = 16P$.

Método rho de Pollard

El rho de Pollard [52] alcanza la misma complejidad temporal que BSGS ($O(\sqrt{n})$) pero con requerimiento de memoria constante $O(1)$, lo que lo convierte en el ataque más práctico contra el ECDLP [23]. Su nombre proviene de la forma de la letra griega ρ que adopta la secuencia de puntos generada por el algoritmo.

La paradoja del cumpleaños. Antes de describir el algoritmo, conviene entender la observación probabilística que lo fundamenta. Consideremos la siguiente pregunta: ¿cuántas personas deben reunirse en una sala para que haya al menos un 50 % de probabilidad de que dos de ellas compartan cumpleaños?

Intuitivamente, como hay 365 días posibles, podríamos pensar que hacen falta muchas personas. Sin embargo, la respuesta es sorprendentemente baja: **basta con 23 personas**. Este resultado, conocido como la *paradoja del cumpleaños*, se explica porque no buscamos que alguien comparta cumpleaños con una persona concreta, sino que *cualquier par* de personas coincida. Con k personas, el número de pares posibles es $\binom{k}{2} = k(k-1)/2$, que crece cuadráticamente.

La probabilidad exacta de que al menos dos personas de un grupo de k compartan cumpleaños (suponiendo $N = 365$ días equiprobables) se calcula como el complemento de que *todos* tengan cumpleaños distintos:

$$P(k) = 1 - \prod_{i=0}^{k-1} \frac{N-i}{N} = 1 - \frac{N}{N} \cdot \frac{N-1}{N} \cdot \frac{N-2}{N} \cdots \frac{N-k+1}{N}.$$

El cuadro 5.1 muestra los valores concretos.

Personas (k)	$P(\text{coincidencia})$
5	2.7 %
10	11.7 %
20	41.1 %
23	50.7 %
30	70.6 %
40	89.1 %
50	97.0 %
57	99.0 %
70	99.9 %

Cuadro 5.1: Probabilidad de que al menos dos personas compartan cumpleaños en un grupo de k personas ($N = 365$ días).

El umbral del 50 % se alcanza con $k = 23 \approx \sqrt{\pi \cdot 365/2} \approx 1,25\sqrt{365}$. Esto no es casualidad: mediante una aproximación estándar ($\ln(1-x) \approx -x$ para x pequeño), se demuestra que el número esperado de elementos que debemos elegir al azar de un conjunto de N para encontrar una colisión es

$$k \approx \sqrt{\frac{\pi N}{2}} \approx 1,25\sqrt{N}. \quad (5.1)$$

Nótese que este resultado es *independiente* de qué elemento concreto se repita: solo depende del tamaño N del conjunto.

De cumpleaños a curvas elípticas. El ataque rho de Pollard traslada esta observación al grupo de puntos de una curva elíptica. Si el generador P tiene orden n , el subgrupo $\langle P \rangle$ contiene exactamente n puntos distintos. Si generamos una secuencia de puntos pseudoaleatorios en este grupo, por la cota (5.1) esperamos encontrar una *colisión* (dos índices $i \neq j$ con $x_i = x_j$) tras aproximadamente $\sqrt{\pi n/2} \approx 1,25\sqrt{n}$ pasos. Para una curva de 256 bits ($n \approx 2^{256}$), esto supone $\approx 2^{128}$ pasos, que es precisamente la seguridad que atribuimos a dicha curva. El cuadro 5.2 muestra esta correspondencia.

Pero la paradoja del cumpleaños, por sí sola, solo dice cuántos pasos necesitamos para una colisión. No dice cómo *detectar* esa colisión eficientemente (sin almacenar los $\sqrt{n}$ puntos previos, lo cual requeriría tanto espacio como BSGS), ni cómo *extraer* el logaritmo discreto de ella. Para esto se necesitan dos ingredientes: un *paseo pseudoaleatorio con invariante* y la *detección de ciclos de Floyd*.

El paseo pseudoaleatorio y el invariante. La idea clave de Pollard es construir la secuencia de puntos de forma *determinista* pero que se *comporte* como aleatoria, manteniendo en todo momento un invariante que permita

Curva (bits)	Orden $n \approx$	Pasos esperados $\approx$
160	2^{160}	2^{80}
224	2^{224}	2^{112}
256	2^{256}	2^{128}
384	2^{384}	2^{192}
521	2^{521}	2^{260}

Cuadro 5.2: Número esperado de pasos del rho de Pollard según el tamaño de la curva. Los bits de seguridad son exactamente la mitad de los bits de la curva.

recuperar d al encontrar una colisión. Cada punto R_i de la secuencia se escribe como $R_i = a_iP + b_iQ$ donde los coeficientes a_i, b_i son conocidos.

El siguiente punto se calcula aplicando una regla que depende solo del punto actual, particionando el grupo en tres subconjuntos según la coordenada x módulo 3:

Partición	Condición	Nuevo punto	Nuevo a	Nuevo b
S_1	$x_R \equiv 0 \pmod{3}$	$R + Q$	a	$b + 1$
S_2	$x_R \equiv 1 \pmod{3}$	$2R$	$2a$	$2b$
S_3	$x_R \equiv 2 \pmod{3}$	$R + P$	$a + 1$	b

Todos los cálculos de coeficientes se realizan módulo n . En los tres casos se preserva el invariante: si $R = aP + bQ$ antes del paso, el nuevo punto R' también se expresa como $R' = a'P + b'Q$ con los coeficientes actualizados según la tabla.

¿Por qué estas tres operaciones? La elección no es arbitraria. Las tres reglas cumplen propiedades esenciales:

- **Preservan el invariante.** Cada regla actualiza (a, b) de forma que $R' = a'P + b'Q$ sigue siendo cierto. Esto es imprescindible para poder extraer d al final.
- **La partición debe ser aproximadamente equilibrada.** Si los tres subconjuntos S_1, S_2, S_3 contienen aproximadamente $n/3$ puntos cada uno, el paseo se comporta como una función aleatoria sobre el grupo. Usar $x_R \pmod{3}$ produce una distribución razonablemente uniforme para la mayoría de las curvas.
- **La duplicación (S_2) aporta “mezcla”.** Si solo sumásemos P o Q , el paseo sería demasiado regular (una progresión aritmética en los coeficientes). La duplicación introduce no linealidad en los coeficientes ($a \mapsto 2a, b \mapsto 2b$), haciendo que la secuencia “salte” por el grupo de forma menos predecible.

El punto inicial $R_0 = a_0P + b_0Q$ puede ser cualquier punto con coeficientes conocidos. La elección $a_0 = 1$, $b_0 = 0$ (es decir, $R_0 = P$) es simplemente la más sencilla.

Nota sobre cuerpos binarios. El algoritmo es *idéntico* para curvas sobre $\mathbb{F}_{2^m}$: la partición se basa igualmente en la coordenada x del punto, las tres reglas $(+Q, 2R, +P)$ utilizan la aritmética de grupo de la curva (que en $\mathbb{F}_{2^m}$ emplea las fórmulas de la sección 5.6.2), y los coeficientes (a, b) se actualizan módulo n de la misma manera. La única diferencia es que las operaciones de cuerpo subyacentes (suma = XOR, multiplicación de polinomios) son distintas, pero esto es transparente al nivel del algoritmo de Pollard.

Estructura ρ . Dado que el grupo $\langle P \rangle$ es finito, la secuencia $x_0, x_1, x_2, \dots$ necesariamente revisita algún punto, formando una cola de longitud λ seguida de un ciclo de longitud μ : se cumple $x_{\lambda+\mu} = x_\lambda$. Esta estructura se ilustra en la figura 5.2.

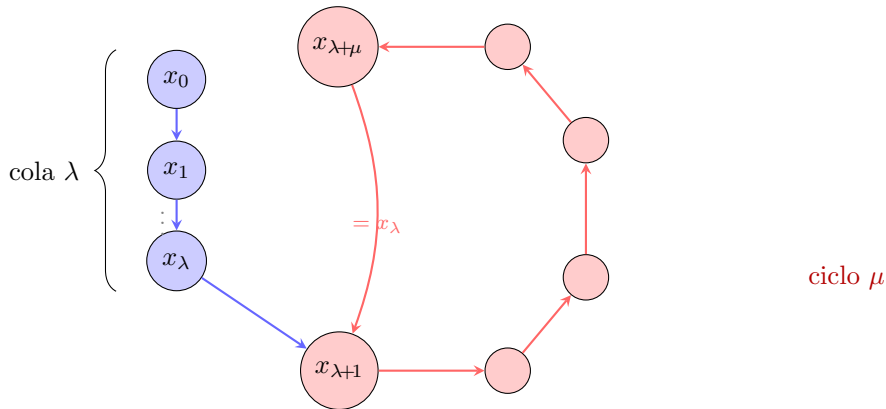


Figura 5.2: Estructura ρ de la secuencia. Los primeros λ puntos (azul) forman la cola. A partir de x_λ la secuencia entra en un ciclo de longitud μ (rojo), cumpliéndose $x_{\lambda+\mu} = x_\lambda$.

Detección de ciclos: tortuga y liebre de Floyd. Para detectar cuándo la secuencia se repite sin almacenar todos los puntos visitados, se emplean dos iteradores que recorren el *mismo* paseo a velocidades distintas. Denotamos por f la aplicación de un paso del paseo (elegir partición según x_R mód 3, actualizar punto y coeficientes). Entonces:

$$\text{Tortuga: } t_i = f(t_{i-1}) = x_i, \quad (5.2)$$

$$\text{Liebre: } \ell_i = f(f(\ell_{i-1})) = x_{2i}. \quad (5.3)$$

La tortuga aplica f una vez por iteración (avanza un paso en la secuencia) y la liebre aplica f dos veces (avanza dos pasos). En cada iteración i se comprueba si $t_i = \ell_i$. Puesto que la liebre recorre la secuencia al doble de velocidad, eventualmente “alcanzará” a la tortuga dentro del ciclo.

Extracción del logaritmo. Cuando se produce la colisión $t_i = \ell_i$, la tortuga y la liebre se encuentran en el mismo punto de la curva, pero llegaron por caminos distintos y por tanto tienen coeficientes potencialmente diferentes:

$$a_t P + b_t Q = a_\ell P + b_\ell Q.$$

Sustituyendo $Q = dP$:

$$(a_t - a_\ell)P = (b_\ell - b_t)dP \implies a_t - a_\ell \equiv (b_\ell - b_t) \cdot d \pmod{n}.$$

Si $\gcd(b_\ell - b_t, n) = 1$ (lo cual ocurre con alta probabilidad cuando n es primo), se despeja:

$$d = (a_t - a_\ell) \cdot (b_\ell - b_t)^{-1} \pmod{n}. \quad (5.4)$$

Pseudocódigo. El algoritmo 4 detalla el procedimiento completo con la lógica de partición expandida, mostrando explícitamente cómo la tortuga aplica un paso y la liebre dos en cada iteración.

Ejemplo numérico. Consideremos la curva $E: y^2 = x^3 + 2x + 1$ sobre $\mathbb{F}_{89}$, con punto generador $P = (1, 2)$ de orden primo $n = 47$. Sea $Q = (87, 73) = 15P$ el punto cuyo logaritmo discreto queremos encontrar.

Ambos iteradores parten de $t_0 = \ell_0 = P = (1, 2)$ con coeficientes $(a, b) = (1, 0)$. En cada iteración, la tortuga aplica un paso del paseo y la liebre aplica dos. La siguiente tabla muestra el desarrollo completo:

i	Tortuga $t_i = x_i$				Liebre $\ell_i = x_{2i}$			$t_i = \ell_i?$
	Punto	a_t	b_t	Regla	Punto	a_ℓ	b_ℓ	
0	(1, 2)	1	0		(1, 2)	1	0	
1	(83, 29)	2	0	S_2	(78, 28)	3	0	No
2	(78, 28)	3	0	S_3	(10, 24)	4	1	No
3	(38, 83)	3	1	S_1	(42, 83)	9	2	No
4	(10, 24)	4	1	S_3	(3, 52)	18	6	No
5	(80, 77)	8	2	S_2	(40, 1)	19	7	No
6	(42, 83)	9	2	S_3	(18, 66)	29	28	No
7	(4, 47)	9	3	S_1	(84, 69)	11	11	No
8	(3, 52)	18	6	S_2	(38, 83)	11	13	No
9	(38, 6)	18	7	S_1	(80, 77)	24	26	No
10	(40, 1)	19	7	S_3	(4, 47)	25	27	No
11	(13, 34)	38	14	S_2	(38, 6)	3	8	No
12	(18, 66)	29	28	S_2	(13, 34)	8	16	No
13	(64, 48)	29	29	S_1	(64, 48)	16	33	Sí

Algorithm 4 Método ρ de Pollard para el ECDLP**Require:** Puntos $P, Q \in E(\mathbb{F}_q)$ con $Q = dP$, orden primo $n = |P|$ **Ensure:** Entero d tal que $Q = dP$

```

1:  $t \leftarrow P$ ;  $a_t \leftarrow 1$ ;  $b_t \leftarrow 0$  ▷ Tortuga:  $t = 1 \cdot P + 0 \cdot Q$ 
2:  $\ell \leftarrow P$ ;  $a_\ell \leftarrow 1$ ;  $b_\ell \leftarrow 0$  ▷ Liebre: igual inicio
3: repeat
4: ▷ — Tortuga: un paso —
5:   if  $x_t \equiv 0 \pmod{3}$  then
6:      $t \leftarrow t + Q$ ;  $b_t \leftarrow b_t + 1$ 
7:   else if  $x_t \equiv 1 \pmod{3}$  then
8:      $t \leftarrow 2t$ ;  $a_t \leftarrow 2a_t$ ;  $b_t \leftarrow 2b_t$ 
9:   else
10:     $t \leftarrow t + P$ ;  $a_t \leftarrow a_t + 1$ 
11:   end if
12: ▷ — Liebre: dos pasos —
13:   for  $j = 1$  to  $2$  do
14:     if  $x_\ell \equiv 0 \pmod{3}$  then
15:        $\ell \leftarrow \ell + Q$ ;  $b_\ell \leftarrow b_\ell + 1$ 
16:     else if  $x_\ell \equiv 1 \pmod{3}$  then
17:        $\ell \leftarrow 2\ell$ ;  $a_\ell \leftarrow 2a_\ell$ ;  $b_\ell \leftarrow 2b_\ell$ 
18:     else
19:        $\ell \leftarrow \ell + P$ ;  $a_\ell \leftarrow a_\ell + 1$ 
20:     end if
21:   end for
22: ▷ Todos los coeficientes se reducen módulo  $n$ 
23: until  $t = \ell$ 
24: if  $b_\ell \equiv b_t \pmod{n}$  then
25:   return Fallo (reintentar con otro punto inicial)
26: else
27:   return  $d \leftarrow (a_t - a_\ell) \cdot (b_\ell - b_t)^{-1} \pmod{n}$ 
28: end if

```

Veamos en detalle cómo se obtiene la fila $i = 1$. La tortuga parte de $t_0 = (1, 2)$; como $x_{t_0} = 1 \equiv 1 \pmod{3}$, estamos en S_2 (duplicar), así que $t_1 = 2 \cdot (1, 2) = (83, 29)$ y los coeficientes se duplican: $(a_t, b_t) = (2, 0)$. La liebre parte del mismo punto pero aplica el paseo *dos veces*: primero obtiene $(83, 29)$ (igual que la tortuga), y después, como $83 \equiv 2 \pmod{3}$ (S_3), suma P : $\ell_1 = (83, 29) + P = (78, 28)$ con $(a_\ell, b_\ell) = (2 + 1, 0) = (3, 0)$. La liebre ya está en x_2 de la secuencia mientras la tortuga está en x_1 .

En el paso $i = 13$ se produce la colisión: $t_{13} = \ell_{13} = (64, 48)$. Los coeficientes son:

$$\text{Tortuga: } (64, 48) = 29P + 29Q, \quad \text{Liebre: } (64, 48) = 16P + 33Q.$$

Aplicando la ecuación (5.4):

$$\begin{aligned} a_t - a_\ell &= 29 - 16 = 13 \pmod{47}, \\ b_\ell - b_t &= 33 - 29 = 4 \pmod{47}, \\ 4^{-1} &\equiv 12 \pmod{47} \quad (\text{pues } 4 \cdot 12 = 48 \equiv 1 \pmod{47}), \\ d &= 13 \cdot 12 \pmod{47} = 156 \pmod{47} = \mathbf{15}. \end{aligned}$$

Verificamos: $15 \cdot P = 15 \cdot (1, 2) = (87, 73) = Q$. □

La secuencia pura generada por el paseo tiene cola $\lambda = 2$ (los puntos x_0, x_1 no se repiten) y ciclo $\mu = 13$ (desde x_2 hasta x_{14} , con $x_{15} = x_2$). La colisión de Floyd se produce en el paso 13 porque es el primer momento en que la tortuga (en x_{13}) y la liebre (en x_{26} , que por el ciclo coincide con x_{13}) se encuentran en el mismo punto. La figura 5.3 muestra la visualización de este proceso.

Complejidad. El número esperado de iteraciones hasta la colisión es $O(\sqrt{n})$ por la paradoja del cumpleaños (cuadro 5.2). Cada iteración consiste en una operación de grupo para la tortuga y dos para la liebre. Crucialmente, el método de Floyd solo almacena dos puntos y sus coeficientes en cada momento: espacio $O(1)$. Esto supone una ventaja decisiva frente a BSGS, que necesita $O(\sqrt{n})$ espacio para su tabla hash.

Para una curva de 256 bits con $n \approx 2^{256}$, el ataque requiere $\approx 2^{128}$ operaciones, fuera del alcance computacional actual. El récord práctico fue la resolución de la curva ECC2K-130 (130 bits sobre cuerpo binario) en 2009, tras un esfuerzo de cálculo distribuido masivo [19].

Observación 5.1. *El rho de Pollard es un algoritmo genérico: no explota ninguna propiedad específica de las curvas elípticas, solo la estructura de grupo finito cíclico. Funciona de forma idéntica sobre cuerpos primos $\mathbb{F}_p$ y cuerpos binarios $\mathbb{F}_{2^m}$, ya que solo depende de la operación de grupo (suma de puntos), no de la aritmética del cuerpo subyacente. Esta genericidad es, paradójicamente, la razón por la que ECC es segura: a diferencia de los ataques*

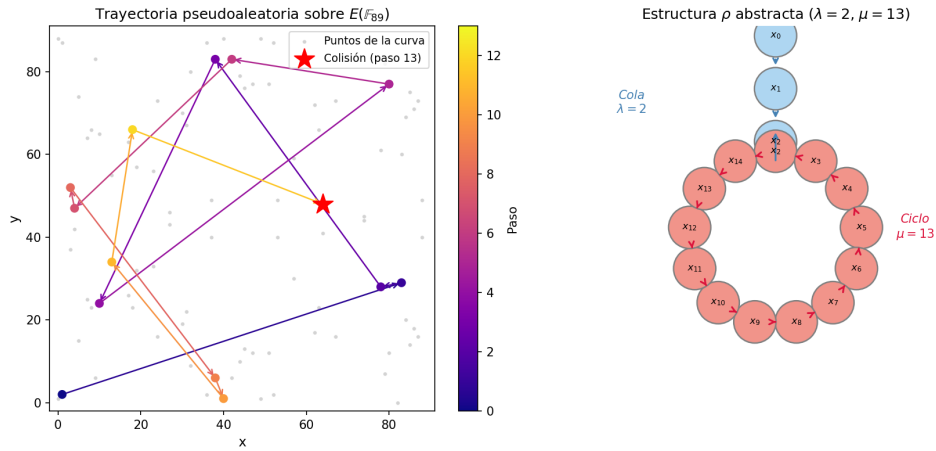


Figura 5.3: Visualización del rho de Pollard sobre la curva $y^2 = x^3 + 2x + 1 \pmod{89}$. Panel izquierdo: trayectoria pseudoaleatoria de la tortuga sobre la curva, coloreada por orden de aparición; la estrella marca la colisión en el paso 13. Panel derecho: estructura abstracta de ρ mostrando la cola ($\lambda = 2$, azul) y el ciclo ($\mu = 13$, rojo).

index calculus contra el DLP en $\mathbb{F}_p^$ (que explotan la estructura aritmética del cuerpo), no se conoce forma de acelerar el ataque en curvas elípticas más allá de $O(\sqrt{n})$.*

5.2.2. Comparativa de tamaños de clave

La ausencia de ataques sub-exponenciales en curvas elípticas tiene una consecuencia práctica directa: ECC necesita claves drásticamente más pequeñas que RSA para ofrecer el mismo nivel de seguridad. El cuadro 5.3 resume las equivalencias según el NIST SP 800-57 [47].

Seguridad (bits)	RSA (bits)	ECC (bits)	Relación RSA/ECC
80	1024	160	6.4:1
112	2048	224	9.1:1
128	3072	256	12:1
192	7680	384	20:1
256	15360	521	29.5:1

Cuadro 5.3: Equivalencia de niveles de seguridad entre RSA y ECC según NIST SP 800-57.

La relación RSA/ECC *crece* a medida que aumenta el nivel de seguridad deseado. Esto se debe a que la complejidad del mejor ataque a RSA (criba

del cuerpo numérico general, sub-exponencial) escala peor que la del mejor ataque a ECC (Pollard rho, exponencial). En la práctica, esto se traduce en:

- Menor consumo de ancho de banda (certificados TLS más pequeños).
- Generación de claves más rápida (generar un escalar aleatorio de 256 bits es trivial comparado con encontrar dos primos de 1536 bits).
- Operaciones criptográficas más eficientes, especialmente relevante en dispositivos con recursos limitados (IoT, tarjetas inteligentes).

5.2.3. Parámetros del dominio de una curva elíptica

Para instanciar un criptosistema basado en curvas elípticas, todas las partes involucradas deben acordar un conjunto de *parámetros del dominio*. En el caso de curvas sobre cuerpos primos $\mathbb{F}_p$, estos parámetros se definen como la séxtupla

$$T = (p, a, b, G, n, h),$$

donde:

- p es un primo que define el cuerpo finito $\mathbb{F}_p$.
- $a, b \in \mathbb{F}_p$ son los coeficientes de la ecuación de Weierstrass $y^2 = x^3 + ax + b$.
- $G = (G_x, G_y) \in E(\mathbb{F}_p)$ es el *punto generador* (o *base point*).
- n es el *orden* de G , es decir, el menor entero positivo tal que $n \cdot G = \mathcal{O}$.
- $h = \#E(\mathbb{F}_p)/n$ es el *cofactor*.

Para que los parámetros proporcionen seguridad criptográfica adecuada, deben satisfacer varios requisitos [64, 23]:

1. n debe ser primo (o al menos tener un factor primo muy grande). Esto garantiza que el subgrupo $\langle G \rangle$ sea cíclico de orden primo, eliminando ataques basados en la estructura de subgrupos (Pohlig–Hellman).
2. El cofactor h debe ser pequeño (idealmente $h = 1$). Si $h > 1$, existen puntos de orden bajo que podrían utilizarse en ataques de curva inválida (*invalid curve attacks*).
3. La curva no debe ser *anómala*: $\#E(\mathbb{F}_p) \neq p$. Las curvas anómalas son vulnerables al ataque de Smart–Semaev–Satoh, que resuelve el ECDLP en tiempo lineal mediante un levantamiento p -ádico [62].

4. La curva no debe tener *grado de inmersión* bajo: $p^k \not\equiv 1 \pmod{n}$ para k pequeño (típicamente $k > 20$). Las curvas con grado de inmersión bajo son vulnerables a ataques de emparejamiento de Weil y Tate (ataque MOV/Frey–Rück), que reducen el ECDLP a un DLP en $\mathbb{F}_{p^k}^*$ [35].
5. El discriminante de la curva debe ser no nulo: $\Delta = -16(4a^3 + 27b^2) \neq 0$.

5.2.4. Generación y verificación de parámetros de dominio

Los parámetros del dominio $T = (p, a, b, G, n, h)$ pueden obtenerse de dos formas: adoptando una curva estandarizada (como hacemos en nuestra implementación) o generando parámetros nuevos siguiendo un procedimiento riguroso. En ambos casos, es fundamental verificar que los parámetros satisfacen los requisitos de seguridad. A continuación describimos ambos procesos, siguiendo la exposición de Hankerson, Menezes y Vanstone [23].

Generación de parámetros sobre $\mathbb{F}_p$

El proceso de generación de una curva elíptica segura sobre un cuerpo primo $\mathbb{F}_p$ consta de los siguientes pasos:

1. **Selección del primo p .** Se elige un primo p de tamaño adecuado al nivel de seguridad deseado (por ejemplo, 256 bits para 128 bits de seguridad). Para eficiencia, se prefieren primos con estructura especial que facilite la reducción modular, como los *primos pseudo-Mersenne* usados por el NIST (e.g., $p_{256} = 2^{256} - 2^{224} + 2^{192} + 2^{96} - 1$).
2. **Selección de los coeficientes a y b .** Se eligen $a, b \in \mathbb{F}_p$ tales que el discriminante $\Delta = -16(4a^3 + 27b^2) \neq 0$. Para curvas verificablemente aleatorias (*verifiably random*), los coeficientes se derivan de una semilla s mediante una función hash: $b = H(s)$, publicando s para que cualquiera pueda verificar que no se seleccionó maliciosamente.
3. **Conteo de puntos.** Se calcula $N = \#E(\mathbb{F}_p)$ mediante el algoritmo de Schoof–Elkies–Atkin (SEA), cuya complejidad es polinómica en $\log p$ [58]. Este es el paso computacionalmente más costoso del proceso de generación.
4. **Verificación del orden.** Se comprueba que N tiene un factor primo grande n con cofactor pequeño: $N = hn$ con $h \leq 4$ (idealmente $h = 1$).
5. **Verificaciones de seguridad adicionales:**
 - La curva no es anómala: $N \neq p$.
 - El grado de inmersión es alto: $p^k \not\equiv 1 \pmod{n}$ para $1 \leq k \leq 20$.

- (Opcional) El endomorfismo de la curva no tiene estructura explotable.
6. **Selección del generador G .** Se elige un punto aleatorio $P \in E(\mathbb{F}_p)$ y se calcula $G = hP$. Se verifica que $G \neq \mathcal{O}$, lo que garantiza que G tiene orden n .

Si alguna verificación falla, se repite el proceso con nuevos coeficientes.

Generación de parámetros sobre $\mathbb{F}_{2^m}$

El proceso es análogo pero con diferencias propias de la característica 2:

1. **Selección del grado m .** Se elige m primo o con propiedades adecuadas. Se prefieren valores para los que existan polinomios irreducibles trinómicos o pentanómicos, que permiten reducción eficiente.
2. **Selección del polinomio irreducible $f(x)$.** Se elige un polinomio irreducible de grado m sobre $\mathbb{F}_2$, preferentemente trinomial ($x^m + x^k + 1$) por eficiencia.
3. **Selección de los coeficientes.** Para la ecuación $y^2 + xy = x^3 + ax^2 + b$, se requiere $b \neq 0$ (condición de no singularidad). Las curvas de Koblitz fijan $a \in \{0, 1\}$ para obtener optimizaciones con el endomorfismo de Frobenius.
4. **Conteo de puntos y verificaciones.** Igual que en el caso primo, usando algoritmos adaptados a la característica 2.

Verificación de parámetros de dominio

La verificación de parámetros de dominio es esencial cuando se reciben de un tercero o se adoptan de un estándar. El algoritmo 5 resume las comprobaciones necesarias para curvas sobre $\mathbb{F}_p$ [23, 63].

Nuestra implementación. En nuestro proyecto utilizamos exclusivamente curvas estandarizadas por el NIST [43] y SECG [64], cuyos parámetros han sido auditados y verificados por la comunidad criptográfica. Nuestra función `CurveParams::validate()` implementa las comprobaciones 1–3 y 6 (parcialmente) del algoritmo 5: discriminante no nulo, punto generador en la curva, y valores positivos de n y h . Las verificaciones 4, 5, 7 y 8 no se implementan por ser redundantes con los parámetros de estándares publicados, pero se describen como requisitos teóricos. En un sistema que aceptase curvas arbitrarias definidas por el usuario, su implementación sería imprescindible.

Algorithm 5 Verificación de parámetros de dominio (p, a, b, G, n, h)

Require: Parámetros del dominio $T = (p, a, b, G, n, h)$
Ensure: Válido o Inválido

- 1: Verificar que p es primo y $p > 3$.
- 2: Verificar que $a, b \in [0, p-1]$ y $4a^3 + 27b^2 \not\equiv 0 \pmod{p}$.
- 3: Verificar que $G \neq \mathcal{O}$ y que G está en la curva: $G_y^2 \equiv G_x^3 + aG_x + b \pmod{p}$.
- 4: Verificar que n es primo.
- 5: Verificar que $n \cdot G = \mathcal{O}$. ▷ G tiene orden n
- 6: Verificar que $h = \lfloor (\sqrt{p} + 1)^2 / n \rfloor$. ▷ Cofactor correcto
- 7: Verificar que $hn \neq p$ (no anómala).
- 8: Verificar que $p^k \not\equiv 1 \pmod{n}$ para $1 \leq k \leq 20$. ▷ Grado de inmersión alto

9: **return** Válido

De forma análoga, `BinaryCurveParams::validate()` comprueba que el grado del polinomio irreducible coincide con m , que $b \neq 0$ (discriminante no nulo en característica 2), y que n y h son positivos.

La validación de claves públicas se realiza en el constructor de `ECPoint`, que verifica automáticamente $y^2 = x^3 + ax + b \pmod{p}$ al crear cualquier punto, lanzando una excepción si el punto no pertenece a la curva. Para las curvas con cofactor $h = 1$ (NIST P-256, P-384, secp256k1), esta verificación es suficiente, ya que todo punto no trivial de la curva tiene orden n . Para curvas con $h > 1$ (como las binarias sect233k1 con $h = 4$), sería necesario verificar adicionalmente que $n \cdot Q = \mathcal{O}$ para garantizar la pertenencia al subgrupo correcto, lo que se documenta como limitación del prototipo.

5.3. Generación de claves

La generación de un par de claves ECC es conceptualmente sencilla: la clave privada es un escalar aleatorio y la clave pública es el punto resultante de multiplicar el generador por dicho escalar.

Definición 5.2 (Par de claves ECC). *Dados los parámetros del dominio $T = (p, a, b, G, n, h)$, un par de claves ECC (d, Q) se genera como sigue:*

1. Seleccionar un entero aleatorio d uniformemente distribuido en el rango $[1, n-1]$.
2. Calcular el punto $Q = d \cdot G$.

La clave privada es d y la clave pública es Q .

Dado que n es primo y $d \in [1, n-1]$, el punto Q es un elemento no trivial de $\langle G \rangle$. Conocer Q y G no permite recuperar d eficientemente, por la dificultad del ECDLP (definición 5.1).

Requisitos del generador de números aleatorios. La seguridad del par de claves depende críticamente de la calidad del generador de números aleatorios (RNG) utilizado para seleccionar d . Si el RNG es predecible, sesgado o tiene baja entropía, un atacante podría reducir drásticamente el espacio de búsqueda de d . En nuestra implementación, empleamos el RNG de NTL inicializado con una semilla de alta entropía, cuya calidad estadística fue validada en la fase 2 del proyecto mediante tests de uniformidad, autocorrelación y rachas (véase el capítulo de implementación).

Validación de la clave pública. Al recibir una clave pública Q de un tercero, es imprescindible verificar que:

1. $Q \neq \mathcal{O}$ (no es el punto en el infinito).
2. Q pertenece a la curva: $Q_y^2 \equiv Q_x^3 + aQ_x + b \pmod{p}$.
3. Q pertenece al subgrupo correcto: $n \cdot Q = \mathcal{O}$.

La tercera comprobación es especialmente importante cuando $h > 1$, para prevenir ataques de contribución de subgrupo pequeño [23].

5.4. Intercambio de claves: ECDH

El protocolo de Diffie–Hellman sobre curvas elípticas (ECDH) es la adaptación directa del intercambio de claves Diffie–Hellman clásico (sección 2.4.1) al grupo de puntos de una curva elíptica. Su seguridad se fundamenta en el *problema de Diffie–Hellman computacional en curvas elípticas* (ECDHP): dados P , aP y bP , calcular abP .

5.4.1. Protocolo básico

Sean Alice y Bob dos entidades que desean establecer un secreto compartido. Ambos acuerdan previamente los parámetros del dominio $T = (p, a, b, G, n, h)$. El protocolo se desarrolla de la siguiente manera:

1. **Generación de claves.** Alice genera su par de claves (d_A, Q_A) con $Q_A = d_A \cdot G$, y Bob genera (d_B, Q_B) con $Q_B = d_B \cdot G$.
2. **Intercambio público.** Alice envía Q_A a Bob, y Bob envía Q_B a Alice. Un intruso observa Q_A y Q_B pero no conoce d_A ni d_B .
3. **Cálculo del secreto compartido.** Alice calcula

$$S = d_A \cdot Q_B = d_A \cdot (d_B \cdot G) = d_A d_B \cdot G,$$

y Bob calcula

$$S = d_B \cdot Q_A = d_B \cdot (d_A \cdot G) = d_B d_A \cdot G.$$

Ambos obtienen el mismo punto S gracias a la asociatividad y conmutatividad de la multiplicación escalar.

4. **Derivación de clave.** La coordenada x del punto S (o una función de derivación de clave aplicada sobre ella) se utiliza como clave simétrica de sesión.

Fase	Operación
Acuerdo público	Parámetros $T = (p, a, b, G, n, h)$
Generación de claves	Alice: $d_A \xleftarrow{\$} [1, n-1]$, $Q_A = d_A \cdot G$ Bob: $d_B \xleftarrow{\$} [1, n-1]$, $Q_B = d_B \cdot G$
Intercambio público	Alice $\rightarrow$ Bob: Q_A ; Bob $\rightarrow$ Alice: Q_B
Secreto compartido	Alice: $S = d_A \cdot Q_B$; Bob: $S = d_B \cdot Q_A$

Cuadro 5.4: Resumen del intercambio de claves ECDH.

Ejemplo 5.2. Consideremos la curva $E : y^2 = x^3 + 2x + 3$ sobre $\mathbb{F}_{97}$ con punto generador $G = (3, 6)$ de orden $n = 5$.

Alice elige $d_A = 3$ y calcula $Q_A = 3 \cdot G = (80, 10)$. Bob elige $d_B = 4$ y calcula $Q_B = 4 \cdot G = (80, 87)$.

Alice calcula $S = 3 \cdot Q_B = 3 \cdot (80, 87) = (3, 91)$ y Bob calcula $S = 4 \cdot Q_A = 4 \cdot (80, 10) = (3, 91)$. Ambos obtienen el punto $S = (3, 91)$ como secreto compartido, y podrían usar $x_S = 3$ como semilla para derivar una clave simétrica.

5.4.2. Derivación de claves

En la práctica, la coordenada x del punto compartido S no se utiliza directamente como clave simétrica, sino que se pasa por una *función de derivación de clave* (KDF, *Key Derivation Function*). Las razones son:

- La coordenada x puede tener bits con distribución no uniforme (por ejemplo, los bits más significativos pueden estar correlacionados con la estructura del cuerpo).
- Se necesita adaptar la longitud del secreto al tamaño de la clave simétrica deseada (128, 192 o 256 bits para AES).
- Un KDF aporta una capa adicional de seguridad: aunque se descubriese alguna debilidad parcial en la coordenada x , la clave derivada permanece segura si el KDF es robusto.

En estándares como NIST SP 800-56A [45] se especifican KDFs basados en funciones hash, como HKDF (RFC 5869). En nuestra implementación educativa utilizamos directamente la coordenada x truncada al número de bits deseados, anotando esta simplificación como limitación del prototipo.

5.4.3. Variantes y extensiones

Existen varias variantes del intercambio ECDH que mejoran distintos aspectos del protocolo:

ECIES (*Elliptic Curve Integrated Encryption Scheme*): Combina ECDH con un cifrado simétrico y un MAC en un esquema de cifrado de clave pública completo. Definido en IEEE P1363a e ISO 18033-2, es el análogo de ElGamal sobre curvas elípticas.

X25519: Diseñada por Bernstein [13], es una curva de Montgomery optimizada para intercambios Diffie–Hellman. Su aritmética se realiza exclusivamente sobre la coordenada x (usando la “escalera de Montgomery”), lo que la hace intrínsecamente resistente a ciertos ataques de canal lateral de tiempo constante. Es la curva predeterminada en TLS 1.3 para intercambio de claves.

5.5. Firmas digitales: ECDSA

El *Elliptic Curve Digital Signature Algorithm* (ECDSA) es la adaptación del algoritmo de firma digital DSA al contexto de curvas elípticas. Fue estandarizado en ANSI X9.62 [11] y posteriormente incorporado en FIPS 186–4 [43]. ECDSA proporciona autenticación, integridad y no repudio con claves significativamente más pequeñas que DSA o RSA.

5.5.1. Función hash: SHA-256

Toda firma digital opera sobre un *resumen* (hash) del mensaje, no sobre el mensaje completo. Esto garantiza que la firma tenga tamaño fijo independientemente de la longitud del mensaje y que cualquier modificación del mensaje (por mínima que sea) invalide la firma.

En nuestra implementación utilizamos SHA-256 (FIPS PUB 180-4 [44]), que produce un resumen de 256 bits. Esta función cumple las propiedades de resistencia a preimagen, segunda preimagen y colisiones descritas en la sección 2.2. Al operar con curvas de 256 bits (como NIST P-256 o secp256k1), SHA-256 proporciona una correspondencia natural entre el tamaño del hash y el orden n del generador.

Formalmente, dado un mensaje m de longitud arbitraria, denotamos su hash como

$$e = H(m),$$

donde H es SHA-256 y e se interpreta como un entero. Si la longitud en bits de n es L_n y la del hash es L_H , se toman los $\min(L_n, L_H)$ bits más significativos de e .

5.5.2. Generación de firma

Dados los parámetros del dominio $T = (p, a, b, G, n, h)$ y un par de claves (d, Q) con d la clave privada, la firma de un mensaje m se genera mediante el siguiente procedimiento:

1. Calcular $e = H(m)$.
2. Seleccionar un entero aleatorio $k \xleftarrow{\$} [1, n - 1]$ (*nonce* o valor efímero).
3. Calcular el punto $R = k \cdot G = (x_R, y_R)$.
4. Calcular $r = x_R \bmod n$. Si $r = 0$, volver al paso 2.
5. Calcular

$$s = k^{-1}(e + d \cdot r) \bmod n.$$

Si $s = 0$, volver al paso 2.

6. La firma es el par (r, s) .

El valor k es un secreto temporal que **no debe revelarse** y **no debe reutilizarse**. La sección 5.5.4 detalla las consecuencias catastróficas de violar esta condición.

5.5.3. Verificación de firma

Dada una firma (r, s) sobre un mensaje m y la clave pública Q del firmante, el verificador ejecuta:

1. Comprobar que $r, s \in [1, n - 1]$. Si no, rechazar.
2. Calcular $e = H(m)$.
3. Calcular $w = s^{-1} \bmod n$.
4. Calcular $u_1 = e \cdot w \bmod n$ y $u_2 = r \cdot w \bmod n$.
5. Calcular el punto

$$R' = u_1 \cdot G + u_2 \cdot Q.$$

6. Si $R' = \mathcal{O}$, rechazar. En caso contrario, calcular $v = x_{R'} \bmod n$.
7. Aceptar la firma si y solo si $v = r$.

Demostración de la corrección. Si la firma fue generada honestamente, entonces $s = k^{-1}(e + dr)$ mód n , de donde

$$k = s^{-1}(e + dr) = s^{-1}e + s^{-1}dr = w \cdot e + w \cdot d \cdot r = u_1 + u_2 \cdot d \quad (\text{mód } n).$$

Por tanto,

$$R' = u_1 \cdot G + u_2 \cdot Q = u_1 \cdot G + u_2 \cdot d \cdot G = (u_1 + u_2d) \cdot G = k \cdot G = R.$$

La coordenada x de R' coincide con la de R , luego $v = x_{R'}$ mód $n = x_R$ mód $n = r$. $\square$

5.5.4. Problemas prácticos y vulnerabilidades

Reutilización del nonce k . Si se utiliza el mismo valor k para firmar dos mensajes distintos m_1 y m_2 , las firmas resultantes (r, s_1) y (r, s_2) comparten la misma coordenada r (puesto que $R = kG$ es idéntico). Un atacante que observe ambas firmas puede recuperar la clave privada d de forma inmediata:

$$s_1 - s_2 = k^{-1}(e_1 - e_2) \quad (\text{mód } n) \implies k = \frac{e_1 - e_2}{s_1 - s_2} \quad (\text{mód } n),$$

y una vez conocido k :

$$d = \frac{s_1 k - e_1}{r} \quad (\text{mód } n).$$

Este ataque no es teórico: en 2010, la clave privada de la consola PlayStation 3 de Sony fue comprometida exactamente por esta razón, al emplear un valor fijo de k en todas las firmas ECDSA del *firmware* [18].

Nonces con baja entropía. Incluso sin reutilizar k exactamente, si el RNG que genera k tiene sesgos o baja entropía, ataques de tipo *lattice* (basados en retículos) pueden recuperar la clave privada a partir de un número moderado de firmas [25].

RFC 6979: nonces determinísticos. Para mitigar los riesgos asociados a la generación aleatoria de k , el RFC 6979 [53] propone un esquema determinístico: k se calcula como $k = \text{HMAC-DRBG}(d, H(m))$, es decir, como función de la clave privada y el hash del mensaje. Así, el mismo mensaje siempre produce la misma firma (reproducibilidad), diferentes mensajes producen distintos k (unicidad), y no se requiere un RNG externo en el momento de firmar.

5.6. Aspectos de implementación

5.6.1. Coordenadas afines frente a Jacobianas

Como se describió en la sección 4.4.1, las fórmulas de suma y duplicación de puntos en coordenadas afines requieren una inversión modular por cada operación. Dado que la inversión en $\mathbb{F}_p$ tiene un coste equivalente a 80–100 multiplicaciones modulares para cuerpos de 256 bits [23], este es el cuello de botella principal.

Coste en coordenadas afines. Para una multiplicación escalar de 256 bits mediante *double-and-add*:

- Se realizan ~ 256 duplicaciones y ~ 128 sumas.
- Cada operación requiere una inversión: ~ 384 inversiones en total.
- Coste equivalente: $\sim 30\,000$ – $38\,000$ multiplicaciones modulares solo en inversiones.

Coordenadas Jacobianas. Las coordenadas Jacobianas (véase definición 4.4.2) representan un punto afín (x, y) como una terna $(X : Y : Z)$ donde $x = X/Z^2$, $y = Y/Z^3$. Las fórmulas de duplicación y suma se reescriben *sin divisiones*:

Duplicación de (X_1, Y_1, Z_1) . Sea a el coeficiente de la curva:

$$\begin{aligned} A &= Y_1^2, & B &= 4X_1A, & C &= 8A^2, \\ D &= 3X_1^2 + aZ_1^4, \\ X_3 &= D^2 - 2B, \\ Y_3 &= D(B - X_3) - C, \\ Z_3 &= 2Y_1Z_1. \end{aligned}$$

Coste: 4 multiplicaciones + 6 cuadrados en el caso general; con $a = -3$ (NIST P-256, P-384), el término $3X_1^2 + aZ_1^4$ se factoriza como $3(X_1 - Z_1^2)(X_1 + Z_1^2)$, ahorrando una multiplicación y reduciendo el coste a $4M + 4S$ [23].

Suma de (X_1, Y_1, Z_1) y (X_2, Y_2, Z_2) :

$$\begin{aligned} U_1 &= X_1Z_2^2, & U_2 &= X_2Z_1^2, \\ S_1 &= Y_1Z_2^3, & S_2 &= Y_2Z_1^3, \\ H &= U_2 - U_1, & R &= S_2 - S_1, \\ X_3 &= R^2 - H^3 - 2U_1H^2, \\ Y_3 &= R(U_1H^2 - X_3) - S_1H^3, \\ Z_3 &= HZ_1Z_2. \end{aligned}$$

Coste: 12 multiplicaciones + 4 cuadrados.

Ventaja clave: una única inversión final. Durante toda la multiplicación escalar, las operaciones se realizan en coordenadas Jacobianas. Solo al final, cuando se necesita el resultado en forma afín (x, y) , se calcula una única inversión: Z^{-1} , y luego $x = X \cdot (Z^{-1})^2$, $y = Y \cdot (Z^{-1})^3$. De las ~ 384 inversiones originales, pasamos a exactamente 1.

En nuestra implementación, las coordenadas Jacobianas proporcionaron un *speedup* medible frente a las afines, cuyos resultados detallados se presentan en el capítulo de experimentación.

5.6.2. Curvas sobre cuerpos binarios $\mathbb{F}_{2^m}$

Las curvas elípticas también se definen sobre cuerpos binarios $\mathbb{F}_{2^m}$, donde los elementos son polinomios de grado menor que m con coeficientes en $\{0, 1\}$, y la aritmética se describe en la sección 3.3. La ecuación de Weierstrass toma una forma diferente en característica 2.

Ecuación no supersingular. Para curvas no supersingulares sobre $\mathbb{F}_{2^m}$, la ecuación simplificada es

$$E: \quad y^2 + xy = x^3 + ax^2 + b, \quad a, b \in \mathbb{F}_{2^m}, \quad b \neq 0. \quad (5.5)$$

El discriminante de esta curva es $\Delta = b \neq 0$, lo que garantiza la no singularidad. La condición $b \neq 0$ es esencial: si $b = 0$, la curva sería singular.

Diferencias en la ley de grupo. Las fórmulas de suma y duplicación difieren de las del caso primo:

- **Negación:** $-P = -(x, y) = (x, x + y)$, a diferencia de $(x, -y)$ en $\mathbb{F}_p$.
- **Suma** de $P = (x_1, y_1)$ y $Q = (x_2, y_2)$ con $x_1 \neq x_2$:

$$\lambda = \frac{y_1 + y_2}{x_1 + x_2}, \quad x_3 = \lambda^2 + \lambda + x_1 + x_2 + a, \quad y_3 = \lambda(x_1 + x_3) + x_3 + y_1.$$

- **Duplicación** de $P = (x_1, y_1)$ con $x_1 \neq 0$:

$$\lambda = x_1 + \frac{y_1}{x_1}, \quad x_3 = \lambda^2 + \lambda + a, \quad y_3 = x_1^2 + \lambda x_3 + x_3.$$

Nótese que todas las operaciones de cuerpo (suma, multiplicación, inversión) son operaciones sobre polinomios binarios. En particular, la suma es una operación XOR, sin propagación de acarreo, lo que la hace extremadamente eficiente en hardware.

Curva	Cuerpo	Seguridad (bits)	Tipo
sect163k1	$\mathbb{F}_{2^{163}}$	~ 80	Koblitz
sect233k1	$\mathbb{F}_{2^{233}}$	~ 112	Koblitz
sect283k1	$\mathbb{F}_{2^{283}}$	~ 128	Koblitz
sect233r1	$\mathbb{F}_{2^{233}}$	~ 112	Aleatoria
sect283r1	$\mathbb{F}_{2^{283}}$	~ 128	Aleatoria

Cuadro 5.5: Curvas sobre cuerpos binarios implementadas (SEC 2).

Curvas estándar implementadas. En nuestra implementación incluimos cinco curvas estándar SEC 2 [64] sobre cuerpos binarios:

Las curvas de tipo Koblitz (con $a \in \{0, 1\}$) admiten optimizaciones adicionales basadas en el endomorfismo de Frobenius, que permite sustituir duplicaciones de punto por operaciones de cuadrado en el cuerpo (mucho más baratas en $\mathbb{F}_{2^m}$). Las curvas aleatorias (sufijo “r”) se generaron con coeficientes pseudoaleatorios verificables para evitar sospechas de puertas traseras.

Razón de inclusión. Aunque el NIST ha ido retirando curvas binarias de sus recomendaciones más recientes en favor de curvas sobre cuerpos primos, su estudio es relevante para este trabajo por tres razones: completitud teórica (las dos familias principales de curvas elípticas sobre cuerpos finitos), comparación de rendimiento (cuerpo primo frente a cuerpo binario), y aplicabilidad en hardware embebido donde la aritmética XOR es especialmente ventajosa.

5.6.3. Multiplicación escalar

La multiplicación escalar $k \cdot P$ es la operación computacionalmente dominante en todos los protocolos ECC (generación de claves, ECDH, ECDSA). El algoritmo más directo es el *double-and-add* (análogo a la exponenciación rápida por cuadrados sucesivos):

Para un escalar de t bits, el algoritmo realiza t duplicaciones y, en promedio, $t/2$ sumas, lo que da una complejidad de $O(t) = O(\log n)$ operaciones de grupo. Es esta eficiencia la que hace viable la criptografía de curvas elípticas: calcular $k \cdot P$ es rápido, pero recuperar k dado P y kP requiere $O(\sqrt{n}) = O(2^{t/2})$ operaciones.

Consideraciones de canal lateral. El algoritmo 6 tiene un flujo de ejecución que depende de los bits de k : cuando $k_i = 0$ solo se duplica, y cuando $k_i = 1$ se duplica y se suma. Un atacante que mida tiempos de ejecución, consumo eléctrico o emisiones electromagnéticas puede distinguir ambos casos y reconstruir k bit a bit [33]. Para mitigar esto, existen variantes de tiempo

Algorithm 6 Double-and-add para multiplicación escalar

Require: Escalar $k = (k_{t-1}, k_{t-2}, \dots, k_1, k_0)_2$ y punto $P \in E(\mathbb{F}_q)$
Ensure: $Q = k \cdot P$

```

1:  $Q \leftarrow \mathcal{O}$  ▷ Punto en el infinito
2: for  $i = t - 1$  down to 0 do
3:    $Q \leftarrow 2Q$  ▷ Duplicación
4:   if  $k_i = 1$  then
5:      $Q \leftarrow Q + P$  ▷ Suma
6:   end if
7: end for
8: return  $Q$ 

```

constante como la escalera de Montgomery o el algoritmo *double-and-add-always*, que ejecutan el mismo número de operaciones independientemente de los bits de k .

5.7. Selección de parámetros y curvas estandarizadas

5.7.1. Criterios de selección

La elección de una curva elíptica para uso criptográfico no es trivial. Además de los requisitos de seguridad descritos en la sección 5.2.3, la comunidad criptográfica evalúa los siguientes criterios adicionales:

1. **Rendimiento.** Las curvas con primos especiales (pseudo-Mersenne, como $p = 2^{256} - 2^{224} + 2^{192} + 2^{96} - 1$ para NIST P-256) permiten reducciones modulares rápidas. El coeficiente $a = -3$ (usado en las curvas NIST) optimiza la fórmula de duplicación Jacobiana.
2. **Resistencia a ataques especializados.** Las curvas deben evitar ser anómalas ($\#E(\mathbb{F}_p) = p$) y tener grado de inmersión alto ($p^k \not\equiv 1 \pmod{n}$ para $k \leq 20$), como se discutió en la sección 5.2.3.
3. **Rigidez en la generación (*nothing up my sleeve*).** La comunidad prefiere curvas cuyos parámetros se hayan generado de forma verificablemente aleatoria, sin posibilidad de insertar puertas traseras.
4. **Estandarización y auditoría.** Se prefieren curvas ampliamente auditadas por la comunidad académica e incluidas en estándares reconocidos (NIST, SECG, Brainpool, IETF CFRG) [30].

5.7.2. Curvas estandarizadas

A continuación se describen las principales familias de curvas utilizadas en la práctica:

Curvas NIST (P-256, P-384, P-521). Publicadas en FIPS 186-4 [43] y NIST SP 800-186 [48], las curvas NIST sobre cuerpos primos utilizan la forma corta de Weierstrass $y^2 = x^3 - 3x + b$ con primos diseñados para reducción rápida. Son las curvas más ampliamente desplegadas en Internet (TLS, SSH, S/MIME) y en aplicaciones gubernamentales. En nuestra implementación incluimos P-256 (128 bits de seguridad) y P-384 (192 bits de seguridad).

secp256k1 (Bitcoin). Definida por SECG [64], esta curva utiliza la ecuación $y^2 = x^3 + 7$ sobre un cuerpo primo ($a = 0$, $b = 7$). El coeficiente $a = 0$ la convierte en una curva de Koblitz sobre $\mathbb{F}_p$, lo que permite ciertas optimizaciones en el endomorfismo GLV. Es la curva utilizada por Bitcoin y Ethereum para las firmas ECDSA de transacciones [72]. En nuestra implementación está incluida como una de las tres curvas principales.

Curve25519 y Ed25519. Diseñadas por Bernstein [13], Curve25519 (para ECDH, usando X25519) y Ed25519 (para firmas, usando EdDSA) emplean una curva de Montgomery y su birracionalmente equivalente curva de Edwards, respectivamente, sobre el cuerpo $\mathbb{F}_{2^{255}-19}$. Su diseño prioriza la resistencia a implementaciones incorrectas: aritmética de tiempo constante, inmunidad a ataques de curva inválida (por diseño), y parámetros completamente rígidos. Son las curvas predeterminadas en TLS 1.3 y SSH moderno.

5.7.3. Controversias y confianza

Los *seeds* de las curvas NIST. Los coeficientes b de las curvas NIST se generaron aplicando SHA-1 a una semilla (*seed*), lo que proporciona verificabilidad parcial: cualquiera puede comprobar que $b = \text{SHA-1}(\text{seed})$. Sin embargo, la semilla en sí no tiene justificación pública, lo que ha generado sospechas de que podría haberse seleccionado mediante búsqueda exhaustiva para producir una curva con alguna debilidad oculta. No existe evidencia de tal debilidad, pero la incertidumbre ha impulsado la adopción de curvas con parámetros completamente rígidos (como Curve25519).

Dual EC DRBG. El caso más grave de sospecha de puerta trasera en curvas elípticas fue el generador de números aleatorios Dual EC DRBG, estandarizado por el NIST en 2006 y retirado en 2014. Este generador utilizaba dos puntos P y Q de una curva elíptica, y se demostró [60] que si

se conoce la relación $Q = eP$ (es decir, el logaritmo discreto de Q respecto a P), es posible predecir toda la salida futura del generador. Documentos filtrados por Snowden en 2013 sugirieron que la NSA había pagado a RSA Security para que Dual EC DRBG fuera el generador predeterminado en su biblioteca BSAFE.

Este episodio, aunque no compromete la seguridad de las curvas elípticas per se, subraya la importancia de la transparencia y la rigidez en la generación de parámetros criptográficos, y es una de las razones por las que la comunidad ha favorecido curvas como Curve25519 con diseño completamente abierto.

Parte IV

Implementación y Experimentación

Capítulo 6

Implementación y entorno experimental

6.1. Visión general

El objetivo de la parte práctica de este trabajo es construir una suite de benchmarking que permita comparar, de forma reproducible y cuantificable, el rendimiento de RSA y ECC a lo largo de tres ejes ortogonales:

1. **RSA frente a ECC**, a niveles de seguridad equivalentes según NIST SP 800-57 [47].
2. **Coordenadas afines frente a Jacobianas**, midiendo el impacto real de eliminar inversiones modulares.
3. **Cuerpo primo $\mathbb{F}_p$ frente a cuerpo binario $\mathbb{F}_{2^m}$** , comparando la aritmética subyacente.

La implementación se ha desarrollado íntegramente en C++17, utilizando la biblioteca NTL [59] para aritmética de precisión arbitraria y GMP [22] como motor de cálculo subyacente. Todo el entorno se ejecuta dentro de un contenedor Docker para garantizar la reproducibilidad de los resultados.

En las secciones siguientes se describe la arquitectura del software, las decisiones de diseño tomadas, el entorno de ejecución, y los procedimientos de validación aplicados al generador de números aleatorios.

6.2. Arquitectura del software

6.2.1. Estructura modular

El proyecto sigue una arquitectura modular en la que cada componente criptográfico reside en un par de archivos (cabecera `.hpp` e implementación `.cpp`). La figura 6.1 muestra el diagrama de dependencias.

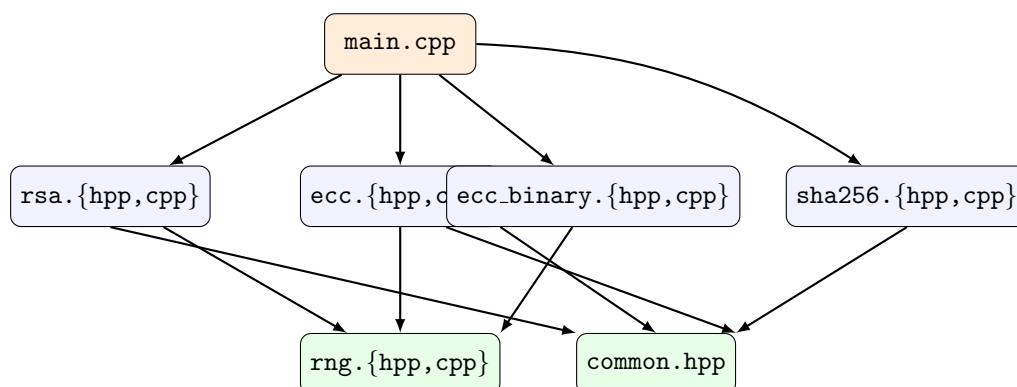


Figura 6.1: Diagrama de dependencias entre los módulos del proyecto. `main.cpp` actúa como punto de entrada y orquesta los módulos criptográficos (RSA, ECC sobre $\mathbb{F}_p$ y $\mathbb{F}_{2^m}$, SHA-256), todos los cuales se apoyan en el generador de números aleatorios `rng` y los tipos comunes definidos en `common.hpp`.

Los módulos del proyecto son los siguientes:

common.hpp Define los tipos base compartidos por todos los módulos. El alias `BigInt = NTL::ZZ` proporciona enteros de precisión arbitraria. Incluye constantes globales (tamaños de clave RSA por defecto, número de iteraciones Miller-Rabin, curva ECC por defecto) y una excepción personalizada `CryptoException`.

rng.hpp / rng.cpp Implementa el generador de números aleatorios. Define una interfaz abstracta `RNG` con métodos `random_bnd`, `random_range`, `random_len`, `random_bits` y `random_prime`. La implementación concreta `NTLRNG` delega en las funciones criptográficas de `NTL` y soporta semillas fijas para reproducibilidad.

rsa.hpp / rsa.cpp Implementa RSA completo: generación de claves (con parámetros CRT precalculados), cifrado, descifrado (con y sin optimización CRT), firma y verificación. La estructura `RSAPrivateKey` almacena p , q , $d_p = d \bmod (p-1)$, $d_q = d \bmod (q-1)$ y $q^{-1} \bmod p$ para descifrado CRT.

ecc.hpp / ecc.cpp Implementa la criptografía de curvas elípticas sobre cuerpos primos. Incluye dos representaciones de puntos: coordenadas afines (`ECPoint`) y coordenadas Jacobianas (`JacobianPoint`). Soporta tres curvas estándar: NIST P-256, NIST P-384 y secp256k1. Implementa generación de claves, ECDH, y ECDSA (firma y verificación). Todas las operaciones aceptan un parámetro `use_jacobian` que permite seleccionar la representación en tiempo de ejecución.

ecc_binary.hpp / ecc_binary.cpp Implementa curvas elípticas sobre cuerpos binarios $\mathbb{F}_{2^m}$ utilizando `NTL::GF2E` para la aritmética de polinomios. Soporta cinco curvas estándar SEC 2: `sect163k1`, `sect233k1`, `sect283k1`, `sect233r1` y `sect283r1`. Incluye generación de claves y ECDH sobre cuerpos binarios.

sha256.hpp / sha256.cpp Implementación completa de SHA-256 siguiendo FIPS PUB 180-4 [44]. Se implementó desde cero (sin delegar en OpenSSL) por dos razones: integridad académica (comprender y documentar cada paso del algoritmo) y control total sobre el benchmarking (evitar que variaciones en la versión de OpenSSL afecten las mediciones).

main.cpp Motor de benchmarking con cinco modos de ejecución, parseo de argumentos CLI mediante `getopt`, y generación de resultados en formato CSV.

6.2.2. Interfaz de línea de comandos

El binario compilado acepta los siguientes parámetros:

```
./bin/bench [opciones]
-a ALGO      RSA | ECC | ECCJ | BIN | CMP      (default: RSA)
-b BITS      Tamaño de clave RSA                (default: 2048)
-c CURVE     Nombre de curva ECC                (default: secp256k1)
-i ITTERS    Numero de iteraciones              (default: 50)
-s SEED      fixed | random                     (default: fixed)
-r FILE      Fichero CSV de datos brutos
-v           Modo verboso (progreso a stderr)
```

Los cinco modos de algoritmo corresponden a los tres ejes de comparación:

Modo	Descripción	Etiqueta CSV
RSA	RSA con tamaños 1024–4096 bits	RSA
ECC	ECC sobre $\mathbb{F}_p$ (coordenadas afines)	ECC
ECCJ	ECC sobre $\mathbb{F}_p$ (coordenadas Jacobianas)	ECC_JACOBIAN
BIN	ECC sobre $\mathbb{F}_{2^m}$	ECC_BINARY
CMP	Comparación completa (los cuatro anteriores)	Todas

Cuadro 6.1: Modos de ejecución del motor de benchmarking.

El modo **CMP** ejecuta automáticamente los cuatro modos restantes y genera aproximadamente 72 benchmarks individuales: 4 tamaños RSA $\times$ 6 operaciones, 3 curvas primas $\times$ 6 operaciones (afines), 3 curvas primas $\times$ 5 operaciones (Jacobianas), y 5 curvas binarias $\times$ 3 operaciones.

6.3. Decisiones de diseño

6.3.1. Progresión pedagógica: afines, después Jacobianas

Una decisión fundamental del proyecto fue implementar primero la aritmética de curvas en coordenadas afines y, solo después de validar su corrección, añadir las coordenadas Jacobianas como optimización explícita. Esta decisión responde a un principio de ingeniería de software y a un criterio académico:

Claridad matemática. La ecuación de Weierstrass $y^2 = x^3 + ax + b$ se verifica directamente sustituyendo (x, y) . En coordenadas Jacobianas, el mismo punto tiene infinitas representaciones equivalentes $(X, Y, Z) \sim (\lambda^2 X, \lambda^3 Y, \lambda Z)$ para cualquier $\lambda \neq 0$, lo que dificulta la comprobación visual.

Verificabilidad. Después de cada operación (suma, duplicación), el resultado se puede comprobar con `is_on_curve()`. En coordenadas Jacobianas hay que convertir a afines antes de comparar con vectores de test conocidos.

Corrección primero. El principio seguido fue “primero que funcione correctamente, luego optimizar”. Las coordenadas afines proporcionan la base de referencia contra la que validar cualquier optimización.

Valor académico. La progresión afines $\rightarrow$ Jacobianas es en sí misma un resultado medible del TFG: el speedup observado ($3\text{--}5\times$ en la práctica, frente al teórico $8.3\times$) constituye un dato experimental propio.

6.3.2. SHA-256 desde cero

La implementación de SHA-256 se realizó siguiendo paso a paso el estándar FIPS PUB 180-4, sin utilizar la implementación de OpenSSL. Las razones son:

- **Integridad académica:** En un TFG sobre criptografía, es importante demostrar la comprensión de los algoritmos, no solo invocar bibliotecas de terceros.
- **Control del benchmarking:** Al implementar SHA-256 nosotros mismos, el hash no introduce una variable externa que pueda variar entre versiones de OpenSSL o configuraciones de compilación.
- **Coherencia:** El mismo principio se aplicó a RSA, ECC y el RNG: toda la criptografía del proyecto está implementada sobre una única base (NTL/GMP), sin dependencias externas salvo las aritméticas.

6.3.3. Uso de curvas estandarizadas

En lugar de generar curvas elípticas propias, el proyecto utiliza exclusivamente curvas estandarizadas publicadas por el NIST [43] y SECG [64]. Los parámetros del dominio (p, a, b, G, n, h) están codificados directamente en `get_curve_params()` y `get_binary_curve_params()`.

Esta decisión se justifica por:

- La generación de curvas seguras requiere algoritmos de conteo de puntos (Schoof–Elkies–Atkin) que están fuera del alcance de este trabajo.
- Las curvas NIST y SEC 2 han sido auditadas extensamente por la comunidad criptográfica.
- Usar curvas estándar permite comparar nuestros resultados con benchmarks de referencia (por ejemplo, OpenSSL).

No obstante, se implementa una validación parcial de los parámetros (sección 6.3.4) para verificar las propiedades que son computacionalmente viables.

6.3.4. Validación de parámetros del dominio

La función `CurveParams::validate()` implementa las siguientes comprobaciones para curvas sobre $\mathbb{F}_p$:

1. $p > 3$.
2. Discriminante no nulo: $4a^3 + 27b^2 \not\equiv 0 \pmod{p}$.
3. El punto generador G pertenece a la curva: $G_y^2 \equiv G_x^3 + aG_x + b \pmod{p}$.
4. $n > 0$ y $h > 0$.

De forma analoga, `BinaryCurveParams::validate()` comprueba que $\deg(f) = m$, que $b \neq 0$ (discriminante en característica 2), y que $n, h > 0$.

Las verificaciones que *no* se implementan —primalidad de n , $nG = \mathcal{O}$, curva no anómala, grado de inmersión alto— se describen como requisitos teóricos en la sección 5.2.3. Su omisión está justificada por el uso exclusivo de curvas pre-auditadas (véase la discusión en la sección 5.2.4).

6.4. Herramientas y entorno de ejecución

6.4.1. Lenguaje y bibliotecas

El cuadro 6.2 resume las herramientas principales del proyecto.

Herramienta	Versión	Función
C++	C++17 (ISO/IEC 14882:2017)	Lenguaje de implementación
NTL	≥ 11.5	Aritmética de precisión arbitraria, $\mathbb{F}_p$, $\mathbb{F}_{2^m}$
GMP	≥ 6.2	Motor aritmético subyacente de NTL
g++	≥ 9.0	Compilador (flags: <code>-std=c++17 -O2</code>)
Docker	≥ 20.0	Contenedorización del entorno
Python 3	≥ 3.8	Visualización y análisis estadístico
matplotlib	≥ 3.5	Generación de gráficas
Pillow	≥ 9.0	Composición de collages

Cuadro 6.2: Herramientas y bibliotecas del proyecto.

Elección de NTL. Se eligió NTL (Number Theory Library) de Victor Shoup [59] como base aritmética por varias razones: proporciona tipos nativos para enteros de precisión arbitraria (ZZ), aritmética modular (ZZ.p), y cuerpos binarios de extensión (GF2E); ofrece funciones criptográficas de bajo nivel (exponenciación modular, test de primalidad Miller-Rabin, generación de primos); y se construye sobre GMP, heredando sus optimizaciones a nivel de ensamblador para las operaciones aritméticas básicas.

Nivel de optimización. Se compila con `-O2` en lugar de `-O3` para mantener un equilibrio entre rendimiento y predictibilidad de los tiempos de ejecución. El nivel `-O3` puede activar vectorización y desenrollado de bucles que introduzcan variabilidad en las mediciones.

6.4.2. Contenedorización con Docker

Para garantizar la reproducibilidad de los resultados, el proyecto incluye un `Dockerfile` que define un entorno de compilación y ejecución completo basado en Ubuntu 24.04. El contenedor instala NTL, GMP, Python 3 con las bibliotecas de visualización, compila el proyecto, y permite ejecutar los benchmarks en un entorno controlado.

El flujo de trabajo típico es:

```
docker build -t ecc-thesis .
docker run -it ecc-thesis /bin/bash
# Dentro del contenedor:
bash run_benchmarks.sh 20
```

Esto asegura que dos ejecuciones en la misma máquina produzcan resultados comparables, independientemente de las bibliotecas instaladas en el sistema anfitrión.

6.5. Generador de números aleatorios

6.5.1. Diseño del RNG

El generador de números aleatorios se implementa siguiendo el patrón de diseño *Strategy*: la interfaz abstracta `RNG` define los métodos que todo generador debe ofrecer, y la implementación concreta `NTLRNG` delega en las funciones de NTL (`SetSeed`, `RandomBnd`, `GenPrime`, etc.).

La característica clave del diseño es el soporte para *semillas fijas*: al inicializar `NTLRNG` con un valor de semilla concreto (por ejemplo, `seed = 0`), toda la secuencia de números aleatorios generados es idéntica en cualquier ejecución. Esto permite:

- Reproducir exactamente los resultados de un benchmark.
- Comparar el rendimiento de dos algoritmos con *los mismos* valores aleatorios.
- Depurar errores en operaciones criptográficas con datos predecibles.

En modo “aleatorio” (`-s random`), la semilla se deriva del timestamp del sistema, proporcionando aleatoriedad real para cada ejecución.

6.5.2. Validación estadística

Para verificar que el RNG produce secuencias con propiedades estadísticas adecuadas para uso criptográfico, se implementó un programa de análisis dedicado (`rng_analysis.cpp`) y un conjunto de tests estadísticos en Python (`analyze_randomness.py`). Los tests realizados fueron:

Test de uniformidad (Chi-cuadrado): Verifica que la distribución de los valores generados se ajusta a una distribución uniforme.

Test de Kolmogorov-Smirnov: Compara la distribución empírica con la teórica uniforme en $[0, 1]$.

Test de rachas (*runs*): Verifica que las secuencias de bits consecutivos iguales tienen la longitud esperada bajo aleatoriedad.

Autocorrelación: Mide la correlación entre valores consecutivos (debe ser cercana a cero).

Análisis de entropía: Calcula la entropía de Shannon de la distribución y la compara con el máximo teórico.

Los resultados de la validación, incluyendo histogramas, gráficas de autocorrelación y resúmenes estadísticos, se generan automáticamente mediante

el script `run_rng_analysis.sh`. Todos los tests pasaron satisfactoriamente con p -values superiores a 0.05, confirmando que el RNG de NTL produce secuencias estadísticamente indistinguibles de secuencias verdaderamente aleatorias bajo los criterios evaluados.

6.6. Motor de benchmarking

6.6.1. Metodología de medición

Cada benchmark sigue un protocolo estandarizado:

1. Se ejecutan 3 iteraciones de calentamiento (*warm-up*) que se descartan, para asegurar que las cachés de CPU estén pobladas y que cualquier inicialización perezosa de NTL se haya completado.
2. Se ejecutan N iteraciones medidas (configurable con `-i`).
3. Para cada iteración, se mide el tiempo transcurrido con `std::chrono::high_resolution_clock` con precisión de microsegundos.
4. Se calculan las siguientes estadísticas: media, mediana, desviación estándar, mínimo, máximo, y percentiles P5 y P95.

El núcleo del sistema de medición es la función genérica `run_benchmark`, implementada como un template de C++ que acepta cualquier operación como argumento `lambda`:

```
template<typename Func>
BenchmarkResult run_benchmark(
    const string& label, Func operation, int iterations);
```

Esta abstracción permite medir cualquier operación criptográfica (generación de claves, cifrado, firma, etc.) con la misma infraestructura de medición, eliminando la posibilidad de inconsistencias entre benchmarks.

6.6.2. Operaciones medidas

El cuadro 6.3 resume las operaciones medidas para cada algoritmo.

Todas las comparaciones RSA-ECC se realizan a niveles de seguridad equivalentes según NIST SP 800-57: RSA-3072 frente a P-256/secp256k1 (128 bits de seguridad) y RSA-7680 frente a P-384 (192 bits).

Algoritmo	Operación	Descripción
RSA	Keygen	Generación de claves (1024, 2048, 3072, 4096 bits)
	Encrypt	Cifrado $c = m^e \bmod n$
	Decrypt	Descifrado directo $m = c^d \bmod n$
	Decrypt CRT	Descifrado con Teorema Chino del Resto
	Sign	Firma $s = h^d \bmod n$
	Verify	Verificación $h = s^e \bmod n$
ECC (afín)	Keygen	Generación $Q = dG$ (P-256, P-384, secp256k1)
	Scalar mult	Multiplicación escalar kP
	ECDH	Secreto compartido $S = d_A Q_B$
	ECDSA sign	Firma con nonce k
	ECDSA verify	Verificación $u_1 G + u_2 Q$
	Point add/double	Operaciones elementales de grupo
ECC (Jacobiano)	Keygen	Igual que afín, con <code>use_jacobian=true</code>
	ECDH	Secreto compartido con aritmética Jacobiana
	ECDSA sign/verify	Firma y verificación con aritmética Jacobiana
ECC (binario)	Keygen	Generación sobre $\mathbb{F}_{2^m}$ (5 curvas SEC 2)
	Scalar mult	Multiplicación escalar en cuerpo binario
	ECDH	Secreto compartido sobre $\mathbb{F}_{2^m}$

Cuadro 6.3: Operaciones criptográficas medidas en el benchmark.

6.6.3. Pipeline de visualización

Los resultados del benchmark se procesan mediante dos scripts de Python:

`visualize_benchmarks.py` Lee los ficheros CSV generados por el motor de benchmarking y produce 11 gráficas individuales: comparaciones de generación de claves, operaciones de firma y verificación, speedup de ECDH, escalado por tamaño de clave, comparación afín vs. Jacobiano, y rendimiento en cuerpos binarios. Utiliza una paleta de cuatro colores consistente (azul/rojo/verde/morado) para distinguir los tipos de algoritmo.

`create_collages.py` Combina las gráficas individuales en 3 collages temáticos correspondientes a los tres ejes de comparación, facilitando su inclusión en la memoria y en presentaciones. Utiliza PIL (Pillow) para la composición de imágenes.

El pipeline completo se orquesta mediante `run_benchmarks.sh`, que automatiza la compilación, ejecución de benchmarks, verificación de datos y generación de visualizaciones en un único comando:

```
bash run_benchmarks.sh 20 # 20 iteraciones por benchmark
```

El script genera ficheros con timestamp (evitando sobreescrituras) y crea enlaces simbólicos `summary_latest.csv` y `raw_latest.csv` para facilitar el acceso a los resultados más recientes.

6.7. Curvas implementadas

6.7.1. Curvas sobre cuerpos primos $\mathbb{F}_p$

Se implementaron tres curvas estándar, con parámetros tomados directamente de FIPS 186-4 y SEC 2:

Curva	Bits	a	h	Seguridad
NIST P-256 (secp256r1)	256	-3	1	~128 bits
NIST P-384	384	-3	1	~192 bits
secp256k1 (Bitcoin)	256	0	1	~128 bits

Cuadro 6.4: Curvas sobre cuerpos primos implementadas.

Las curvas NIST utilizan $a = -3$, lo que permite una optimización en la formula de duplicación Jacobiana: el termino $3X_1^2 + aZ_1^4$ se simplifica a $3(X_1 - Z_1^2)(X_1 + Z_1^2)$, ahorrando una multiplicación. La curva secp256k1 utiliza $a = 0$, lo que elimina completamente el termino aZ_1^4 .

6.7.2. Curvas sobre cuerpos binarios $\mathbb{F}_{2^m}$

Se implementaron cinco curvas del estándar SEC 2 [64]:

Curva	m	h	Seguridad	Polinomio irreducible
sect163k1	163	2	~ 80 bits	$x^{163} + x^7 + x^6 + x^3 + 1$
sect233k1	233	4	~ 112 bits	$x^{233} + x^{74} + 1$
sect283k1	283	4	~ 128 bits	$x^{283} + x^{12} + x^7 + x^5 + 1$
sect233r1	233	2	~ 112 bits	$x^{233} + x^{74} + 1$
sect283r1	283	2	~ 128 bits	$x^{283} + x^{12} + x^7 + x^5 + 1$

Cuadro 6.5: Curvas sobre cuerpos binarios implementadas (SEC 2).

Las curvas con sufijo “k” (Koblitz) tienen $a \in \{0, 1\}$, lo que permite optimizaciones basadas en el endomorfismo de Frobenius. Las curvas con sufijo “r” (random) utilizan coeficientes pseudoaleatorios verificables. En ambos casos, la aritmética del cuerpo se realiza íntegramente con $\text{NTL} : \text{GF2E}$, donde la suma es la operación XOR y la multiplicación es el producto de polinomios módulo el polinomio irreducible correspondiente.

6.8. Limitaciones del prototipo

Es importante señalar que esta implementación es un prototipo educativo y de benchmarking, *no* un sistema criptográfico de producción. Las principales limitaciones son:

1. **Resistencia a canales laterales.** El algoritmo *double-and-add* tiene un flujo de ejecución que depende de los bits de la clave privada, lo que lo hace vulnerable a ataques de tiempo, consumo eléctrico y emisión electromagnética. Una implementación de producción utilizaría la escalera de Montgomery o algoritmos de tiempo constante.
2. **Nonces ECDSA.** Los nonces k se generan de forma aleatoria. Una implementación robusta utilizaría nonces deterministas según RFC 6979 [53] para evitar los riesgos descritos en la sección 5.5.4.
3. **Derivación de clave (KDF).** El secreto compartido ECDH se deriva truncando la coordenada x del punto compartido, en lugar de usar un KDF estándar como HKDF (RFC 5869).
4. **Esquemas de padding.** RSA se implementa sin padding (textbook RSA). Una implementación segura requeriría OAEP para cifrado y PSS para firmas.
5. **Validación incompleta de parámetros.** Como se detalla en la sección 6.3.4, no se verifica la primalidad de n , $nG = \mathcal{O}$, ni las condiciones de inmersión y anomalía, confiando en los parámetros estandarizados.

Estas limitaciones no afectan la validez de las comparaciones de rendimiento, que es el objetivo principal del proyecto, pero deben tenerse en cuenta si se considerase reutilizar el código en un contexto de seguridad real.

Parte V

Conclusión

Bibliografía

- [1] Modern cryptographic attacks: A guide for the perplexed. <https://research.checkpoint.com/2024/modern-cryptographic-attacks-a-guide-for-the-perplexed/>. Accessed: May 2025.
- [2] Types of cryptographic attacks to know. <https://fiveable.me/lists/types-of-cryptographic-attacks>. Accessed: May 2025.
- [3] Active and passive attacks in information security. <https://www.geeksforgeeks.org/active-and-passive-attacks-in-information-security/>. Accessed: May 2025.
- [4] Cryptanalysis and types of attacks. <https://www.geeksforgeeks.org/cryptanalysis-and-types-of-attacks/>. Accessed: May 2025.
- [5] What are cryptographic attacks? <https://www.goallsecure.com/blog/cryptographic-attacks-complete-guide/>. Accessed: May 2025.
- [6] Different types of cryptography attacks. <https://www.infosectrain.com/blog/different-types-of-cryptography-attacks/>. Accessed: May 2025.
- [7] Cryptography attacks: 6 types & prevention. <https://www.packetlabs.net/posts/cryptography-attacks/>. Accessed: May 2025.
- [8] The quantum computing threat. <https://docs.paloaltonetworks.com/network-security/quantum-security/administration/quantum-security-concepts/the-quantum-computing-threat>. Accessed: May 2025.
- [9] Shor's algorithm: A quantum threat to modern cryptography. <https://postquantum.com/post-quantum/shors-algorithm-a-quantum-threat/>. Accessed: May 2025.

- [10] Expert advice: Encryption 101 – triple DES explained. <https://www.techtarget.com/searchsecurity/tip/Expert-advice-Encryption-101-Triple-DES-explained>. Accessed: May 2025.
- [11] Public key cryptography for the financial services industry: ECDSA, 1998.
- [12] Paulo Barreto and Reinaldo Dahab. Efficient multiplication on $\mathbb{F}_{2^n}$ and $\mathbb{F}_p$ via OEF. In *Selected Areas in Cryptography (SAC)*, pages 1–13, 2003.
- [13] Daniel J. Bernstein. Curve25519: New Diffie-Hellman speed records. In *Public Key Cryptography – PKC 2006*, pages 207–228. Springer, 2006. ISBN 978-3-540-33852-9.
- [14] Binary Terms. Block cipher. <https://binaryterms.com/block-cipher.html>.
- [15] Dan Boneh and Matt Franklin. Identity-based encryption from the Weil pairing. *SIAM Journal on Computing*, 32(3):586–615, 2003. doi: 10.1137/S0097539701398295.
- [16] Crypto Museum. The vernam cipher. <https://www.cryptomuseum.com/crypto/vernam.htm>, 2012.
- [17] W. Diffie and M. Hellman. New directions in cryptography. *IEEE Transactions on Information Theory*, 22(6):644–654, 1976. doi: 10.1109/TIT.1976.1055638.
- [18] fail0verflow. Console hacking 2010: PS3 epic fail. 27th Chaos Communication Congress (27C3), 2010. URL <https://events.ccc.de/congress/2010/Fahrplan/events/4087.en.html>.
- [19] Pierrick Gaudry, Emmanuel Thomé, and Marion Videau. Breaking ECC2K-130: Discrete logarithm records over large fields. In *Advances in Cryptology – ASIACRYPT 2009*, volume 5912 of *LNCS*, pages 109–128. Springer, 2009.
- [20] GeeksforGeeks. What is RC4 encryption? <https://www.geeksforgeeks.org/what-is-rc4-encryption/>. Accessed: May 2025.
- [21] Google Security Blog and CWI Amsterdam. Announcing the first SHA-1 collision. Google Online Security Blog, 2017. URL <https://security.googleblog.com/2017/02/announcing-first-sha1-collision.html>.

- [22] Torbjörn Granlund and the GMP development team. GNU MP: The GNU multiple precision arithmetic library. <https://gmplib.org/>, 2024. Accessed: May 2025.
- [23] Darrel Hankerson, Alfred Menezes, and Scott Vanstone. *Guide to Elliptic Curve Cryptography*. Springer, New York, NY, 2004.
- [24] Daniel Hodges, Richard Lee, and Christof Paar. Efficient elliptic curve exponentiation with precomputation on smart cards. In *Cryptographic Hardware and Embedded Systems (CHES 2001)*, volume 2162 of *LNCS*, pages 129–143. Springer, 2001.
- [25] Nick Howgrave-Graham and Nigel P. Smart. Lattice attacks on digital signature schemes. *Designs, Codes and Cryptography*, 23(3):283–290, 2001. doi: 10.1023/A:1011214926272.
- [26] IETF. Elliptic curve cipher suites for TLS. RFC 4492, 2006. URL <https://tools.ietf.org/html/rfc4492>.
- [27] IETF. Suite B profile for transport layer security (TLS). RFC 5430, 2009. URL <https://tools.ietf.org/html/rfc5430>.
- [28] IETF. Elliptic curve algorithm for secure shell (SSH). RFC 5656, 2009. URL <https://tools.ietf.org/html/rfc5656>.
- [29] IETF. Elliptic curve cryptography (ECC) cipher suites for TLS. RFC 8422, 2018. URL <https://tools.ietf.org/html/rfc8422>.
- [30] IETF CFRG. Elliptic curve cryptography (ECC) and curve generation. <https://datatracker.ietf.org/wg/cfrg/documents/>. IETF Internet-Draft.
- [31] Oliver Damsgaard Jensen and Kristoffer Alvern Andersen. A5 encryption in GSM. Technical report, University of California, Santa Barbara, 2017. URL <https://sites.cs.ucsb.edu/~koclab/teaching/cren/project/2017/jensen%2Bandersen.pdf>.
- [32] Neal Koblitz. Elliptic curve cryptosystems. *Mathematics of Computation*, 48(177):203–209, 1987.
- [33] Paul C. Kocher, Joshua Jaffe, and Benjamin Jun. Differential power analysis. In *Advances in Cryptology – CRYPTO ’99*, pages 388–397, 1999.
- [34] Arjen K. Lenstra and Eric R. Verheul. Selecting cryptographic key sizes. *Journal of Cryptology*, 14(4):255–293, 2001.

- [35] Alfred J. Menezes, Tatsuaki Okamoto, and Scott A. Vanstone. Reducing elliptic curve logarithms to logarithms in a finite field. *IEEE Transactions on Information Theory*, 39(5):1639–1646, 1993. doi: 10.1109/18.259647.
- [36] Microsoft. Microsoft security advisory: SHA-1 deprecation for SSL/TLS certificates. Microsoft Support, 2017.
- [37] Hermann Miltzer and Christof Paar. Efficient implementation of OEF arithmetic for ECC. *Cryptographic Hardware and Embedded Systems*, pages 123–134, 2004.
- [38] Victor S. Miller. Use of elliptic curves in cryptography. In *Advances in Cryptology – CRYPTO ’85*, pages 417–426. Springer, 1986.
- [39] Mozilla Security Blog. The end of SHA-1 on the public web. Mozilla Security Blog, 2017. URL <https://blog.mozilla.org/security/2017/02/23/the-end-of-sha-1-on-the-public-web/>.
- [40] National Institute of Standards and Technology (NIST). FIPS PUB 180-1: Secure hash standard. Technical report, NIST, 1995. URL <https://csrc.nist.gov/publications/detail/fips/180/1/final>.
- [41] National Institute of Standards and Technology (NIST). FIPS 186-2: Digital signature standard. <https://csrc.nist.gov/publications/detail/fips/186/2/final>, 2000.
- [42] National Institute of Standards and Technology (NIST). FIPS 197: Advanced encryption standard (AES). Technical report, NIST, 2001. URL <https://csrc.nist.gov/pubs/fips/197/final>.
- [43] National Institute of Standards and Technology (NIST). FIPS 186-4: Digital signature standard (DSS). Technical report, NIST, 2013. URL <https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.186-4.pdf>.
- [44] National Institute of Standards and Technology (NIST). FIPS PUB 180-4: Secure hash standard (SHS). Technical report, NIST, 2015. URL <https://csrc.nist.gov/publications/detail/fips/180/4/final>.
- [45] National Institute of Standards and Technology (NIST). SP 800-56A Rev. 3: Recommendation for pair-wise key-establishment schemes using discrete logarithm cryptography. Technical report, NIST, 2018. URL <https://csrc.nist.gov/publications/detail/sp/800-56a/rev-3/final>.
- [46] National Institute of Standards and Technology (NIST). SP 800-131A Rev. 2: Transitions: Recommendation for transitioning the use of cryptographic algorithms and key lengths. Technical report, NIST, 2019.

- URL <https://csrc.nist.gov/publications/detail/sp/800-131a/rev-2/final>.
- [47] National Institute of Standards and Technology (NIST). SP 800-57 Part 1 Rev. 5: Recommendation for key management. Technical report, NIST, 2020. URL <https://csrc.nist.gov/publications/detail/sp/800-57-part-1/rev-5/final>.
- [48] National Institute of Standards and Technology (NIST). SP 800-186: Recommendations for discrete logarithm-based cryptography: Elliptic curve domain parameters. Technical report, NIST, 2023. URL <https://csrc.nist.gov/publications/detail/sp/800-186/final>.
- [49] NIST. Security strength of RSA in relation with the modulus size. Crypto StackExchange, 2012. URL <https://crypto.stackexchange.com/questions/8687/security-strength-of-rsa-in-relation-with-the-modulus-size>.
- [50] OpenSSL Project. Openssl benchmarking, 2021. URL <https://www.openssl.org/docs/manmaster/man1/openssl-speed.html>.
- [51] Fabien A. P. Petitcolas. Kerckhoffs principle. In Henk C. A. van Tilborg and Sushil Jajodia, editors, *Encyclopedia of Cryptography and Security*. Springer, 2nd edition, 2011. doi: 10.1007/978-1-4419-5906-5.
- [52] John M. Pollard. Monte carlo methods for index computation (mód p). *Mathematics of Computation*, 32(143):918–924, 1978. doi: 10.1090/S0025-5718-1978-0491431-9.
- [53] Thomas Pornin. Deterministic usage of DSA and ECDSA. RFC 6979, 2013. URL <https://tools.ietf.org/html/rfc6979>.
- [54] Eric Rescorla. The transport layer security (TLS) protocol version 1.3. RFC 8446, 2018. URL <https://tools.ietf.org/html/rfc8446>.
- [55] Matt Rickard. Elliptic curve cryptography for beginners. <https://mattrickard.com/elliptic-curve-cryptography>. Accessed: May 2025.
- [56] Ronald Rivest. RFC 1321: MD5 message-digest algorithm. IETF RFC 1321, 1992. URL <https://www.rfc-editor.org/rfc/rfc1321.html>.
- [57] Bruce Schneier. Twofish. <https://www.schneier.com/academic/twofish/>. Accessed: May 2025.
- [58] René Schoof. Elliptic curves over finite fields and the computation of square roots mod p . *Mathematics of Computation*, 44(170):483–494, 1985. doi: 10.1090/S0025-5718-1985-0777280-6.

- [59] Victor Shoup. NTL: A library for doing number theory. <https://libnt1.org/>, 2024. Accessed: May 2025.
- [60] Dan Shumow and Niels Ferguson. On the possibility of a back door in the NIST SP800-90 Dual Ec Prng. In *CRYPTO 2007 Rump Session*, 2007. URL <http://rump2007.cr.yp.to/15-shumow.pdf>.
- [61] Joseph H. Silverman. *The Arithmetic of Elliptic Curves*, volume 106 of *Graduate Texts in Mathematics*. Springer, 1986.
- [62] Nigel P. Smart. The discrete logarithm problem on elliptic curves of trace one. volume 12, pages 193–196, 1999. doi: 10.1007/s001459900052.
- [63] Standards for Efficient Cryptography Group (SECG). SEC 1: Elliptic curve cryptography. Technical report, SECG, 2009. URL <https://www.secg.org/sec1-v2.pdf>. Version 2.0.
- [64] Standards for Efficient Cryptography Group (SECG). SEC 2: Recommended elliptic curve domain parameters. Technical report, SECG, 2010. URL <https://www.secg.org/sec2-v2.pdf>. Version 2.0.
- [65] Marc Stevens, Arjen K. Lenstra, and Benne de Weger. Chosen-prefix collisions for MD5 and the creation of a rogue CA certificate. In *Advances in Cryptology – EUROCRYPT 2009*, volume 5479 of *LNCS*, pages 55–72. Springer, 2009.
- [66] Thomas Stockinger. GSM network and its privacy – the A5 stream cipher. Technical report, nop.at, 2005. URL https://www.nop.at/gsm_a5/GSM_A5.pdf.
- [67] Joachim Strombergson and Simon Josefsson. Test vectors for the stream cipher RC4. Informational RFC 6229, IETF, 2011. URL <https://www.rfc-editor.org/rfc/rfc6229.html>.
- [68] The Chromium Project. A further update on SHA-1 certificates in Chrome. Chromium Security, 2016. URL <https://www.chromium.org/Home/chromium-security/education/tls/sha-1/>.
- [69] Xiaoyun Wang, Dengguo Feng, Xuejia Lai, and Hongbo Yu. Collisions for hash functions MD4, MD5, HAVAL-128 and RIPEMD. Cryptology ePrint Archive, Paper 2004/199, 2004. URL <https://eprint.iacr.org/2004/199>.
- [70] Xiaoyun Wang, Yiqun Yin, and Hongbo Yu. Finding collisions in the full SHA-1. In *Advances in Cryptology – CRYPTO 2005*, pages 17–36, 2005.
- [71] William C. Waterhouse. Abelian varieties over finite fields. *Annales scientifiques de l’École Normale Supérieure*, 2(4):521–560, 1969.

- [72] Pieter Wuille and Gregory Maxwell. secp256k1: A high-speed elliptic curve library. In *Proceedings of the 1st Bitcoin Core Developer Conference*, 2017. URL <https://github.com/bitcoin-core/secp256k1>.

Otro material

- Diversas consultas puntuales al sitio *StackOverflow*.
- Material docente de las asignaturas Álgebra Lineal y Estructuras Matemáticas, Metodología de la Programación, Aprendizaje Automático y Fundamentos de Redes impartidas en el Grado de Ingeniería Informática en la Universidad de Granada.

